Data Mining 2026 - Project Notebook

Course: Data Mining
Project track: ☑ Standard Analysis ☐ Research-Oriented
Group members:

- Tereza Knápková
- Mária Matušisková
- Dominik Kysel
- Dominik Ott

Dataset:

- Bike Sharing Dataset (https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset)

Initial task description (Module 1 perspective): RESEARCH QUESTIONS:

- RQ1: Which temporal and weather factors are most associated with hourly bike demand (`cnt`)?
  - Before (expectations):
    - We expect that factors like time (hr) and temperature might be high related to demand, similarly registered and casual users, suspect that features like humidity, weathersit and season might have mild association with demand.
  - After (our analysis):
    - **Top Associated Factors (by correlation strength):** `registered` (r=0.972) dominates the most, followed by `casual` (0.695), then `temp` (0.405), `atemp` (0.401), `hr` (0.394), `hum` (-0.323), `weathersit` (-0.142), and `season` (0.178).
      - Registered users contain ~80% of total demand.
    - **Key Temporal Patterns:**
      - two distinct daily peaks
        - 8:00 AM (morning commute)
        - 5-6 PM (evening commute)
      - high registered user concentration during working hours
      - consistent low demand overnight (12 AM-5 AM) across all seasons
    - **Seasonal Variation:**
      - summer shows highest peaks
      - winter the lowest
      - all seasons display same commute pattern, but with different intensity levels
    - **Weather Impact:**
      - warmer temperature (r=0.405) and lower humidity (r=-0.323) correlate with higher demand
      - adverse weather slightly reduces demand (r=-0.142)
    - **Conclusion:** Our preliminary expectations were confirmed and refined-registered users, time of day, and temperature are primary drivers; humidity and weather have weaker but meaningful negative associations.
- RQ2: Can clustering reveal distinct bike-usage regimes (e.g., commute, leisure, low-demand periods)?
  - Before (expectations):
    - We expected at least low and high demand regimes to be revealed.
  - After (our analysis):

        - **K-Means Clustering (k=4, silhouette=0.4661):** Four distinct hourly regimes identified:
            - **Highest-demand commute** (avg 514 rentals, 96% working-day, 91% registered) morning/evening peaks.
            - **High-demand mixed** (442 rentals, 12% working-day, 64% registered) leisure/weekend focused.
            - **Medium-demand** (175 rentals, mixed usage).
            - **Low-demand off-peak** (51 rentals, late night/early morning).
        - **DBSCAN Clustering (2 regimes + 5.1% noise):** Detected two dense usage regimes.
            - **High-demand regime** (peak evening commute, hr≈17.5, 823 avg rentals, silhouette=0.5335).
            - **Medium-high demand regime** (broader daytime, hr≈11.3, 166 avg rentals), ~5% sparse outliers.
        - **Seasonality Clustering (2 regimes, silhouette=0.4252):**
            - **High-demand regime** (warm/dry: temp≈0.63, hum≈0.56, 329 avg rentals).
            - **Low-demand regime** (cool/humid: temp≈0.41, hum≈0.67, 98 avg rentals).
        - **Conclusion:** Yes, clustering successfully revealed clear usage regimes. Main distinction is commute driven (working hours with registered users) vs. leisure/off-peak. Weather based clustering confirms temperature and humidity strongly modulate demand. Regimes are interpretable and stable across algorithms.
- RQ 3: Can anomaly detection methods identify periods of unexpectedly high or low bike rental demand that may correspond to external events or unusual conditions not explicitly captured in the dataset?
    - Before (expectations):
        - We expected only a small fraction of hourly records to look anomalous.
    - After
        - **Anomaly detection identified a small but meaningful set of unusual periods.** All three selected methods flagged `965` rows (`5.55%` of the dataset).
            - **Statistical outliers:** mainly detected globally extreme demand combinations, especially hours with unusually high `cnt` / `registered` values.
            - **Isolation Forest:** found a similar number of anomalies and had moderate agreement with the statistical baseline (`Jaccard = 0.537`), suggesting both methods capture broadly similar global anomalies.
            - **LOF:** also flagged `965` rows, but with very low overlap with the other methods (`0.106` with Statistical, `0.142` with Isolation Forest), meaning it captures more **local/contextual anomalies** rather than only globally extreme points.
        - **Overall:** Anomaly detection successfully identified unusual periods. These are best interpreted as candidate anomalous demand hours.
- RQ 4: Is there noise in the data? If so, what can be done with it?
    - Before (expectations):
        - We expected little data quality noise because the dataset is structured and well prepared, withouh NaN values, but we expected some sparse or irregular observations at the edges of normal demand regimes.
    - After
        - **DBSCAN identified `879` rows (`5.06%`) as noise**, meaning these points do not belong to the main dense behavioral regimes in the chosen feature space. The question is, are they related with outliers or not?
        - The noise is considered according to us as non-patttern data rather than error.
        - Some of these points may correspond to very low-demand hours, unusual combinations of demand and weather, or rare edge cases between normal regimes.

        - **What could we done with it?**
            - keep it and analyze it separately as irregular behavior
            - add a `noise` flag as an informative feature -> to filter it later
            - remove it
- RQ 5: How to decide which ouliers are valid. And how to compare them with the noise from clustering?
    - Before (expectations):
        - We expected that not every detected outlier would be equally trustworthy.
        - We also expected only partial overlap between anomaly-detection outliers and DBSCAN noise, because they represent different concepts.
    - After:
        - The rows found by both **Statistical Outllier** and **Isolation Forest** are great candidates for globally unusual demand periods.
        - **LOF outliers** are also valid candidates, but they should be interpreted differently: they are more likely **local anomalies**.
        - **Clustering noise is not the same as an outlier.**
            - **DBSCAN noise** -> not part of any dense cluster.
            - **Outlier detection** -> unusual according to a specific anomaly criterion

Initial task description (Module 2 perspective): RESEARCH QUESTIONS:

- RQ 6: How can a graph of `(weekday, hour)` patterns show repeated bike rentals behaviors?
    - Before (expectations):
        - We expected the graph to show repeated patterns more clearly than CP1.
        - The weekday × hour heatmaps showed that these groups follow stable weekly patterns.
    - After:
        - Louvain community detection identified meaningful groups such as **commuter peaks**, **weekend leisure periods**, **transitional daytime hours**, and **low-demand night periods**.
- RQ 7: Does graph analysis show more detail than K-Means from CP1?
    - Before (expectations):
        - We expected the graph to confirm the main CP1 patterns, like commute peaks and low-demand hours.
        - We also expected it to show more detail, because it connects time patterns instead of treating each row separately.
    - After:
        - Yes, the graph confirmed CP1 but also showed more detail.
        - The most similar output was with CP1 in night off-peak and peak commute patterns.
- RQ 8: Are CP1 outliers really anomalies, or are they part of normal graph groups?
    - Before (expectations):
        - We expected that some CP1 outliers might actually belong to meaningful groups once time structure was included.
    - After:
        - Many CP1 outliers were actually inside clear graph communities.
        - The highest outlier rates were in groups like evening commute peaks and weekend leisure.
        - This means that many points that looked extreme in CP1 are actually normal inside their own behavior group.
        - Hovewer, LOF behaved differently, which suggests that some points are only unusual globally, not locally.
- RQ 9: Do Louvain and Spectral Clustering give similar results?
    - Before (expectations):
        - We expected the two methods to differ in some edge cases, but still find the same similar patterns.

- After:
  - Yes, Louvain and Spectral Clustering gave very similar results, more than we expected.
  - Their differences were mostly in boundary hours, where patterns were mixed.
  - Both found the same main groups: night/off-peak, commute peaks, leisure, and transition periods.
  - ARI and NMI showed strong agreement between the two methods.

Initial task description (Module 3 perspective): RESEARCH QUESTIONS:

- RQ 10: Which combinations of time, weather, and daytype rules appear most frequently in the hourly bike-sharing data?
  - Before (expectations):
    - We expected that certain temporal periods (especially commute hours) and better weather conditions (clear/dry) would be the most common in the dataset's item combinations. We expected working days to be more frequent than weekends. We also expected temperature and humidity to be part of the discrete groups.
  - After:
    - **Most Frequent Itemsets** (min_support=0.02):
      - **day_type=workingday** (68.3% support) the most frequent item, reflecting the dataset's working day concentration.
      - **weathersit=Clear/Few clouds** (65.7% support) good weather conditions are most common.
      - **Temperature/Humidity/Windspeed** groups (~33.3% support each) due to tertile discretization.
      - **High_demand and Low_demand targets** (~25% support each) an expected result of 75th/25th percentile thresholds. The apriori algorithm found **36 valid rules** after filtering (requiring confidence ≥0.55, and lift ≥1.10).
- RQ 11: Which combinations of rules are most associated with high demand and low demand rental periods?
  - Before (expectations):
    - We expected that evening commute hours combined with warm temperatures and clear weather would predict high demand. For low demand, we anticipated overnight/early-morning hours combined with winter season and bad weather (rain/snow) as main predictors. We expected confidence and lift metrics would show these relationships.
  - After:
    - **High-Demand Rules (29 of 36 rules):**
      - **hour=evening_commute + temp=high → high_demand**: (support 6.3%, confidence 89.4%, lift 3.57) the strongest pattern, warm evenings result in higher demand.
      - **hour=evening_commute + season=summer → high_demand** (support 3.7%, confidence 86.5%, lift 3.45) similar to previous rule, evening and warmer season result in low demand..
      - **hour=evening_commute alone → high_demand** (support 10.9%, confidence 64.8%, lift 2.59) evening commute predicts high demand.
    - **Low-Demand Rules (7 of 36 rules):**
      - **hour=overnight + weathersit=Light Snow/Rain → low_demand** (support 1.7%, confidence 92.9%, lift 3.69) high confidence here with pattern showing that hour=overnight and bad weather result in low demand.
      - **hour=overnight + season=winter → low_demand** (support 5.4%, confidence 92.7%, lift 3.68) similar to previous rule, overnight and colder season result in low demand.

        - **hour=overnight alone → low_demand** shows that overnight hours dominate low demand rules.

    Note: Most rules include hour=evening_commute/hour=overnight, meaning time has a big impact on demand, but weather/season aplify the effect of the rule.

- RQ 12: Does pattern mining confirm the main findings from EDA, clustering, and graph mining, or does it reveal new demand regimes?
    - Before (expectations):
        - We expected pattern mining to confirm the four demand regimes identified by K-Means (highest demand commute, high demand mixed, medium demand, low demand off-peak) and the communities found by Louvain (commute peaks, weekend leisure, night off-peak, transitional hours). We expected that confidence/lift metrics would provide more context but suspected that no entirely new regimes would emerge.
    - After:
        - **Confirmation of CP1/CP2 Findings:** Pattern mining confirms findings from previous checkpoints.
        - **Evening commute regime** confirmed with 29 of 36 rules including hour=evening_commute, directly matching K-Means Cluster 0 ("highest demand commute") and Louvain's "commute peaks" community. Rules like `hour=evening_commute + temp=high → high_demand` (lift 3.57) show that conditioning on **both time AND weather creates better predictions** than hour alone (lift 2.59). DBSCAN's 5.06% **noise overlaps with itemsets** falling below the 2% support threshold, this shows that rare combinations like (evening_commute + extreme_cold + bad_weather) form no strong rules. **Some CP1 anomalies are actually frequent patterns**: a row with high demand + evening_commute + high_temp is not an outlier but a 6.3% support frequent pattern. **No new regimes** were discovered.

0. Reproducibility and Setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from ucimlrepo import fetch_ucirepo
from sklearn.cluster import KMeans, DBSCAN, SpectralClustering
from sklearn.neighbors import NearestNeighbors, LocalOutlierFactor
from matplotlib.colors import ListedColormap
from matplotlib.patches import Patch
# Clustering Evaluation Metrics - Required measures:
# 1. Contingency table, 2. Purity, 3. F-measure, 4. Maximum matching,
# 5. Conditional entropy, 6. Mutual information
from scipy.optimize import linear_sum_assignment
from scipy.stats import entropy as scipy_entropy
from sklearn.metrics import mutual_info_score, silhouette_score,
         adjusted_rand_score, normalized_mutual_info_score
# Visualize K-Means contingency tables with normalized values
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.lines import Line2D
from sklearn.ensemble import IsolationForest
import networkx as nx
from scipy.sparse import csr_matrix
```

```
from scipy.linalg import eigh
### Data Mining
# from string to one-hot encoding (True/False)
from mlxtend.preprocessing import TransactionEncoder
# apriori: find frequent itemsets and association rules
# association_rules: generate association rules from frequent itemsets
from mlxtend.frequent_patterns import apriori, association_rules
```

1. Dataset Description and Loading

1.1 Dataset Overview

- Source: https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset
- Number of instances: 17,379 hourly records
- Number of features / entities: 17 variables in the hourly table
- Missing values: 0 missing values in the loaded hourly table
- Basic statistics: summarized in the following output tables
- Location: the data was collected from the Capital Bikeshare system in **Washington, D.C.**
- Preprocessing: the dataset comes **already preprocessed** by the original authors - `temp`, `atemp`, `hum`, and `windspeed` are all min-max normalised to [0, 1] before we receive them (see column descriptions below for the original scales)

```
bike_sharing = fetch_ucirepo(id=275)

print(f"Dataset {bike_sharing.metadata['name']}")
print(f"Description: {bike_sharing.metadata['abstract']}")
print(f"Created {bike_sharing.metadata['year_of_dataset_creation']}")
# data
df = bike_sharing.data.original
df.head()
```

```
Dataset Bike Sharing
Description: This dataset contains the hourly and daily count of rental bikes
between years 2011 and 2012 in Capital bikeshare system with the
corresponding weather and seasonal information.
Created 2013
```

	instant	dteday	season	yr	mnth	hr	holiday	weekday	workingday
0	1	2011-01-01	1	0	1	0	0	6	0
1	2	2011-01-01	1	0	1	1	0	6	0
2	3	2011-01-01	1	0	1	2	0	6	0
3	4	2011-01-01	1	0	1	3	0	6	0
4	5	2011-01-01	1	0	1	4	0	6	0

```
print(f"Dataset shape: {df.shape}")
print(f"Dataset columns: {df.columns}")
```

```
print(f"missing values: {df.isnull().sum()}")

Dataset shape: (17379, 17)
Dataset columns: Index(['instant', 'dteday', 'season', 'yr', 'mnth', 'hr',
'holiday', 'weekday',
       'workingday', 'weathersit', 'temp', 'atemp', 'hum', 'windspeed',
       'casual', 'registered', 'cnt'],
      dtype='str')
missing values: instant       0
dteday        0
season        0
yr            0
mnth          0
hr            0
holiday       0
weekday       0
workingday    0
weathersit    0
temp          0
atemp         0
hum           0
windspeed     0
casual        0
registered    0
cnt           0
dtype: int64
```

There are 17 columns:

- **instant:** record index
- **dteday:** date
- **season:**
  - 1:winter
  - 2:spring
  - 3:summer
  - 4:fall
- **yr:** year (0: 2011, 1: 2012)
- **mnth:** month (1 to 12)
- **hr:** hour (0 to 23)
- **holiday:** whether the day is a holiday (from the Washington, D.C. holiday calendar)
- **weekday:** day of the week
- **workingday:** 1 if the day is neither weekend nor holiday, otherwise 0
- **weathersit:**
  - 1: Clear, Few clouds, Partly cloudy
  - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist
  - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds
  - 4: Heavy Rain + Ice Pellets + Thunderstorm + Mist, Snow + Fog
- **temp:** normalized temperature in Celsius, scaled from t_min = -8 to t_max = +39 (hourly)
- **atemp:** normalized feeling temperature in Celsius, scaled from t_min = -16 to t_max = +50 (hourly)
- **hum:** normalized humidity (divided by 100)
- **windspeed:** normalized wind speed (divided by 67)
- **casual:** count of casual users
- **registered:** count of registered users
- **cnt:** total count of rented bikes (casual + registered)

1.2 Exploratory Data Analysis

- Basic statistics
- Distributions
- Sparsity / density

This section follows the lecture focus on exploratory statistics, variable distributions, and identifying dense vs. sparse regions in feature space.

1.2.1 Basic Statistics (from the Lecture 1)

```
numeric_cols = df.select_dtypes(include=np.number).columns

# T here means transpose, which flips the rows and columns of the DataFrame
basic_stats_extended = df[numeric_cols].describe().T
# replace 50% with median, to make it readable
basic_stats_extended["median"] = df[numeric_cols].median()
basic_stats_extended["mode"] = df[numeric_cols].mode().iloc[0]
basic_stats_extended["iqr"] = basic_stats_extended["75%"] -
         basic_stats_extended["25%"]
basic_stats_extended["skew"] = df[numeric_cols].skew()

basic_stats_extended = basic_stats_extended[
    ["count", "mean", "std", "min", "25%", "median", "mode", "75%", "max",
         "iqr", "skew"]
].round(3)

print("Extended basic statistics (numeric features):")
basic_stats_extended
```

```
Extended basic statistics (numeric features):
```

	count	mean	std	min	25%	median	mode
instant	17379.0	8690.000	5017.029	1.00	4345.500	8690.000	1.000
season	17379.0	2.502	1.107	1.00	2.000	3.000	3.000
yr	17379.0	0.503	0.500	0.00	0.000	1.000	1.000
mnth	17379.0	6.538	3.439	1.00	4.000	7.000	5.000
hr	17379.0	11.547	6.914	0.00	6.000	12.000	16.000
holiday	17379.0	0.029	0.167	0.00	0.000	0.000	0.000
weekday	17379.0	3.004	2.006	0.00	1.000	3.000	6.000
workingday	17379.0	0.683	0.465	0.00	0.000	1.000	1.000
weathersit	17379.0	1.425	0.639	1.00	1.000	1.000	1.000
temp	17379.0	0.497	0.193	0.02	0.340	0.500	0.620
atemp	17379.0	0.476	0.172	0.00	0.333	0.485	0.621
hum	17379.0	0.627	0.193	0.00	0.480	0.630	0.880
windspeed	17379.0	0.190	0.122	0.00	0.104	0.194	0.000
casual	17379.0	35.676	49.305	0.00	4.000	17.000	0.000
registered	17379.0	153.787	151.357	0.00	34.000	115.000	4.000

	count	mean	std	min	25%	median	mode
cnt	17379.0	189.463	181.388	1.00	40.000	142.000	5.000

Where:

- **count** - number of non-null values in each column (here, 17,379 for the hourly table)
- **mean** - average value in each column
- **std** - standard deviation (how spread out the values are around the mean)
- **min** - smallest value in each column
- **25%** - value at the first quartile
- **median (50%)** - median value
- **mode** - the most frequent value (for ties, the first mode is shown)
- **75%** - value at the third quartile
- **max** - largest value in each column
- **iqr (75% - 25%)** - a robust spread measure, interquartile range
- **skew** - shape/asymmetry (positive = right tail, negative = left tail, near 0 = more symmetric)

Observations:

- **mean vs median** - are almost symmetric

According to rows:

- **instant:** row index, so nothing interesting here
- **season (1..4):** ***mean*** 2.502 (looks like seasons are balanced), ***skew*** near 0
- **yr (0=2011, 1=2012):** ***mean*** 0.503 (roughly 50/50 split between years)
- **mnth (1..12):** quartiles ***(25%, 50%. 75%)*** 4, 7, 10, ***skew*** near-zero, month coverage looks balanced
- **hr (0..23):** quartiles ***(25%, 50%. 75%)*** 6, 12, 18, ***skew*** near-zero, full-day coverage looks balanced
- **holiday (0/1):** ***mean*** 0.029 means only ~2.9% records are holidays, ***median***=0, ***75%***=0, ***iqr***=0, ***skew***=5.639 (mostly non-holiday).
- **weekday (0..6):** ***skew*** near-zero, week coverage looks balanced
- **workingday (0/1):** ***mean*** 0.683 means ~68.3% working-day hours, ***skew*** -0.785 which supports mean theory
- **weathersit (1..4):** ***mean*** 1.425, ***median*** 1, ***75%***=2, ***skew*** 1.228, severe weather is rare
- **temp (normalized 0..1):** ***mean*** 0.497, ***median*** 0.500, ***skew*** -0.006 -> near-symmetric
  - temp is normalised as `(t - t_min) / (t_max - t_min)` where `t_min = -8°C` and `t_max = 39°C`, a normalised value of 0.5 means the real average temperature is about **15.4°C** (roughly `-8 + 0.5 × 47`). apparently this is consistent with Washington D.C.'s climate ☀️
- **atemp (normalized):** ***skew*** slightly left-skewed (-0.090)
- **hum (normalized):** ***mean*** 0.627, ***median*** 0.630
- **windspeed (normalized):** ***mean*** 0.190, ***median*** 0.194, ***skew*** 0.575, mostly moderate wind with some high-wind tail
- **casual:** ***mean*** 35.676, ***median*** 17, ***skew***2.499 -> very right-skewed
- **registered:** ***mean*** 153.787, ***median*** 115, ***skew*** 1.558 0> right-skewed
- **cnt:** overall pattern is right-skewed

Covariance / Correlation

- summarization of pairwise covariance and correlation among numeric variables (matrices)

Covariance:

- **positive covariance:** when values (X, Y) is above its mean
- **negative covariance:** when X is above mean, Y tends to be below mean
- **near 0:** two variables do not show a clear straight-line relationship

Correlation:

- covariance normalized by both standard deviations (between -1 and +1)

```
cov_matrix = df[numeric_cols].cov()
corr_matrix = df[numeric_cols].corr()

print("Covariance matrix (numeric):")
display(cov_matrix.round(3))

print("Correlation matrix (numeric):")
display(corr_matrix.round(3))

# remove corr with itself, and sort by absolute value
corr_with_cnt = corr_matrix["cnt"].drop("cnt")
top_abs_with_cnt_corr =
         corr_with_cnt.reindex(corr_with_cnt.abs().sort_values(ascending=False).index)
print("Strongest correlations with cnt (absolute ranking):")
display(top_abs_with_cnt_corr.head(10).round(3))
```

```
Covariance matrix (numeric):
```

	instant	season	yr	mnth	hr	holiday
instant	2.517058e+07	2243.844	2172.443	8439.269	-165.637	12.348
season	2.243844e+03	1.225	-0.006	3.161	-0.047	-0.002
yr	2.172443e+03	-0.006	0.250	-0.018	-0.013	0.001
mnth	8.439269e+03	3.161	-0.018	11.825	-0.137	0.011
hr	-1.656370e+02	-0.047	-0.013	-0.137	47.809	0.001
holiday	1.234800e+01	-0.002	0.001	0.011	0.001	0.028
weekday	1.365400e+01	-0.005	-0.004	0.072	-0.049	-0.034
workingday	-7.976000e+00	0.007	-0.001	-0.006	0.007	-0.020
weathersit	-4.554100e+01	-0.010	-0.006	0.012	-0.089	-0.002
temp	1.315560e+02	0.067	0.004	0.134	0.183	-0.001
atemp	1.186480e+02	0.061	0.003	0.123	0.159	-0.001
hum	9.270000e+00	0.032	-0.008	0.109	-0.369	-0.000
windspeed	-4.573000e+01	-0.020	-0.001	-0.057	0.116	0.000
casual	3.915671e+04	6.560	3.520	11.607	102.684	0.260
registered	2.141754e+05	29.190	19.199	63.641	391.555	-1.198
cnt	2.533321e+05	35.750	22.719	75.248	494.239	-0.938

```
Correlation matrix (numeric):
```

	instant	season	yr	mnth	hr	holiday	weekday	wo
instant	1.000	0.404	0.866	0.489	-0.005	0.015	0.001	-0.(
season	0.404	1.000	-0.011	0.830	-0.006	-0.010	-0.002	0.0
yr	0.866	-0.011	1.000	-0.010	-0.004	0.007	-0.004	-0.(
mnth	0.489	0.830	-0.010	1.000	-0.006	0.018	0.010	-0.(
hr	-0.005	-0.006	-0.004	-0.006	1.000	0.000	-0.003	0.0
holiday	0.015	-0.010	0.007	0.018	0.000	1.000	-0.102	-0.2
weekday	0.001	-0.002	-0.004	0.010	-0.003	-0.102	1.000	0.0
workingday	-0.003	0.014	-0.002	-0.003	0.002	-0.252	0.036	1.0
weathersit	-0.014	-0.015	-0.019	0.005	-0.020	-0.017	0.003	0.0
temp	0.136	0.312	0.041	0.202	0.138	-0.027	-0.002	0.0
atemp	0.138	0.319	0.039	0.208	0.134	-0.031	-0.009	0.0
hum	0.010	0.151	-0.084	0.164	-0.276	-0.011	-0.037	0.0
windspeed	-0.075	-0.150	-0.009	-0.135	0.137	0.004	0.012	-0.(
casual	0.158	0.120	0.143	0.068	0.301	0.032	0.033	-0.3
registered	0.282	0.174	0.254	0.122	0.374	-0.047	0.022	0.1
cnt	0.278	0.178	0.250	0.121	0.394	-0.031	0.027	0.0

```
Strongest correlations with cnt (absolute ranking):

registered    0.972
casual        0.695
temp          0.405
atemp         0.401
hr            0.394
hum          -0.323
instant       0.278
yr            0.250
season        0.178
weathersit   -0.142
Name: cnt, dtype: float64
```

Observations:

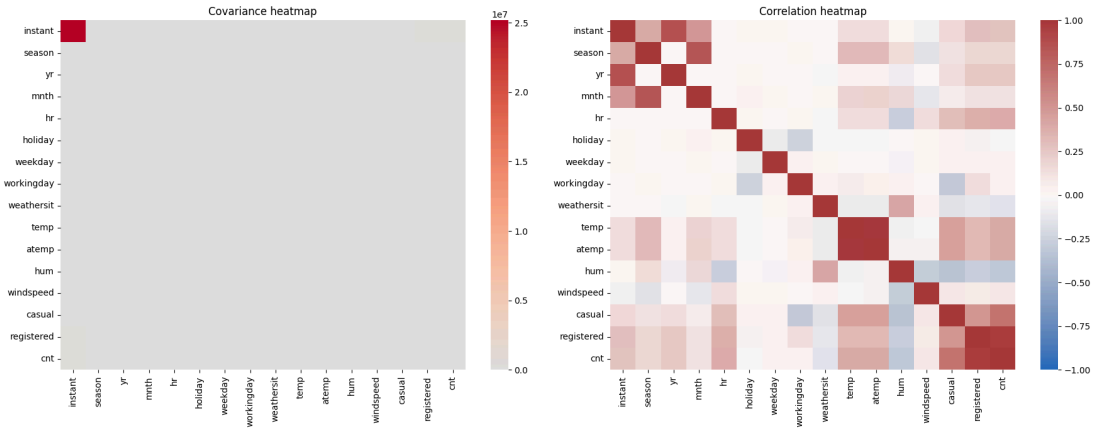
- **registered:** strong relationship, expected cause it is a part of *cnt*
- **casual:** strong relationship, expected cause it is a part of *cnt*
- **temp:** positive relationship (warmer conditions might be associated with more rentals)
- **atemp:** positive relationship (warmer conditions might be associated with more rentals)
- **hr:** the linear correlation with `cnt` is weak/moderate even though hour clearly matters for demand. this is because the relationship is **non-linear**: demand has two peaks (morning ~8 and evening ~17) and a valley (night ~2-5), so the pattern is roughly double-peaked across 0-23. Pearson correlation only measures linear association, so it underestimates the real dependence. this does not mean hour is unimportant, just that correlation is the wrong tool to measure it
- **hum:** negative relationship (higher humidity tends to be associated with fewer rentals)

- **instant:** index, so useless
- **yr:** weaker relationship (2012 has slightly more impact than 2011)
- **season:** weaker relationship (it is more cathegorical, so hard to say here)
- **weathersit:** weaker negative relationship (worse weather categories are linked with slightly lower interest)

Feedback question: Why does the hour of the day seem to have no impact?

- in our results, **hour of day does have a strong impact on demand**, the confusion might come from the fact that the **distribution of `hr` is balanced** across the 24 hours, so every hour is represented evenly in the dataset, but this only describes how the data is distributed, not how demand behaves
- when we analysed `cnt` by hour, we saw a clear **two peaks**: one around **8:00** and another around **17:00-18:00**

```
fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# center=0 -> color is centered at zero (positive vs negative shown clearly)
sns.heatmap(cov_matrix, cmap="coolwarm", center=0, ax=axes[0])
axes[0].set_title("Covariance heatmap")

# vmin=-1, vmax=1 -> fix the color scale to full correlation range
sns.heatmap(corr_matrix, cmap="vlag", center=0, vmin=-1, vmax=1, ax=axes[1])
axes[1].set_title("Correlation heatmap")

plt.tight_layout()
plt.show()
```



Observations:

- **Covariance Heatmap:**
  - mostly grey, might be because if *instant* feature, which has a very large scale
  - not useful plot for us
- **Correlation heatmap:**
  - since data are normalised, it's more useful than *Covariance Heatmap*
  - dark red on diagonal -> each variable with itself (1.0)
  - *temp* and *atemp* are strongly positively correlated (near duplicate information), whick makes sense, since they are very similar metrics
  - *cnt* is strongly positive with *registered* and *casual*, as expected
  - *cnt* is positive with *temp* and *hr*, this confirms our observation from the previous matrices section
  - *cnt* is negative with *hum* and slightly negative with worse *weathersit*

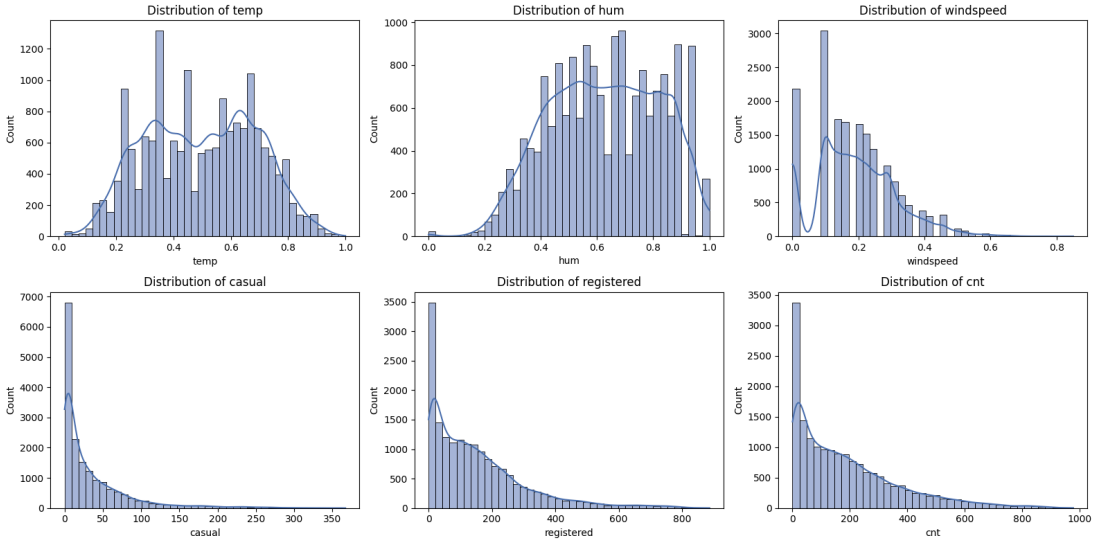
1.2.2 Distributions

We inspect numerical and categorical distributions because clustering and outlier behavior depend on where probability mass is concentrated.

Count Distribution (numerical)

- how many times the value in x-axis appears in the y-axis

```
dist_cols = ["temp", "hum", "windspeed", "casual", "registered", "cnt"]

fig, axes = plt.subplots(2, 3, figsize=(16, 8))
for ax, col in zip(axes.flatten(), dist_cols):
    sns.histplot(df[col], bins=40, kde=True, ax=ax, color="#4C72B0")
    ax.set_title(f"Distribution of {col}")
    ax.set_xlabel(col)

# makes plots readable and displays them
plt.tight_layout()
plt.show()
```



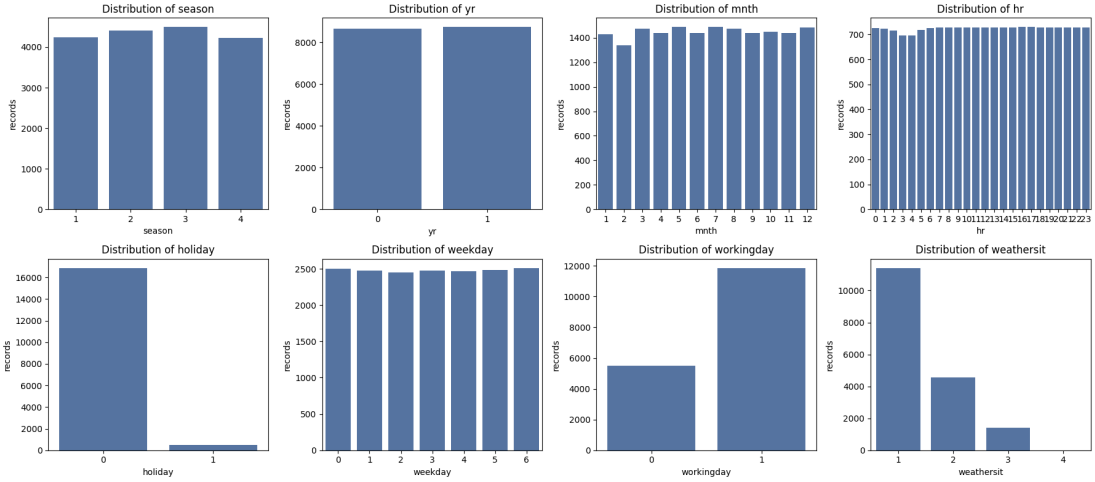
Observations:

- **temp:** not perfectly bell-shaped -> mixture of models (more peaks), could be because of seasons
- **hum:** most values are in the range 0.4–0.9 (medium-high humidity), low huminidy is more rare
- **windspeed:** long tail to high wind values, high wind conditions are uncommon?
- **casual:** low casual rentals, only few have very high casual rentals
- **registered:** similarly to ***casual***, but better
- **cnt:** shape close to ***registered***

Count Distribution (categorical)

- how many times the value in x-axis appears in the y-axis

```
category_cols = ["season", "yr", "mnth", "hr", "holiday", "weekday",
         "workingday", "weathersit"]

fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, col in zip(axes.flatten(), category_cols):
    counts = df[col].value_counts().sort_index()
    sns.barplot(x=counts.index, y=counts.values, ax=ax, color="#4C72B0")
    ax.set_title(f"Distribution of {col}")
    ax.set_xlabel(col)
    ax.set_ylabel("records")

plt.tight_layout()
plt.show()
```



Observations:

- **season:** balanced, 3 sligtly higher (summer)
- **yr:** balanced
- **mnth:** coverage is balanced
- **hr:** suprisingly balanced
- **holiday:** imbalanced, holidays are a small minority
- **weekday:** balanced
- **workingday:** imbalanced, weekdays\holidays are fewer
- **weathersit:** imbalanced, the weather is mostly good

1.2.3 Sparsity/Density

- **Sparsity:** many values are exactly 0 (or missing) in a feature
- **Density:** how concentrated records are in regions of feature space (many points close together)

In EDA we used only desriptive code, no clustering yet

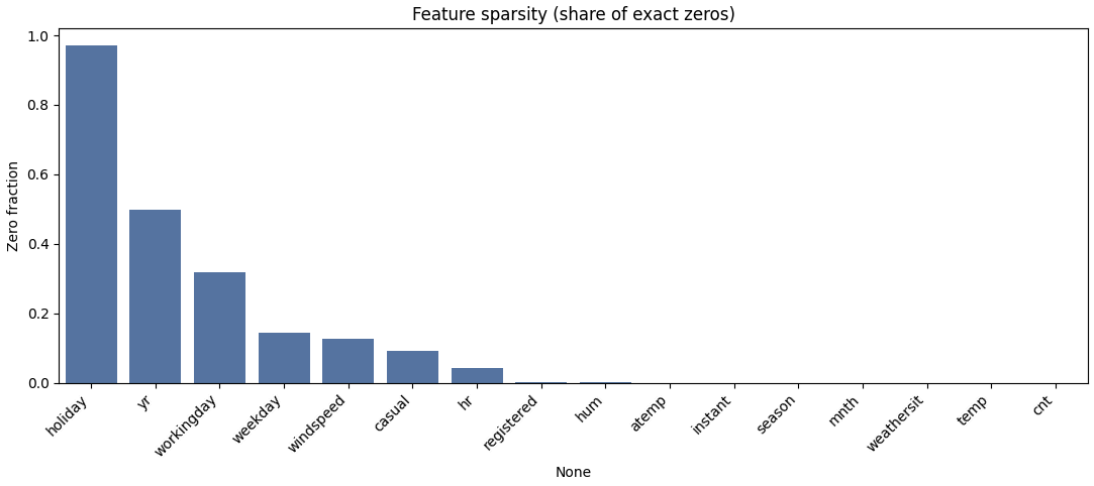
Sparsity

```
sparsity = pd.DataFrame({
    # nan
    "missing_fraction": df[numeric_cols].isna().mean(),
    # exactly zero
```

```
    "zero_fraction": (df[numeric_cols] == 0).mean(),
    # non-zero values
    "non_zero_fraction": (df[numeric_cols] != 0).mean()
    # puts most sparse by zeros features on top
}).sort_values("zero_fraction", ascending=False).round(3)

print("Sparsity summary (numeric features):")
display(sparsity)

plt.figure(figsize=(11, 5))
sns.barplot(x=sparsity.index, y=sparsity["zero_fraction"], color="#4C72B0")
plt.xticks(rotation=45, ha="right")
plt.ylabel("Zero fraction")
plt.title("Feature sparsity (share of exact zeros)")
plt.tight_layout()
plt.show()
```

```
Sparsity summary (numeric features):
```

	missing_fraction	zero_fraction	non_zero_fraction
holiday	0.0	0.971	0.029
yr	0.0	0.497	0.503
workingday	0.0	0.317	0.683
weekday	0.0	0.144	0.856
windspeed	0.0	0.125	0.875
casual	0.0	0.091	0.909
hr	0.0	0.042	0.958
registered	0.0	0.001	0.999
hum	0.0	0.001	0.999
atemp	0.0	0.000	1.000
instant	0.0	0.000	1.000
season	0.0	0.000	1.000
mnth	0.0	0.000	1.000
weathersit	0.0	0.000	1.000
temp	0.0	0.000	1.000
cnt	0.0	0.000	1.000



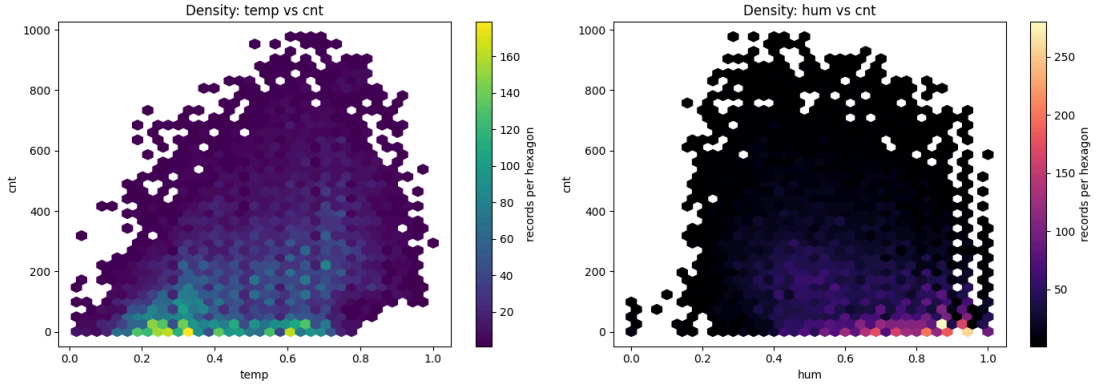
Observations:

- higher bar = more zeros = more sparse feature
  - **holiday:** means 97.1% of rows are non-holidays, only 2.9% are holidays
  - **yr:** means about half the rows are year 2011 and half year 2012 (roughly balanced)
  - **workingday:** means 31.7% non-working days, 68.3% working days
  - **weekday:** does not mean sparse in the usual way, it means category 0 appears 14.4% of the time
  - **hr:** similarly to *weekday*
  - **registered** and **hum:** near 0.001 mean almost never zero
- as we mentioned earlier, no missing values

Density

- let's see comparison of **temp**, **hum** and **cnt** since we saw correlation between them

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

hexagon_bin_1 = axes[0].hexbin(df["temp"], df["cnt"], gridsize=35,
         cmap="viridis", mincnt=1)
axes[0].set_title("Density: temp vs cnt")
axes[0].set_xlabel("temp")
axes[0].set_ylabel("cnt")
fig.colorbar(hexagon_bin_1, ax=axes[0], label="records per hexagon")

hexagon_bin_2 = axes[1].hexbin(df["hum"], df["cnt"], gridsize=35,
         cmap="magma", mincnt=1)
axes[1].set_title("Density: hum vs cnt")
axes[1].set_xlabel("hum")
axes[1].set_ylabel("cnt")
fig.colorbar(hexagon_bin_2, ax=axes[1], label="records per hexagon")

plt.tight_layout()
plt.show()
```



Observations:

- **temp vs cnt:**
  - as **temp** moves from cold to moderate, higher **cnt** becomes more possible
  - so warmer temperature = more rentals
- **hum vs cnt:**
  - high humidity (**hum** near 0.8–1.0) is mostly associated with low **cnt** (similarly with high temperature)
  - higher **cnt** values are more common from low to medium humidity (**hum**)
  - so less humidity = more rentals

1.3 Cleaning & Transformations

- Handling missing values
- Feature engineering
- Graph construction (if Checkpoint 2)

1.3.1 Handling missing values

The bike-sharing hourly data has no missing values.

1.3.2 Feature engineering: column cleanup

Drop non-informative columns:

- **instant:** sequential index with no analytical signal, as we also saw in EDA

Extract day from dteday and rename column:

- **dteday:** redundant information, as 'yr' and 'mnth' exist as seperate columns, but we need ***day*** as part of the day during our observation

```
df_clean = df.drop(columns=["instant"])

df_clean["dteday"] = pd.to_datetime(df_clean["dteday"]).dt.day
df_clean = df_clean.rename(columns={"dteday": "day"})

print(f"Columns after cleaning: {df_clean.columns.tolist()}")
print(f"Shape: {df_clean.shape}")
```

```
Columns after cleaning: ['day', 'season', 'yr', 'mnth', 'hr', 'holiday',
'weekday', 'workingday', 'weathersit', 'temp', 'atemp', 'hum', 'windspeed',
'casual', 'registered', 'cnt']
Shape: (17379, 16)
```

Validate data integrity for data consistency issues: casual users + registered users should equal total count

```
validation_error = (df_clean["casual"] + df_clean["registered"] -
        df_clean["cnt"]).abs().sum()

print(f"Data integrity check (should be 0): {validation_error}")

if validation_error == 0:
    print("Data are consistent: casual + registered = cnt")
else:
    print("Data are inconsistent!")
```

```
Data integrity check (should be 0): 0
Data are consistent: casual + registered = cnt
```

1.3.3 Feature engineering: categorical encoding

```
season_labels = {1: "winter", 2: "spring", 3: "summer", 4: "fall"}
year_labels = {0: "2011", 1: "2012"}
weekday_labels = {0: "Sunday", 1: "Monday", 2: "Tuesday", 3: "Wednesday", 4:
        "Thursday", 5: "Friday", 6: "Saturday"}
weathersit_labels = {1: "Clear/Few clouds", 2: "Mist/Cloudy", 3: "Light
        Snow/Rain", 4: "Heavy Rain/Snow"}

season_dummies = pd.get_dummies(df_clean["season"].map(season_labels),
        prefix="season")
year_dummies = pd.get_dummies(df_clean["yr"].map(year_labels), prefix="year")
weekday_dummies = pd.get_dummies(df_clean["weekday"].map(weekday_labels),
        prefix="weekday")
weathersit_dummies =
        pd.get_dummies(df_clean["weathersit"].map(weathersit_labels),
        prefix="weathersit")

df_encoded = pd.concat([
    df_clean.drop(columns=["season", "weathersit", "weekday", "yr"]),
    season_dummies,
    year_dummies,
    weekday_dummies,
    weathersit_dummies
], axis=1)

print(f"Columns after encoding: {df_encoded.columns.tolist()}")
print(f"Shape after encoding: {df_encoded.shape}")
print(f"\nData types:\n{df_encoded.dtypes}")
```

```
Columns after encoding: ['day', 'mnth', 'hr', 'holiday', 'workingday',
'temp', 'atemp', 'hum', 'windspeed', 'casual', 'registered', 'cnt',
'season_fall', 'season_spring', 'season_summer', 'season_winter',
'year_2011', 'year_2012', 'weekday_Friday', 'weekday_Monday',
'weekday_Saturday', 'weekday_Sunday', 'weekday_Thursday', 'weekday_Tuesday',
```

```
'weekday_Wednesday', 'weathersit_Clear/Few clouds', 'weathersit_Heavy
Rain/Snow', 'weathersit_Light Snow/Rain', 'weathersit_Mist/Cloudy']
Shape after encoding: (17379, 29)

Data types:
day                              int32
mnth                             int64
hr                               int64
holiday                          int64
workingday                       int64
temp                           float64
atemp                          float64
hum                            float64
windspeed                      float64
casual                           int64
registered                       int64
cnt                              int64
season_fall                       bool
season_spring                     bool
season_summer                     bool
season_winter                     bool
year_2011                         bool
year_2012                         bool
weekday_Friday                    bool
weekday_Monday                    bool
weekday_Saturday                  bool
weekday_Sunday                    bool
weekday_Thursday                  bool
weekday_Tuesday                   bool
weekday_Wednesday                 bool
weathersit_Clear/Few clouds       bool
weathersit_Heavy Rain/Snow        bool
weathersit_Light Snow/Rain        bool
weathersit_Mist/Cloudy            bool
dtype: object
```

2. Module 1 - Vector-Space Analysis

Standardize numerical features to zero mean and unit variance to make scales comparable across variables.

- This reduces dominance of large-scale variables in distance-based methods.
- The `hr` variable is kept unscaled in this version because time-of-day can be handled with cyclical encoding (`sin`/`cos`) in later steps.

```
scaler = StandardScaler()

numerical_cols = [
    "temp", "atemp", "hum", "windspeed",
    "cnt", "casual", "registered"
]

df_encoded[numerical_cols] = scaler.fit_transform(
    df_encoded[numerical_cols]
)
```

```
print("Scaling verification (should be ~0 mean, ~1 std):")
print(f"\nMeans:\n{df_encoded[numerical_cols].mean().round(4)}")
print(f"\nStd Devs:\n{df_encoded[numerical_cols].std().round(4)}")
print(f"\nNote: 'hr' column kept unscaled (range 0-23)")
```

```
Scaling verification (should be ~0 mean, ~1 std):

Means:
temp          0.0
atemp        -0.0
hum           0.0
windspeed     0.0
cnt          -0.0
casual        0.0
registered   -0.0
dtype: float64

Std Devs:
temp          1.0
atemp         1.0
hum           1.0
windspeed     1.0
cnt           1.0
casual        1.0
registered    1.0
dtype: float64

Note: 'hr' column kept unscaled (range 0-23)
```

Final Dataset Check

```
# Final cleaned and processed dataset
print("Dataset Overview:")
print(f"Shape: {df_encoded.shape}")
print(f"\nFirst 5 rows:")
print(df_encoded.head())
print(f"\nData Info:")
df_encoded.info()
print(f"\nBasic Statistics:")
print(df_encoded.describe().round(3))
```

```
Dataset Overview:
Shape: (17379, 29)

First 5 rows:
   day  mnth  hr  holiday  workingday      temp     atemp       hum  \
0    1     1   0        0           0 -1.334648 -1.093281  0.947372
1    1     1   1        0           0 -1.438516 -1.181732  0.895539
2    1     1   2        0           0 -1.438516 -1.181732  0.895539
3    1     1   3        0           0 -1.334648 -1.093281  0.636370
4    1     1   4        0           0 -1.334648 -1.093281  0.636370

   windspeed    casual  ...  weekday_Monday  weekday_Saturday  weekday_Sunday
\
0  -1.553889 -0.662755  ...           False              True           False
```

```
1  -1.553889 -0.561343  ...           False              True           False
2  -1.553889 -0.622190  ...           False              True           False
3  -1.553889 -0.662755  ...           False              True           False
4  -1.553889 -0.723603  ...           False              True           False

   weekday_Thursday  weekday_Tuesday  weekday_Wednesday  \
0             False            False              False
1             False            False              False
2             False            False              False
3             False            False              False
4             False            False              False

   weathersit_Clear/Few clouds  weathersit_Heavy Rain/Snow  \
0                         True                       False
1                         True                       False
2                         True                       False
3                         True                       False
4                         True                       False

   weathersit_Light Snow/Rain  weathersit_Mist/Cloudy
0                       False                   False
1                       False                   False
2                       False                   False
3                       False                   False
4                       False                   False

[5 rows x 29 columns]

Data Info:
<class 'pandas.DataFrame'>
RangeIndex: 17379 entries, 0 to 17378
Data columns (total 29 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   day                          17379 non-null  int32
 1   mnth                         17379 non-null  int64
 2   hr                           17379 non-null  int64
 3   holiday                      17379 non-null  int64
 4   workingday                   17379 non-null  int64
 5   temp                         17379 non-null  float64
 6   atemp                        17379 non-null  float64
 7   hum                          17379 non-null  float64
 8   windspeed                    17379 non-null  float64
 9   casual                       17379 non-null  float64
 10  registered                   17379 non-null  float64
 11  cnt                          17379 non-null  float64
 12  season_fall                  17379 non-null  bool
 13  season_spring                17379 non-null  bool
 14  season_summer                17379 non-null  bool
 15  season_winter                17379 non-null  bool
 16  year_2011                    17379 non-null  bool
 17  year_2012                    17379 non-null  bool
 18  weekday_Friday               17379 non-null  bool
 19  weekday_Monday               17379 non-null  bool
 20  weekday_Saturday             17379 non-null  bool
 21  weekday_Sunday               17379 non-null  bool
 22  weekday_Thursday             17379 non-null  bool
```

```
 23  weekday_Tuesday              17379 non-null  bool
 24  weekday_Wednesday            17379 non-null  bool
 25  weathersit_Clear/Few clouds  17379 non-null  bool
 26  weathersit_Heavy Rain/Snow   17379 non-null  bool
 27  weathersit_Light Snow/Rain   17379 non-null  bool
 28  weathersit_Mist/Cloudy       17379 non-null  bool
dtypes: bool(17), float64(7), int32(1), int64(4)
memory usage: 1.8 MB

Basic Statistics:
             day       mnth         hr    holiday  workingday       temp  \
count  17379.000  17379.000  17379.000  17379.000   17379.000  17379.000
mean      15.683      6.538     11.547      0.029       0.683      0.000
std        8.789      3.439      6.914      0.167       0.465      1.000
min        1.000      1.000      0.000      0.000       0.000     -2.477
25%        8.000      4.000      6.000      0.000       0.000     -0.815
50%       16.000      7.000     12.000      0.000       1.000      0.016
75%       23.000     10.000     18.000      0.000       1.000      0.847
max       31.000     12.000     23.000      1.000       1.000      2.612

            atemp        hum  windspeed     casual  registered        cnt
count  17379.000  17379.000  17379.000  17379.000   17379.000  17379.000
mean      -0.000      0.000      0.000      0.000      -0.000     -0.000
std        1.000      1.000      1.000      1.000       1.000      1.000
min       -2.769     -3.251     -1.554     -0.724      -1.016     -1.039
25%       -0.829     -0.763     -0.700     -0.642      -0.791     -0.824
50%        0.053      0.014      0.032     -0.379      -0.256     -0.262
75%        0.846      0.792      0.520      0.250       0.437      0.505
max        3.051      1.932      5.400      6.720       4.838      4.342
```

2.1 Vector Representation

Each hourly record is represented as a numeric feature vector after cleaning, encoding, and scaling.

- **Weather-impact vector (`X_weather`)** uses weather variables plus demand context (`cnt`) for exploratory structure analysis.
- **User-behavior vector (`X_users`)** uses hour with user counts (`casual`, `registered`) to capture usage composition over time.

Distance-based analysis in this module uses Euclidean distance, which is appropriate for standardized continuous features and aligns with K-Means assumptions.

For predictive modeling sections, `cnt` is treated as the target variable and is excluded from predictors to avoid leakage.

```python
# Weather Impact Vector:
X_weather = df_encoded[["temp", "atemp", "hum", "windspeed", "cnt"]].values

# User Segmentation Vector
X_users = df_encoded[["hr", "casual", "registered"]].values

print(f"X_weather shape: {X_weather.shape} - Weather impact on bike usage")
print(f"X_users shape: {X_users.shape} - User type segmentation by hour")
```

```
X_weather shape: (17379, 5) - Weather impact on bike usage
X_users shape: (17379, 3) - User type segmentation by hour
```

2.2 Clustering Method

This section applies baseline clustering methods to the constructed vectors and compares cluster quality.

Planned reporting items:

- chosen algorithm and parameters
- validation metric(s) (e.g., silhouette score)
- interpretation of discovered cluster profiles

2.2.1 K-Means

Commute pattern clusters (hourly behavior)

- Features: `hr`, `workingday`, `casual`, `registered`, `cnt`
- Candidate patterns: morning commute peak, evening commute peak, off-peak low demand, late-night minimal demand

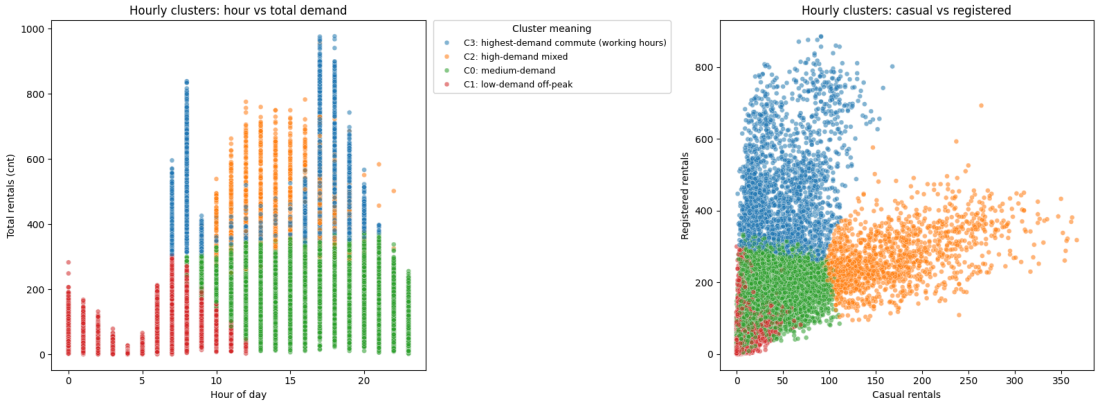
```
features  = ["hr", "casual", "registered"]
X = df_clean[features].values

scaler = StandardScaler()
X = scaler.fit_transform(X)

k_values  = range(2, 10)
silhouette_scores = []

for k in k_values:
    k_means = KMeans(n_clusters=k)
    labels = k_means.fit_predict(X)
    silhouette_scores.append(silhouette_score(X,labels))

best_k = k_values[np.argmax(silhouette_scores)]
print("Silhouette by k:", dict(zip(k_values, np.round(silhouette_scores,
         4))))
print("Best k:", best_k)


k_means_hourly = KMeans(n_clusters=best_k)
df_clean["cluster_hourly"] = k_means_hourly.fit_predict(X)

profile = (
    df_clean
    .groupby("cluster_hourly")[["hr", "casual", "registered", "cnt",
         "workingday", "holiday"]]
    .mean()
    .round(2)
    .sort_values("cnt", ascending=False)
)
```

```
print("\nHourly behavior cluster profiles:")
print(profile)
```

```
Silhouette by k: {2: np.float64(0.3986), 3: np.float64(0.4315), 4:
np.float64(0.4659), 5: np.float64(0.4518), 6: np.float64(0.4412), 7:
np.float64(0.4365), 8: np.float64(0.4547), 9: np.float64(0.3777)}
Best k: 4

Hourly behavior cluster profiles:
                    hr  casual  registered     cnt  workingday  holiday
cluster_hourly
3                13.93   51.03      463.32  514.34        0.96     0.01
2                14.29  170.05      271.83  441.88        0.12     0.06
0                16.74   32.58      141.95  174.53        0.73     0.03
1                 4.27    6.24       44.38   50.62        0.66     0.03
```

```
cluster_order = profile.index.tolist()
label_templates = [
    "highest-demand commute (working hours)",
    "high-demand mixed",
    "medium-demand",
    "low-demand off-peak",
    "very-low demand late-night",
]
cluster_name_map = {}
for rank, cid in enumerate(cluster_order):
    label = label_templates[rank] if rank < len(label_templates) else
        f"pattern-{rank+1}"
    cluster_name_map[cid] = f"C{cid}: {label}"

df_clean["cluster_hourly_name"] =
        df_clean["cluster_hourly"].map(cluster_name_map)
hue_order = [cluster_name_map[cid] for cid in cluster_order]
palette = dict(zip(hue_order, sns.color_palette("tab10",
        n_colors=len(hue_order))))

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.scatterplot(
    data=df_clean,
    x="hr", y="cnt",
    hue="cluster_hourly_name",
    hue_order=hue_order,
    palette=palette,
    alpha=0.55, s=24,
    ax=axes[0]
)
axes[0].set_title("Hourly clusters: hour vs total demand")
axes[0].set_xlabel("Hour of day")
axes[0].set_ylabel("Total rentals (cnt)")

leg = axes[0].legend(title="Cluster meaning", bbox_to_anchor=(1.02, 1),
        loc="upper left", borderaxespad=0)
for text in leg.get_texts():
    text.set_fontsize(9)
```

```
sns.scatterplot(
    data=df_clean,
    x="casual", y="registered",
    hue="cluster_hourly_name",
    hue_order=hue_order,
    palette=palette,
    alpha=0.55, s=24,
    ax=axes[1],
    legend=False
)
axes[1].set_title("Hourly clusters: casual vs registered")
axes[1].set_xlabel("Casual rentals")
axes[1].set_ylabel("Registered rentals")

plt.tight_layout()
plt.show()
```



Observations (based on current output):

- best cluster count is **k = 4** with the highest silhouette score (**0.4661**), so 4 groups provide the best separation among tested values
- **C1: highest-demand commute (working hours):** has the highest mean demand (`cnt = 514.34`) and is more focused on registered (`registered = 463.32`, `casual = 51.03`), with very high working-day ratio (`workingday = 0.96`)
- **C2: high-demand mixed:** is the second-highest demand group (`cnt = 441.96`) and has much higher casual rentals (`casual = 170.10`) than C1, with lower working-day ratio (`workingday = 0.12`), suggesting stronger leisure contribution
- **C0: medium-demand** has moderate totals (`cnt = 174.55`) with registered still larger than casual (`141.97` vs `32.58`)
- **C3: low-demand off-peak** has the lowest demand (`cnt = 50.62`) and very low user counts (`casual = 6.24`, `registered = 44.38`), matching low-activity hours
- in the `casual` vs `registered` scatter, C1 and C2 are separated very clearly: C1 is high registered and C2 is high casual, while C3 and C0 stay near the origin
- **note on labelling:** the regime labels (e.g. "highest-demand commute", "low-demand off-peak") are **not predefined categories from the data**. K-Means produces unlabelled cluster IDs (0, 1, 2, 3), and we assigned descriptive names manually by inspecting each cluster's centroid values (mean `cnt`, mean `hr`, `workingday` ratio, `casual` vs `registered` balance). so the labels are post-hoc interpretations of what each cluster looks like, not ground truth labels

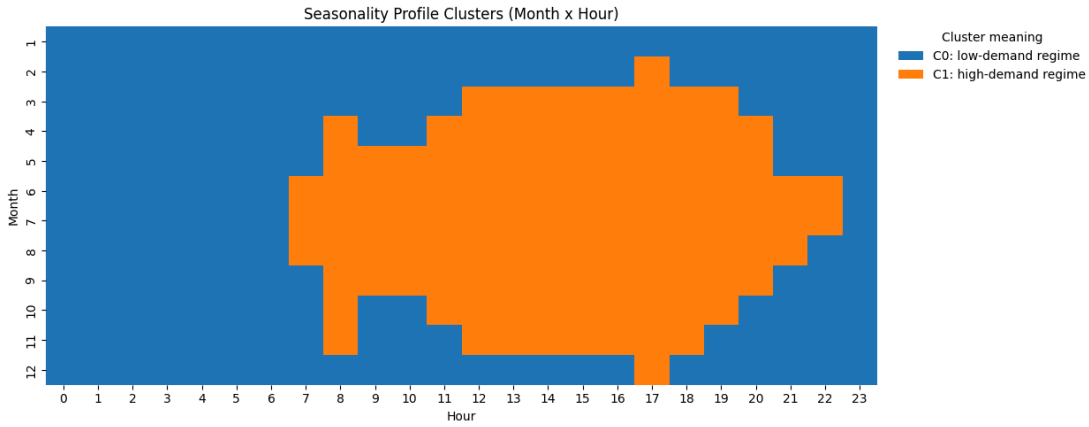
Hourly demand composition (casual vs registered)

This section summarizes average demand by hour and season, then compares how `casual` and `registered` usage patterns differ over the day.

```
seasonality = (
    df_clean
    .groupby(["mnth", "hr"], as_index=False)
    .agg(
        cnt_mean=("cnt", "mean"),
        casual_mean=("casual", "mean"),
        registered_mean=("registered", "mean"),
        temp_mean=("temp", "mean"),
        hum_mean=("hum", "mean"),
    )
)
features = ["cnt_mean", "casual_mean", "registered_mean", "temp_mean",
         "hum_mean"]
X_season = StandardScaler().fit_transform(seasonality[features])

# Choose k by silhouette
k_values = range(2, 8)
sil_scores = []
for k in k_values:
    labels = KMeans(n_clusters=k, random_state=42,
         n_init=20).fit_predict(X_season)
    sil_scores.append(silhouette_score(X_season, labels))

best_k = k_values[int(np.argmax(sil_scores))]
print("Silhouette by k:", dict(zip(k_values, np.round(sil_scores, 4))))
print("Best k:", best_k)

# Fit final model
km_season = KMeans(n_clusters=best_k, random_state=42, n_init=20)
seasonality["season_cluster"] = km_season.fit_predict(X_season)

# Cluster profiles
cluster_profile = (
    seasonality
    .groupby("season_cluster")[features]
    .mean()
    .round(2)
    .sort_values("cnt_mean", ascending=False)
)
print("\nSeasonality cluster profiles:")
print(cluster_profile)

# Build readable cluster labels by demand rank
ranked_ids = cluster_profile.index.tolist()
if len(ranked_ids) == 2:
    demand_labels = ["high-demand regime", "low-demand regime"]
else:
    demand_labels = [
        "high-demand regime",
        "medium-high demand regime",
```

```
        "medium-demand regime",
        "medium-low demand regime",
        "low-demand regime",
    ]

cluster_meaning = {}
for rank, cid in enumerate(ranked_ids):
    label = demand_labels[rank] if rank < len(demand_labels) else f"pattern-
        {rank+1}"
    cluster_meaning[cid] = label

print("\nCluster ID legend:")
for cid in sorted(cluster_meaning):
    print(f"C{cid}: {cluster_meaning[cid]}")

# Month x Hour heatmap with clean, discrete legend
pivot_clusters = (
    seasonality
    .pivot(index="mnth", columns="hr", values="season_cluster")
    .sort_index()
)

cluster_ids = sorted(seasonality["season_cluster"].unique())
id_to_idx = {cid: i for i, cid in enumerate(cluster_ids)}
pivot_idx = pivot_clusters.replace(id_to_idx)

palette = sns.color_palette("tab10", n_colors=len(cluster_ids))
cmap = ListedColormap(palette)

fig, ax = plt.subplots(figsize=(15, 5))
sns.heatmap(
    pivot_idx,
    cmap=cmap,
    cbar=False,
    ax=ax
)
ax.set_title("Seasonality Profile Clusters (Month x Hour)")
ax.set_xlabel("Hour")
ax.set_ylabel("Month")

legend_handles = [
    Patch(facecolor=palette[id_to_idx[cid]], edgecolor="none",
        label=f"C{cid}: {cluster_meaning[cid]}")
    for cid in cluster_ids
]
ax.legend(
    handles=legend_handles,
    title="Cluster meaning",
    bbox_to_anchor=(1.02, 1),
    loc="upper left",
    borderaxespad=0,
    frameon=False
)
```

```
plt.tight_layout(rect=[0, 0, 0.84, 1])
plt.show()

Silhouette by k: {2: np.float64(0.4252), 3: np.float64(0.321), 4:
np.float64(0.3235), 5: np.float64(0.3373), 6: np.float64(0.3587), 7:
np.float64(0.3587)}
Best k: 2

Seasonality cluster profiles:
                cnt_mean  casual_mean  registered_mean  temp_mean  hum_mean
season_cluster
1                 329.24        71.53           257.70       0.63      0.56
0                  98.33        12.34            85.99       0.41      0.67

Cluster ID legend:
C0: low-demand regime
C1: high-demand regime
```

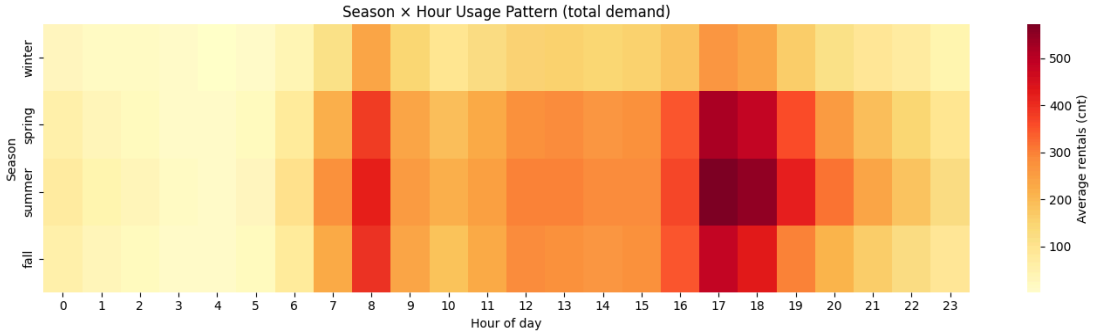


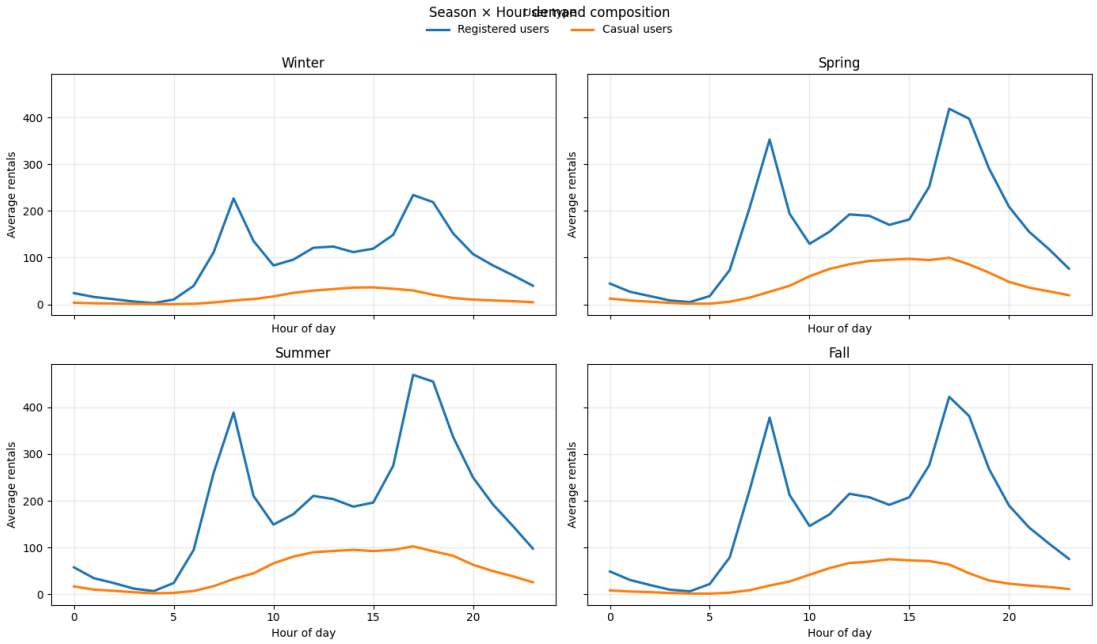
Observations (clustering according to seasons):

- the silhouette scan selects **k = 2** (`0.4252`), so the data separates best into two clusters
- **Cluster 1 (high-demand regime)**: `cnt_mean ≈ 329.24`, where `casual_mean` (`71.53`) and `registered_mean` (`257.70`), plus **warmer and drier** conditions (`temp_mean ≈ 0.63`, `hum_mean ≈ 0.56`)
- **Cluster 0 (low-demand regime)**: `cnt_mean ≈ 98.33`, with much lower user counts (`casual_mean ≈ 12.34`, `registered_mean ≈ 85.99`), and **cooler / more humid** conditions (`temp_mean ≈ 0.41`, `hum_mean ≈ 0.67`)
- in the Month × Hour heatmap, the high-demand cluster appears mostly during **daytime and early evening**, while nights or early hours remain in the low-demand cluster

```
# Season x Hour usage patterns (real demand)
season_map = season_labels
season_order = ["winter", "spring", "summer", "fall"]

season_hour = (
    df_clean.assign(season_name=df_clean["season"].map(season_map))
    .groupby(["season_name", "hr"], as_index=False)
    .agg(
        cnt_mean=("cnt", "mean"),
        casual_mean=("casual", "mean"),
        registered_mean=("registered", "mean"),
```

```
    )
)

# Heatmap: total usage by season and hour
pivot_cnt = season_hour.pivot(index="season_name", columns="hr",
          values="cnt_mean").reindex(season_order)

plt.figure(figsize=(14, 4))
sns.heatmap(
    pivot_cnt,
    cmap="YlOrRd",
    cbar_kws={"label": "Average rentals (cnt)"}
)
plt.title("Season × Hour Usage Pattern (total demand)")
plt.xlabel("Hour of day")
plt.ylabel("Season")
plt.tight_layout()
plt.show()

# Line plots: casual vs registered by hour for each season
reg_color = "#1f77b4"
cas_color = "#ff7f0e"

fig, axes = plt.subplots(2, 2, figsize=(14, 8), sharex=True, sharey=True)
axes = axes.flatten()

for i, season_name in enumerate(season_order):
    d = season_hour[season_hour["season_name"] == season_name]
    axes[i].plot(d["hr"], d["registered_mean"], label="Registered users",
         color=reg_color, linewidth=2.2)
    axes[i].plot(d["hr"], d["casual_mean"], label="Casual users",
         color=cas_color, linewidth=2.2)
    axes[i].set_title(season_name.capitalize())
    axes[i].set_xlabel("Hour of day")
    axes[i].set_ylabel("Average rentals")
    axes[i].grid(alpha=0.25)

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, title="User type", loc="upper center", ncol=2,
          frameon=False)
fig.suptitle(f'Season × Hour demand composition', y=1.02)
plt.tight_layout()
plt.show()
```





Observations (based on the Season × Hour outputs):

- the heatmap shows a clear **two peak daily pattern** in all seasons:
  - morning peak around **08:00**
  - evening peak around **17:00-18:00**.
- **Summer** has the highest overall demand of **registered users** (blue line), especially in late afternoon and early evening, strongest **casual users** activity in summer (and spring)
- **Spring** and **Fall** are also high for **registered users** category, strongest casual **user** activity in spring (and summer), falls also relatively high
- **Winter** is consistently lower for **registered users** category, for **casual users** much lower values in winter
- the line plots confirm that **registered users dominate total demand** in every season and largely define the commute peaks compared to **casual users**
- **casual users note:** overnight and early-morning hours (**00:00-05:00**) remain low-demand for both user groups across all seasons

2.2.2 DBSCAN (density-based clustering)

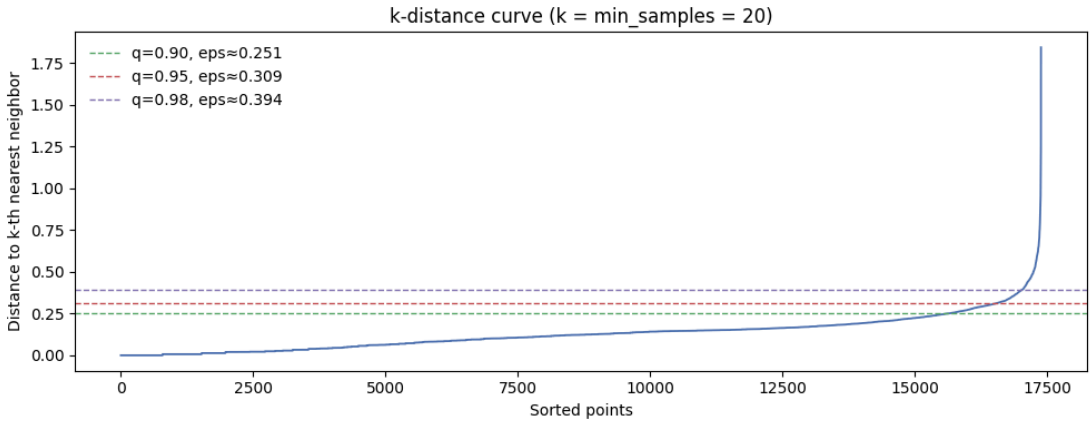
We used another baseline -> **DBSCAN** to detect dense usage regimes and naturally mark sparse points as noise (`label = -1`). Following Week 3 Exercise, we tuned `eps` from the k-distance curve and evaluate cluster/noise behavior.

```
# I hope the explanation helps, sorry if it is annoying :D

features  = ["hr", "casual", "registered"]
X = df_clean[features].to_numpy(dtype=float)
# we need to standardize for distance-based clustering (if too bug distance =
        troubles)
X = StandardScaler().fit_transform(X)


# 1) k-distance diagnostic for eps selection
# neighbours (big dataset so 20 neighbours at least?)
db_min_samples = 20
```

```
# to find the closest 20 neighbors for each point
nearest_neigh = NearestNeighbors(n_neighbors=db_min_samples)
# to know where to search for neighbors (reference)
nearest_neigh.fit(X)
# get distances to the 20 nearest neighbors for each point
distances, _ = nearest_neigh.kneighbors(X)
# sort the distances to the k-th nearest neighbor for each point
k_distances = np.sort(distances[:, -1])

# to create candidate eps values from k-distance quantiles
# creates 12 quantiles between 90% and 99.5%
grid_df = np.linspace(0.90, 0.995, 12)
eps_candidates = np.unique(np.round(np.quantile(k_distances, grid_df), 3))

print("DBSCAN setup:")
print(f"features={features}")
print(f"min_samples={db_min_samples}")
print(f"eps candidates (from k-distance quantiles):
         {eps_candidates.tolist()}")

# visualize k-distance curve with quantile thresholds
plt.figure(figsize=(10, 4))
plt.plot(k_distances, color="#4C72B0", linewidth=1.3)
for quantile, color in [(0.90, "#55A868"), (0.95, "#C44E52"), (0.98,
         "#8172B2")]:
    # actual k-distance value at quantile
    eps_quantile = np.quantile(k_distances, quantile)
    plt.axhline(eps_quantile, linestyle="--", linewidth=1, color=color,
         label=f"q={quantile:.2f}, eps≈{eps_quantile:.3f}")
plt.title(f"k-distance curve (k = min_samples = {db_min_samples})")
plt.xlabel("Sorted points")
plt.ylabel("Distance to k-th nearest neighbor")
plt.legend(frameon=False)
plt.tight_layout()
plt.show()

# 2) evaluate each eps candidate
rows = []
for eps in eps_candidates:
    # output cluster IDs for each point (-1 means noise)
    labels = DBSCAN(eps=float(eps),
         min_samples=db_min_samples).fit_predict(X)
    # number of clusters (excluding noise)
    num_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    noise_ratio = (labels == -1).mean()
    non_noise = labels != -1
    if num_clusters >= 2 and non_noise.sum() > db_min_samples:
        # clustering-quality measure for non-noise points (silhouette can be
         negative if clusters are not well separated)
        silhouette = silhouette_score(X[non_noise], labels[non_noise])
    else:
        silhouette = np.nan
    rows.append({
        "eps": float(eps),
        "min_samples": db_min_samples,
```

```
        "n_clusters": int(num_clusters),
        "noise_ratio": float(noise_ratio),
        "silhouette_non_noise": silhouette
    })

# summarize results in a DataFrame and sort by silhouette (desc) and noise
        ratio (asc)
dbscan_search = pd.DataFrame(rows).sort_values(["silhouette_non_noise",
        "noise_ratio"], ascending=[False, True])
print("DBSCAN candidate summary:")
display(dbscan_search.round(4))

# 3) select best params: prioritize silhouette, then noise ratio, then eps
valid = dbscan_search.dropna(subset=["silhouette_non_noise"])
if not valid.empty:
    db_best = valid.iloc[0]
else:
    fallback = dbscan_search[dbscan_search["n_clusters"] > 0]
    db_best = fallback.sort_values(["noise_ratio", "eps"], ascending=[True,
        True]).iloc[0]

# extract best params for final model
db_best_eps = float(db_best["eps"])
db_best_min_samples = int(db_best["min_samples"])
print(f"Selected DBSCAN params: eps={db_best_eps}, min_samples=
        {db_best_min_samples}")
```

```
DBSCAN setup:
features=['hr', 'casual', 'registered']
min_samples=20
eps candidates (from k-distance quantiles): [0.251, 0.259, 0.268, 0.28,
0.291, 0.301, 0.312, 0.326, 0.353, 0.387, 0.446, 0.568]
```



DBSCAN candidate summary:

	eps	min_samples	n_clusters	noise_ratio	silhouette_non_noise
0	0.251	20	2	0.0506	0.5335
6	0.312	20	2	0.0179	0.4881
2	0.268	20	2	0.0412	0.4742
3	0.280	20	3	0.0357	0.4415

	eps	min_samples	n_clusters	noise_ratio	silhouette_non_noise
4	0.291	20	3	0.0292	0.4395
11	0.568	20	1	0.0009	NaN
10	0.446	20	1	0.0035	NaN
9	0.387	20	1	0.0065	NaN
8	0.353	20	1	0.0102	NaN
7	0.326	20	1	0.0137	NaN
5	0.301	20	1	0.0237	NaN
1	0.259	20	1	0.0467	NaN

```
Selected DBSCAN params: eps=0.251, min_samples=20
```

Observations:

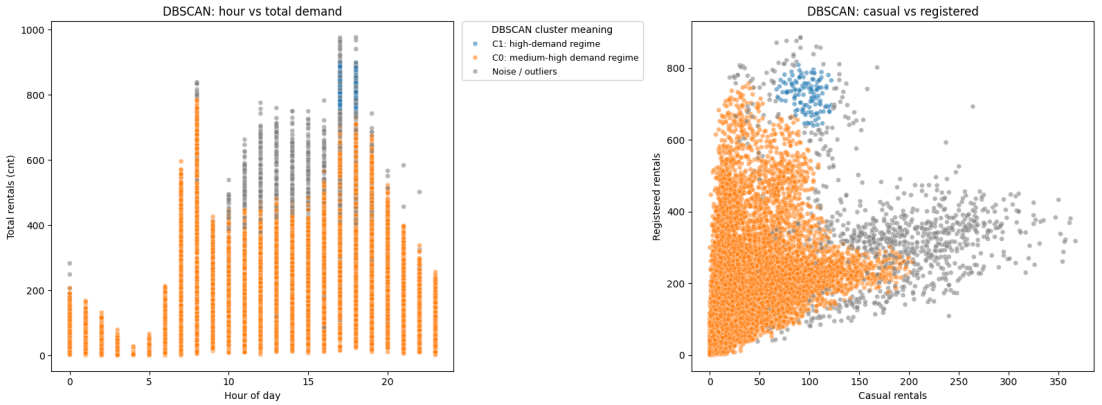
- the k-distance curve stays relatively flat for most points and rises near the right tail, which indicates a dense core plus a smaller sparse boundary (noise) region
- with `min_samples = 20`, the tested `eps` range was `0.251` to `0.568` (from high k-distance quantiles (from 90%))
- the best candidate was **`eps = 0.251`** with **2 clusters**, **noise ratio = 0.0506** (~5.1%), and the highest non-noise silhouette (**0.5335**)
- nearby values (`eps = 0.312`, `0.268`) also produced 2 clusters but with lower silhouette (`0.4881`, `0.4742`)
- mid values (`eps = 0.280`, `0.291`) produced 3 clusters but weaker separation (silhouette around `0.44`)
- large `eps` values (`>= 0.326`) collapsed the data into **1 cluster** (silhouette becomes `NaN`), so they are not useful for cluster structure discovery here

```
# 1) fit final DBSCAN with selected best params
# create DBSCAN instance with best parameters
dbscan_final = DBSCAN(eps=db_best_eps, min_samples=db_best_min_samples)
# run it on X to get final cluster labels
df_clean["cluster_dbscan"] = dbscan_final.fit_predict(X)

# 2) extract labels and compute summary statistics
labels_db = df_clean["cluster_dbscan"].to_numpy()
# number of clusters (excluding noise)
num_clusters_db = len(set(labels_db)) - (1 if -1 in labels_db else 0)
noise_ratio_db = (labels_db == -1).mean()
non_noise_mask = labels_db != -1

# 3) compute silhouette for non-noise points if we have at least 2 clusters
        and enough non-noise points
if num_clusters_db >= 2 and non_noise_mask.sum() > db_best_min_samples:
    db_silhouette = silhouette_score(X[non_noise_mask],
        labels_db[non_noise_mask])
else:
    db_silhouette = np.nan

print(f"DBSCAN clusters (excluding noise): {num_clusters_db}")
print(f"Noise fraction: {noise_ratio_db:.4f}")
```

```
print(f"Silhouette (non-noise only): {db_silhouette if
        pd.notna(db_silhouette) else "N/A"}")

# 4) builds a cluster profile table (to interpret what each cluster
        represents)
cluster_profile = (
    df_clean[df_clean["cluster_dbscan"] != -1]
    .groupby("cluster_dbscan")[["hr", "casual", "registered", "cnt",
        "workingday", "holiday"]]
    .mean()
    .round(2)
    .sort_values("cnt", ascending=False)
)

print("\nDBSCAN cluster profiles (excluding noise):")
display(cluster_profile)

# 5) create readable names for clusters (legends)
ranked_ids = cluster_profile.index.tolist()
demand_labels = [
        "high-demand regime",
        "medium-high demand regime",
        "medium-demand regime",
        "medium-low demand regime",
        "low-demand regime",
    ]
db_name_map = {-1: "Noise / outliers"}
for rank, cid in enumerate(ranked_ids):
    label = demand_labels[rank] if rank < len(demand_labels) else f"dense-
        pattern-{rank+1}"
    db_name_map[cid] = f"C{cid}: {label}"

# add readable cluster names to the DataFrame for visualization (noise gets
        its own label)
df_clean["cluster_dbscan_name"] =
        df_clean["cluster_dbscan"].map(db_name_map).fillna("Unlabeled")

# build hue order for consistent coloring in plots (ranked by demand, with
        noise last)
hue_order = [db_name_map[cid] for cid in ranked_ids]
if (labels_db == -1).any():
    hue_order = hue_order + ["Noise / outliers"]

cluster_colors = sns.color_palette("tab10", n_colors=max(len(ranked_ids), 1))
palette = {db_name_map[cid]: cluster_colors[i] for i, cid in
        enumerate(ranked_ids)}
if (labels_db == -1).any():
    palette["Noise / outliers"] = "#7f7f7f"

# 6) visualize clusters with readable legends
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.scatterplot(
    data=df_clean, x="hr", y="cnt",
    hue="cluster_dbscan_name",
    hue_order=hue_order,
```

```
    palette=palette,
    alpha=0.55, s=24, ax=axes[0]
)
axes[0].set_title("DBSCAN: hour vs total demand")
axes[0].set_xlabel("Hour of day")
axes[0].set_ylabel("Total rentals (cnt)")
leg = axes[0].legend(title="DBSCAN cluster meaning", bbox_to_anchor=(1.02,
        1), loc="upper left", borderaxespad=0)
for text in leg.get_texts():
    text.set_fontsize(9)

sns.scatterplot(
    data=df_clean, x="casual", y="registered",
    hue="cluster_dbscan_name",
    hue_order=hue_order,
    palette=palette,
    alpha=0.55, s=24, ax=axes[1], legend=False
)
axes[1].set_title("DBSCAN: casual vs registered")
axes[1].set_xlabel("Casual rentals")
axes[1].set_ylabel("Registered rentals")

plt.tight_layout()
plt.show()

DBSCAN clusters (excluding noise): 2
Noise fraction: 0.0506
Silhouette (non-noise only): 0.5334736000479774

DBSCAN cluster profiles (excluding noise):
```

	hr	casual	registered	cnt	workingday	holiday
cluster_dbscan						
1	17.48	97.01	726.47	823.48	1.00	0.00
0	11.33	27.38	138.61	165.99	0.71	0.03



Observations (final DBSCAN output):

- final DBSCAN with chosen parameteres found **2 dense clusters** (excluding noise), with **noise fraction = 0.0506** (~5.1%) and **silhouette = 0.5335** on non-noise points,

which indicates reasonably good separation
- **C1: high-demand regime:** is comune (working hours): `hr ≈ 17.48`, very high `registered ≈ 726.47`, `casual ≈ 97.01`, and very high total demand `cnt ≈ 823.48`
- **C0: medium-high demand regime:** is wider and less extreme: `hr ≈ 11.33`, `registered ≈ 138.61`, `casual ≈ 27.38`, `cnt ≈ 165.99`, with mixed day type (`workingday ≈ 0.71`)
- in the comparison of the `casual vs registered` plot, **C1** occupies the high-registered corner, **C0** occupies the main dense body, and gray points labeled **Noise / outliers** are sparse/extreme profiles outside dense regions

Overall

- this only confirms the pattern we analysed earlier, that **registered users** in the high-demand regime (working hours) the registered users are in the peak and for the **casual users** it is more wider regime (24 hours)
- about 5% (0.0506 * 100 = 5.06%) of records are sparse (noise or unusual patterns) rather than belonging to stable dense regimes

2.3 Outlier Detection

- **Statistical outliers** (some of it already used in EDA)
  - mean of each fearure
  - standard deviation of each feature
  - z-score
- **Local Outlier Factor (LOF)**
  - local density-based outlier method
  - in simple words, the point is suspicious if its position in a region is much less dense than the region of its neighbors
  - steps
    - choose *k* neigbours
    - find the neighbours for each point
    - estimate how dense the point's neighborhood is
    - compare that density to the densities of its neighbors
- **Isolation Forest**
  - three method, so how easy is it to isolate this point by random splits?
    - the smallet the height, the more anomalous data
  - dense points are hard to isolate, only sparse is easy
  - steps
    - build many random trees
    - in each tree, randomly choose a feature and split value
    - how many splits are needed before a point is isolated?
    - average this over many trees

Statistical Outliers

```
# z-score
def statistical_outliers(X, stds=3.0):
    # each column is centered around 0
    X_centered = X - X.mean(axis=0, keepdims=True)
    X_std = X_centered.std(axis=0, keepdims=True)
    # if all values ar eidentical
    X_std[X_std == 0] = 1.0
    # absolute z-score
    z = np.abs(X_centered) / X_std
    # lables (-1 -> outlier, 1 -> inlier)
```

```
    labels = np.where(np.any(z > stds, axis=1), -1, 1)
    # ouutlier score (biigger score -> more extreme row)
    scores = z.max(axis=1)
    return labels, scores

# continuos values
continuos_features = ["cnt", "casual", "registered", "temp", "atemp", "hum",
         "windspeed"]
X_outlier_df = df_clean[continuos_features].astype(float).copy()
# standarise features to range 0-1
X_outlier = StandardScaler().fit_transform(X_outlier_df)
y_stat, stat_scores = statistical_outliers(X_outlier, stds=3.0)
```

LOF and Isolation Forest

```
expected_outlier_rate = float(np.clip((y_stat == -1).mean(), 0.01, 0.1))

lof = LocalOutlierFactor(n_neighbors=20, contamination=expected_outlier_rate)
y_lof = lof.fit_predict(X_outlier)

# n_estimators -> number of trees
isolation_forest = IsolationForest(n_estimators=300,
         contamination=expected_outlier_rate, random_state=42)
y_isolation_forest = isolation_forest.fit_predict(X_outlier)
```

Summary

- table with jaccard overlap $J = \frac {|A \bigcap B|} {|A \bigcup B|}$

```
df_clean["outlier_stat"] = y_stat
df_clean["outlier_isolation_forest"] = y_isolation_forest
df_clean["outlier_lof"] = y_lof

# summary table
outlier_summary = pd.DataFrame({
    "outlier_rows": {
        "Statistical": int((y_stat == -1).sum()),
        "IsolationForest": int((y_isolation_forest == -1).sum()),
        "LOF": int((y_lof == -1).sum()),
    },
    # counts how many rows are flagged as outliers by each method
    "fraction_of_dataset": {
        "Statistical": float((y_stat == -1).mean()),
        "IsolationForest": float((y_isolation_forest == -1).mean()),
        "LOF": float((y_lof == -1).mean()),
    }
}).round(4).sort_values("fraction_of_dataset", ascending=False)

method_labels = {
    "Statistical": y_stat,
    "IsolationForest": y_isolation_forest,
    "LOF": y_lof,
}
```

```
jaccard_overlap_matrix = pd.DataFrame(index=method_labels.keys(),
         columns=method_labels.keys(), dtype=float)
for method_i, label_i in method_labels.items():
    mask_i = label_i == -1
    for method_j, labels_j in method_labels.items():
        mask_j = labels_j == -1
        union = (mask_i | mask_j).sum()
        jaccard_overlap_matrix.loc[method_i, method_j] = ((mask_i &
         mask_j).sum() / union) if union > 0 else 1.0

print(f"features = {continuos_features}")
print(f"shared expected_outlier_rate baseline = {expected_outlier_rate:.4f}")
print("\nOutlier rate by method:")
display(outlier_summary)
print("Pairwise overlap of flagged rows (Jaccard):")
display(jaccard_overlap_matrix.round(3))
```

```
features = ['cnt', 'casual', 'registered', 'temp', 'atemp', 'hum',
'windspeed']
shared expected_outlier_rate baseline = 0.0555

Outlier rate by method:
```

	outlier_rows	fraction_of_dataset
Statistical	965	0.0555
IsolationForest	965	0.0555
LOF	965	0.0555

```
Pairwise overlap of flagged rows (Jaccard):
```

	Statistical	IsolationForest	LOF
Statistical	1.000	0.537	0.106
IsolationForest	0.537	1.000	0.142
LOF	0.106	0.142	1.000

Observations (outlier rate by method + Jaccard overlap):

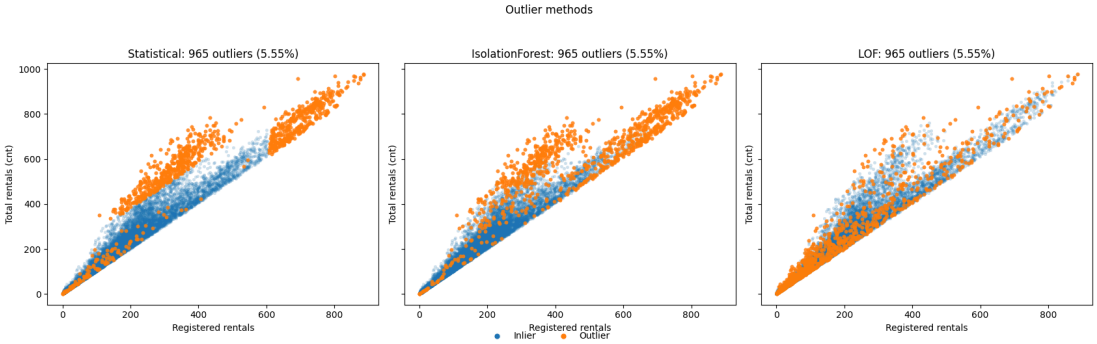
- **features:**
  - `cnt`, `casual`, `registered`, `temp`, `atemp`, `hum`, `windspeed`
  - these are continuous type
- **shared expected_outlier_rate baseline:**
  - value is `0.0555`, so all three methods were matched around about **5.55%** expected anomalies
- **Statistical:**
  - flagged `965` rows (`5.55%`)
  - the baseline found rows that are globally extreme compared to the mean / standard deviation pattern
- **IsolationForest:**
  - also flagged `965` rows (`5.55%`)
  - this is expected because the method uses the same expected outlier rate as input
  - but the rows might be different as the statistical method
- **LOF:**

    - also flagged `965` rows (`5.55%`)
    - again, this is expected because it was calibrated with the same expected outlier rate
    - LOF is local-density based, so it can still find different rows than the other two methods
- **Pairwise overlap (Jaccard):**
    - `Statistical` vs `IsolationForest` = `0.537` -> moderate overlap, so these two methods agree on a fair part of the same flagged rows
    - `Statistical` vs `LOF` = `0.106` -> very small overlap, so LOF is detecting a rather different type of outlier
    - `IsolationForest` vs `LOF` = `0.142` -> also small overlap, which again shows that LOF behaves differently from the more global methods

Overall:

- all three methods flag the same **number** of outliers, but not the same **rows**
- **Statistical** and **IsolationForest** are much closer to each other, while **LOF** identifies more local neighborhood-based anomalies

```
method_plot_order = [
    ("Statistical", "outlier_stat"),
    ("IsolationForest", "outlier_isolation_forest"),
    ("LOF", "outlier_lof"),
]

fig, axes = plt.subplots(1, 3, figsize=(17, 5), sharex=True, sharey=True)

for ax, (title, col) in zip(axes, method_plot_order):
    inlier_mask = df_clean[col] == 1
    outlier_mask = df_clean[col] == -1

    ax.scatter(df_clean.loc[inlier_mask, "registered"],
         df_clean.loc[inlier_mask, "cnt"],
               s=12, c="#1f77b4", alpha=0.22, linewidths=0)
    ax.scatter(df_clean.loc[outlier_mask, "registered"],
         df_clean.loc[outlier_mask, "cnt"],
               s=18, c="#ff7f0e", alpha=0.85, linewidths=0)

    out_n = int(outlier_mask.sum())
    out_pct = outlier_mask.mean() * 100
    ax.set_title(f"{title}: {out_n} outliers ({out_pct:.2f}%)")
    ax.set_xlabel("Registered rentals")
    ax.set_ylabel("Total rentals (cnt)")

legend_handles = [
    Line2D([0], [0], marker="o", color="w", markerfacecolor="#1f77b4",
         markersize=7, label="Inlier"),
    Line2D([0], [0], marker="o", color="w", markerfacecolor="#ff7f0e",
         markersize=7, label="Outlier"),
]
fig.legend(handles=legend_handles, loc="lower center", ncol=2, frameon=False,
           bbox_to_anchor=(0.5, -0.02))
fig.suptitle("Outlier methods", y=1.04)
plt.tight_layout()
plt.show()
```



Observations (outlier method plots):

- **Statistical:**
  - the orange points are mostly concentrated in the more extreme upper part of the diagonal
  - so this method mainly flags records that are globally extreme, especially high-demand combinations
- **IsolationForest:**
  - it also captures many of the upper / outer points, so it looks partly similar to the statistical method
  - however, the orange points are spread a bit more through the middle structure of the diagonal
  - this suggests it is not only looking at extreme values, but also at unusual multivariate combinations
- **LOF:**
  - the orange points are much more spread along lower-density edges and sparse local regions of the diagonal
  - it marks many points that do not look globally extreme, but are locally isolated compared to their neighbors
  - visually, LOF is clearly the most different method here, which is consistent with the low Jaccard overlap above

Overall:

- even though all three methods flag the same number of rows (`965`), the plots show that they focus on different regions of the data
- **Statistical** and **IsolationForest** are closer to global / overall unusual demand patterns -> as it was visible in the table too
- **LOF** captures more local sparse-pattern anomalies, so it complements the other two methods instead of repeating them, the suprise is that it looks like almost the opposite compare to previous ones 😄

2.4 Clustering evaluation

This section implements comprehensive clustering quality metrics to evaluate both **K-Means** and **DBSCAN** clustering against ground truth labels (derived from temporal and weather categories).

Implemented metrics:

- **Contingency Table:** Cross-tabulation of predicted clusters vs. ground truth labels
- **Purity:** Fraction of correctly classified samples (majority class in each cluster)
- **Mutual Information (MI):** Shared information between cluster assignment and ground truth

- **Conditional Entropy:** Uncertainty in ground truth given cluster assignment
- **Normalized Mutual Information (NMI):** MI normalized by entropies
- **F-Measure:** Harmonic mean of precision and recall at cluster-level
- **Maximum Matching:** Optimal one-to-one assignment between clusters and ground truth classes

```
def compute_contingency_table(predictions, ground_truth):
    """Compute contingency table (confusion matrix)."""
    return pd.crosstab(predictions, ground_truth, rownames=["Predicted"],
         colnames=["Ground Truth"])

def compute_mutual_information(predictions, ground_truth):
    """
    Compute mutual information between predictions and ground truth using
         sklearn.
    Measures shared information between clustering and ground truth.
    Range: [0, ∞), higher is better
    """
    # Filter out noise points if present
    valid_mask = predictions != -1
    pred = predictions[valid_mask].astype(int)
    truth = ground_truth[valid_mask].astype(int)

    if len(pred) == 0:
        return 0.0

    # Use sklearn's mutual_info_score
    return mutual_info_score(truth, pred)

def compute_conditional_entropy(predictions, ground_truth):
    """
    Compute conditional entropy H(Y|Y_hat) - entropy of ground truth given
         clustering.
    Measures remaining uncertainty in ground truth labels given the cluster
         assignments.
    Lower values indicate better clustering (less uncertainty remains).
    """
    valid_mask = predictions != -1
    pred = predictions[valid_mask].astype(int)
    truth = ground_truth[valid_mask].astype(int)

    if len(pred) == 0:
        return 0.0

    n = len(pred)
    clusters = np.unique(pred)
    classes = np.unique(truth)

    contingency = np.zeros((len(clusters), len(classes)))
    cluster_map = {c: i for i, c in enumerate(clusters)}
    class_map = {c: i for i, c in enumerate(classes)}

    for p, t in zip(pred, truth):
        contingency[cluster_map[p], class_map[t]] += 1
```

```
    # H(Y|Y_hat) = sum_i P(Y_hat=i) * H(Y|Y_hat=i)
    h_conditional = 0
    for i in range(len(clusters)):
        cluster_size = contingency[i, :].sum()
        if cluster_size > 0:
            p_cluster = cluster_size / n
            # Entropy within this cluster
            h_cluster = scipy_entropy(contingency[i, :] / cluster_size,
     base=2)
            h_conditional += p_cluster * h_cluster

    return h_conditional

def compute_purity(predictions, ground_truth):
    """
    Compute Purity: proportion of correctly labeled points.
    Purity = (1/n) * sum_i max_j(n_ij)
    Each cluster is assigned the most common class label, accuracy is
         computed.
    Range: [0, 1], higher is better
    """
    valid_mask = predictions != -1
    pred = predictions[valid_mask].astype(int)
    truth = ground_truth[valid_mask].astype(int)

    if len(pred) == 0:
        return 0.0

    n = len(pred)
    clusters = np.unique(pred)
    classes = np.unique(truth)

    contingency = np.zeros((len(clusters), len(classes)))
    cluster_map = {c: i for i, c in enumerate(clusters)}
    class_map = {c: i for i, c in enumerate(classes)}

    for p, t in zip(pred, truth):
        contingency[cluster_map[p], class_map[t]] += 1

    # For each cluster, find the most common class
    purity = contingency.max(axis=1).sum() / n

    return purity

def compute_maximum_matching(predictions, ground_truth):
    """
    Compute maximum matching using Hungarian algorithm between clusters and
         classes.
    Returns the maximum achievable accuracy with optimal label assignment.
    Range: [0, 1], higher is better
    """
    valid_mask = predictions != -1
    pred = predictions[valid_mask].astype(int)
    truth = ground_truth[valid_mask].astype(int)
```

```
    if len(pred) == 0:
        return 0.0

    n = len(pred)
    clusters = np.unique(pred)
    classes = np.unique(truth)

    # Contingency matrix
    contingency = np.zeros((len(clusters), len(classes)))
    cluster_map = {c: i for i, c in enumerate(clusters)}
    class_map = {c: i for i, c in enumerate(classes)}

    for p, t in zip(pred, truth):
        contingency[cluster_map[p], class_map[t]] += 1

    # Use Hungarian algorithm to find optimal assignment
    # Negate to convert max problem to min problem
    row_ind, col_ind = linear_sum_assignment(-contingency)

    # Compute accuracy with optimal assignment
    maximum_matching = contingency[row_ind, col_ind].sum() / n

    return maximum_matching

def compute_f_measure(predictions, ground_truth, beta=1.0):
    """
    Compute F-measure using optimal label assignment via Hungarian algorithm.
    F = (1 + beta²) * (precision * recall) / (beta² * precision + recall)
    With optimal assignment, F-measure combines cluster precision and recall.
    Range: [0, 1], higher is better
    """
    valid_mask = predictions != -1
    pred = predictions[valid_mask].astype(int)
    truth = ground_truth[valid_mask].astype(int)

    if len(pred) == 0:
        return 0.0

    n = len(pred)
    clusters = np.unique(pred)
    classes = np.unique(truth)

    # Build contingency matrix
    contingency = np.zeros((len(clusters), len(classes)))
    cluster_map = {c: i for i, c in enumerate(clusters)}
    class_map = {c: i for i, c in enumerate(classes)}

    for p, t in zip(pred, truth):
        contingency[cluster_map[p], class_map[t]] += 1

    # Find best matching using Hungarian algorithm
    row_ind, col_ind = linear_sum_assignment(-contingency)
```

```
    # Sum of correct assignments
    tp = contingency[row_ind, col_ind].sum()

    # Since we're using optimal matching, precision and recall are same
    accuracy = tp / n if n > 0 else 0

    # F-measure equals accuracy after optimal matching
    f_measure = accuracy

    return f_measure

def evaluate_clustering(predictions, ground_truth, label_name="Ground
     Truth"):
    """Clustering evaluation using required metrics."""
    # Filter out noise points (label -1) if present
    valid_mask = predictions != -1
    pred_clean = predictions[valid_mask]
    truth_clean = ground_truth[valid_mask]

    if len(pred_clean) == 0:
        # Handle case where all points are noise
        return {
            "label": label_name,
            "purity": 0,
            "f_measure": 0,
            "maximum_matching": 0,
            "mutual_information": 0,
            "conditional_entropy": 0,
            "contingency_table": None
        }

    # Get metrics
    purity = compute_purity(predictions, ground_truth)
    f_measure = compute_f_measure(predictions, ground_truth)
    max_matching = compute_maximum_matching(predictions, ground_truth)
    mutual_info = compute_mutual_information(predictions, ground_truth)
    cond_entropy = compute_conditional_entropy(predictions, ground_truth)
    contingency = compute_contingency_table(pred_clean, truth_clean)

    results = {
        "label": label_name,
        "purity": purity,
        "f_measure": f_measure,
        "maximum_matching": max_matching,
        "mutual_information": mutual_info,
        "conditional_entropy": cond_entropy,
        "contingency_table": contingency
    }

    return results
```

2.4.1 K-Means Clustering Evaluation

```
# Ground truth labels for evaluation
ground_truth_labels = {
    "season": df_clean["season"].values,
    "workingday": df_clean["workingday"].values,
    "weather": df_clean["weathersit"].values,
    "hour_binned": pd.cut(df_clean["hr"], bins=[0, 6, 12, 18, 24], labels=
        ["night", "morning", "afternoon", "evening"],
        right=False).cat.codes.values,
}

# Evaluate K-Means against different ground truth labels
print("K-MEANS clustering evaluation\n")

kmeans_results = {}
kmeans_contingency = {}

for label_name in ground_truth_labels.keys():
    eval_result =
        evaluate_clustering(df_clean["cluster_hourly"].values.astype(int),
        ground_truth_labels[label_name], label_name)
    kmeans_results[label_name] = eval_result
    kmeans_contingency[label_name] = eval_result.pop("contingency_table",
        None)  # Remove contingency for now

kmeans_metrics_df = pd.DataFrame(kmeans_results).T
print("Metrics Summary:")
display(kmeans_metrics_df)

print("\nContingency tables for K-MEANS:")
for label_name, table in kmeans_contingency.items():
    if table is not None:
        print(f"\n{label_name.upper()}:")
        display(table)
```

```
K-MEANS clustering evaluation

Metrics Summary:
```

	label	purity	f_measure	maximum_matching	mutu
season	season	0.302146	0.284999	0.284999	0.028
workingday	workingday	0.742275	0.440934	0.440934	0.087
weather	weather	0.656712	0.383049	0.383049	0.007
hour_binned	hour_binned	0.545716	0.545716	0.545716	0.514

```
Contingency tables for K-MEANS:

SEASON:
```

Ground Truth	1	2	3	4
Predicted				
0	2166	1748	1728	1806
1	1836	1601	1519	1549
2	68	500	511	282
3	172	560	738	595

WORKINGDAY:

Ground Truth	0	1
Predicted		
0	2014	5434
1	2229	4276
2	1198	163
3	73	1992

WEATHER:

Ground Truth	1	2	3	4
Predicted				
0	4778	1974	694	2
1	4090	1800	614	1
2	1068	261	32	0
3	1477	509	79	0

HOUR_BINNED:

Ground Truth	0	1	2	3
Predicted				
0	0	1124	2844	3480
1	4276	2215	14	0
2	0	235	942	184
3	0	786	575	704

Observations (K-Means metrics + contingency tables):

- **season:**
  - **bins**
    - **1:** winter
    - **2:** spring
    - **3:** summer

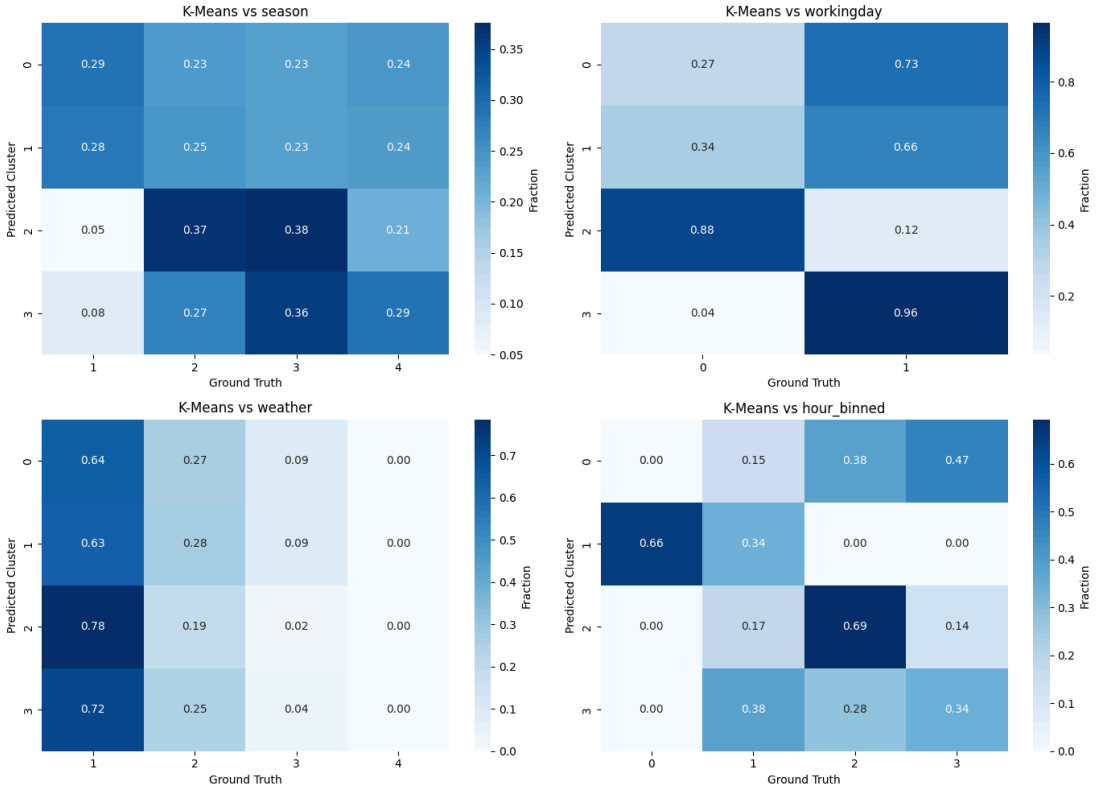
        - **4:** fall
    - `purity = 0.3017` is lowest from all 4 classes
    - contingency counts are distributed across all season columns, indicating K-Means is not primarily learning season structure from these features, that was also confirmed in the EDA analysis
- **workingday:**
    - **bins**
        - **0:** non-working day
        - **1:** working day
    - the contingency table shows a clear workday-heavy cluster (`cluster 0` -> [5454 working day vs 2020 non-working] and `cluster 3` -> [1978 working day vs 69 non-working])
    - the strongest alignment (`purity = 0.7422`) among tested labels, so best-matching score for K-Means (cluster to label)
    - `f_measure = 0.4420` -> lower than purity (only checks majority label in each cluster) because F-measure is stricter (penalizes missed points and mixed assignments), so not the best aligned label with cluster
    - this confirms our analysis above
- **weather:**
    - **bins**
        - **1:** Clear/Few clouds
        - **2:** Mist/Cloudy
        - **3:** Light Snow/Rain
        - **4:** Heavy Rain/Snow
    - `purity = 0.6567` is moderate -> in each cluster, one weather class is often the majority
    - `mutual_information = 0.0072` is very low -> the cluster gives almost no extra information about true weather class, so no much informative
        - this makes sense because weather class **1** (Clear/Few clouds), is very common in the dataset (as visible in the contigency table)
    - overall, weather might not be the most important value of out clusters in K-Means
- **hour_binned:**
    - **bins**
        - **0:** night for hours $[0, 6)$
        - **1:** morning for hours $[6, 12)$
        - **2:** afternoon for hours $[12, 18)$
        - **3:** evening for hours $[18, 24)$
    - in the contingency table, each cluster row has one or two large columns -> not equal values across all 4 bins
        - cluster 1 is mostly night or morning (4276,2206) and almost none in the afternoon or evening (14,0)
        - cluster 0 is mostly the afternoon or evening (2850,3487) and none at night
    - purity is about 54.6%
    - `f_measure = 0.5459` and `maximum_matching = 0.5459`, performance is similar
    - `mutual_information = 0.5137` -> relatively strong and it means cluster labels share substantial information with time bins
    - K-Means is learning houtly behaviour, which is fine, because we see these data as valid in our dataset

Overall:

- best by **purity** is `workingday = 0.7422`
- best overall informative match `hour_binned`, it has highest `mutual_information` anf highest `f_measure`, as well as `maximum_matching`

```python
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

print("K-MEANS Contingency Tables")
print("Rows = Predicted Clusters | Columns = Ground Truth Labels\n")

for idx, label_name in enumerate(ground_truth_labels.keys()):
    contingency = kmeans_contingency[label_name]
    # Normalize by rows (cluster totals)
    contingency_norm = contingency.div(contingency.sum(axis=1), axis=0)

    sns.heatmap(
        contingency_norm,
        annot=True,
        fmt=".2f",
        cmap="Blues",
        cbar_kws={"label": "Fraction"},
        ax=axes[idx]
    )
    axes[idx].set_title(f"K-Means vs {label_name}")
    axes[idx].set_xlabel("Ground Truth")
    axes[idx].set_ylabel("Predicted Cluster")

plt.tight_layout()
plt.show()

# Metrics summary for required evaluation metrics
print("\nK-MEANS Clustering Evaluation Metrics Summary")
print("\nMetrics for each ground truth label:")
print(kmeans_metrics_df[['purity', 'f_measure', 'maximum_matching',
                         'mutual_information',
         'conditional_entropy']].round(4))
print("\nMetric Descriptions:")
print("  • Purity: Fraction of correctly classified samples (0-1, higher
         better)")
print("  • F-Measure: Harmonic mean after optimal assignment (0-1, higher
         better)")
print("  • Maximum Matching: Accuracy with optimal label assignment (0-1,
         higher better)")
print("  • Mutual Information: Shared information with ground truth (0-∞,
         higher better)")
print("  • Conditional Entropy: Uncertainty in labels given clusters (0-∞,
         lower better)")
```

```
K-MEANS Contingency Tables
Rows = Predicted Clusters | Columns = Ground Truth Labels
```




```
K-MEANS Clustering Evaluation Metrics Summary

Metrics for each ground truth label:
               purity f_measure maximum_matching mutual_information  \
season       0.302146  0.284999         0.284999             0.0281
workingday   0.742275  0.440934         0.440934           0.087206
weather      0.656712  0.383049         0.383049           0.007252
hour_binned  0.545716  0.545716         0.545716            0.51409

            conditional_entropy
season                 1.958981
workingday             0.775587
weather                1.191233
hour_binned            1.258263

Metric Descriptions:
  • Purity: Fraction of correctly classified samples (0-1, higher better)
  • F-Measure: Harmonic mean after optimal assignment (0-1, higher better)
  • Maximum Matching: Accuracy with optimal label assignment (0-1, higher
better)
  • Mutual Information: Shared information with ground truth (0-∞, higher
better)
  • Conditional Entropy: Uncertainty in labels given clusters (0-∞, lower
better)
```

Observations (K-Means heatmaps + metrics summary):

- **season:** is mixed across columns in each row, consistent with weak season alignment (purity ~0.30), as we mention from tables above
- **workingday:** has the highest focus from the normalised heatmaps: from the one cluster is visible that non-workingday value is high (~0.88), while another is almost

entirely workingday (~0.97)
- **weather:** is only partially separated -> most clusters are still dominated by class 1, which matches moderate purity but low mutual informaiton
- **hour_binned:** also shows structured concentration (one cluster is mostly bin 0 at ~0.66 and another mostly bin 2 at ~0.69, a in bin 3 at ~0.47)
- **metric summary:** shows again that workingday and hour_binned are the stongest, while season is the weakest among evaluated labels

2.4.2 DBSCAN Clustering Evaluation

```
# Evaluate DBSCAN against different ground truth labels
print("DBSCAN clustering evaluation")
print("Note: Noise points (label = -1) are excluded from metrics\n")

dbscan_results = {}
dbscan_contingency = {}

for label_name in ground_truth_labels.keys():
    eval_result =
         evaluate_clustering(df_clean["cluster_dbscan"].values.astype(int),
         ground_truth_labels[label_name], label_name)
    dbscan_results[label_name] = eval_result
    dbscan_contingency[label_name] = eval_result.pop("contingency_table",
         None)

dbscan_metrics_df = pd.DataFrame(dbscan_results).T
print("Metrics Summary:")
display(dbscan_metrics_df)

print(f"\nNoise points in DBSCAN: {(df_clean['cluster_dbscan'] == -1).sum()}
         / {len(df_clean)} ({100*(df_clean['cluster_dbscan'] ==
         -1).sum()/len(df_clean):.2f}%)")

print("\nContingency tables for DBSCAN:")
for label_name, table in dbscan_contingency.items():
    if table is not None:
        print(f"\n{label_name.upper()}:")
        display(table)
```

```
DBSCAN clustering evaluation
Note: Noise points (label = -1) are excluded from metrics

Metrics Summary:
```

	label	purity	f_measure	maximum_matching	mutu
season	season	0.258485	0.258485	0.258485	0.002
workingday	workingday	0.708485	0.700788	0.700788	0.002
weather	weather	0.648424	0.642909	0.642909	0.000
hour_binned	hour_binned	0.262727	0.262727	0.262727	0.005

```
Noise points in DBSCAN: 879 / 17379 (5.06%)
```

Contingency tables for DBSCAN:

SEASON:

Ground Truth	1	2	3	4
Predicted				
0	4191	4075	4108	3999
1	4	34	74	15

WORKINGDAY:

Ground Truth	0	1
Predicted		
0	4810	11563
1	0	127

WEATHER:

Ground Truth	1	2	3	4
Predicted				
0	10593	4385	1392	3
1	106	15	6	0

HOUR_BINNED:

Ground Truth	0	1	2	3
Predicted				
0	4269	4237	3729	4138
1	0	0	66	61

Observations (DBSCAN metrics + contingency tables):

- **season:**
  - **bins**
    - **1:** winter
    - **2:** spring
    - **3:** summer
    - **4:** fall
  - the contingency table is almost same across season columns for predicted cluster 0 (`4191, 4075, 4108, 3999`), so DBSCAN is not learning season structure from these features
  - `purity = 0.2585` is low (close to random for 4 classes)
  - `f_measure = 0.2585` is also low
  - the result very similar to K-Means
- **workingday:**

    - **bins**
        - **0:** non-working day
        - **1:** working day
    - the contingency table shows strong workday concentration in predicted cluster 0 -> [`11563` working-day vs `4810` non-working] and cluster 1 is small (`127` vs `0`, only working day :D)
    - strongest matching label in DBSCAN by `purity = 0.7085`, `f_measure = 0.7008`, and `maximum_matching = 0.7008`
    - `mutual_information = 0.0027` is very low
- **weather:**
    - **bins**
        - **1:** Clear/Few clouds
        - **2:** Mist/Cloudy
        - **3:** Light Snow/Rain
        - **4:** Heavy Rain/Snow
    - the contingency table confirms dominance of class 1 in predicted cluster 0 (`10593`)
    - `purity = 0.6484` and `f_measure = 0.6429` are moderate values
    - `mutual_information = 0.0007` is near zero, so weather classes are not strongly informative globally for DBSCAN partitions
- **hour_binned:**
    - **bins**
        - **0:** night for hours $[0, 6)$
        - **1:** morning for hours $[6, 12)$
        - **2:** afternoon for hours $[12, 18)$
        - **3:** evening for hours $[18, 24)$
    - the contingency table for predicted cluster 0 is spread across all bins (`4269, 4237, 3729, 4138`) instead of concentrating in specific time bins
    - `purity = 0.2627`, `f_measure = 0.2627`, `maximum_matching = 0.2627` -> very weak alignment
    - `mutual_information = 0.0056` is very low
- **noise points:**
    - DBSCAN marked `879 / 17379` points as noise (`5.06%`) -> this is expected for density-based clustering and helps isolate sparse/extreme points

Overall:

- best DBSCAN matching label is **workingday** (highest purity/f-measure/maximum matching)
- for multiclass labels (**season**, **weather**, **hour_binned**), `mutual_information` remains near zero, so DBSCAN captures dense regimes but not detailed class structure in this feature space
- we would say the result are very similar compare to K-Means

```
# Detailed contingency tables for DBSCAN
print("DBSCAN Contingency Tables")
print("Rows = Predicted Clusters (incl. -1 for noise) | Columns = Ground
        Truth Labels\n")

for label_name in ground_truth_labels.keys():
    print(f"{label_name.upper()}")
    display(dbscan_contingency[label_name])
    print()

# Visualize contingency tables as heatmaps
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
```

```
axes = axes.flatten()

for idx, label_name in enumerate(ground_truth_labels.keys()):
    contingency = dbscan_contingency[label_name]
    contingency_norm = contingency.div(contingency.sum(axis=1), axis=0)

    sns.heatmap(
        contingency_norm,
        annot=True,
        fmt=".2f",
        cmap="Oranges",
        cbar_kws={"label": "Fraction"},
        ax=axes[idx]
    )
    axes[idx].set_title(f"DBSCAN vs {label_name}")
    axes[idx].set_xlabel("Ground Truth")
    axes[idx].set_ylabel("Predicted Cluster (incl. noise -1)")

plt.tight_layout()
plt.show()

# DBSCAN metrics summary
print("\nDBSCAN Clustering Evaluation Metrics Summary")
print("\nMetrics for each ground truth label:")
print(dbscan_metrics_df[['purity', 'f_measure', 'maximum_matching',
                         'mutual_information',
         'conditional_entropy']].round(4))
print("\nMetric Descriptions:")
print("  • Purity: Fraction of correctly classified samples (0-1, higher
         better)")
print("  • F-Measure: Harmonic mean after optimal assignment (0-1, higher
         better)")
print("  • Maximum Matching: Accuracy with optimal label assignment (0-1,
         higher better)")
print("  • Mutual Information: Shared information with ground truth (0-∞,
         higher better)")
print("  • Conditional Entropy: Uncertainty in labels given clusters (0-∞,
         lower better)")
print(f"\nNoise Statistics:")
print(f"  • Points classified as noise: {(df_clean['cluster_dbscan'] ==
         -1).sum()} / {len(df_clean)} ({100*(df_clean['cluster_dbscan'] ==
         -1).sum()/len(df_clean):.2f}%)")

DBSCAN Contingency Tables
Rows = Predicted Clusters (incl. -1 for noise) | Columns = Ground Truth
Labels

SEASON
```

Ground Truth	1	2	3	4
Predicted				
0	4191	4075	4108	3999
1	4	34	74	15

WORKINGDAY

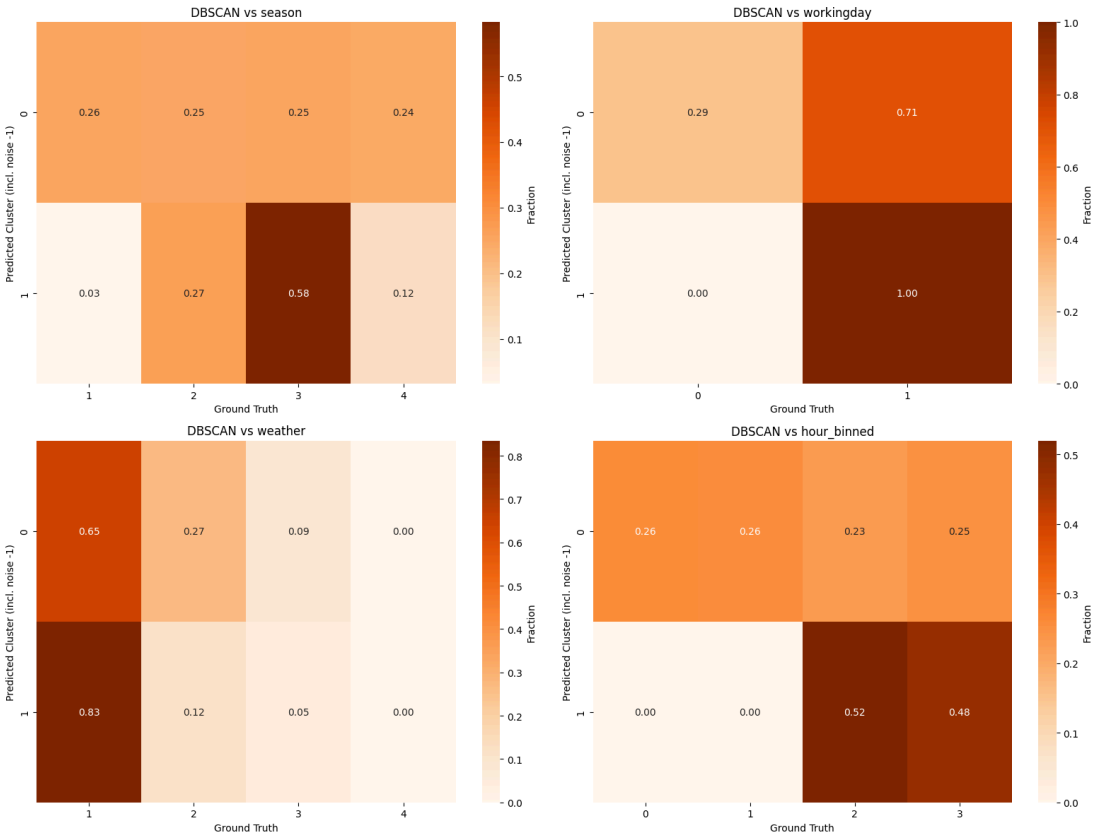
Ground Truth	0	1
Predicted		
0	4810	11563
1	0	127

WEATHER

Ground Truth	1	2	3	4
Predicted				
0	10593	4385	1392	3
1	106	15	6	0

HOUR_BINNED

Ground Truth	0	1	2	3
Predicted				
0	4269	4237	3729	4138
1	0	0	66	61



```
DBSCAN Clustering Evaluation Metrics Summary

Metrics for each ground truth label:
               purity f_measure maximum_matching mutual_information  \
season       0.258485  0.258485         0.258485            0.00274
workingday   0.708485  0.700788         0.700788           0.002665
weather      0.648424  0.642909         0.642909           0.000674
hour_binned  0.262727  0.262727         0.262727           0.005641

            conditional_entropy
season                 1.995827
workingday             0.866826
weather                1.216766
hour_binned            1.990269

Metric Descriptions:
  • Purity: Fraction of correctly classified samples (0-1, higher better)
  • F-Measure: Harmonic mean after optimal assignment (0-1, higher better)
  • Maximum Matching: Accuracy with optimal label assignment (0-1, higher
better)
  • Mutual Information: Shared information with ground truth (0-∞, higher
better)
  • Conditional Entropy: Uncertainty in labels given clusters (0-∞, lower
better)

Noise Statistics:
  • Points classified as noise: 879 / 17379 (5.06%)
```

Observations (DBSCAN contingency heatmaps + metrics summary):

- **season:**
  - predicted cluster 0 is almost same across seasons (`0.26, 0.25, 0.25, 0.24`) -> as above from the contigency tables
  - predicted cluster 1 is mostly season 3 (`~0.58`) and partly season 2 (`~0.27`)
  - confirms weak season matching (`purity = 0.2585`, `f_measure = 0.2585`)
- **workingday:**
  - predicted cluster 1 is only working-day (`1.00`)
  - predicted cluster 0 is still mostly working-day (`~0.71`)
  - this is why DBSCAN has its best scores here (`purity = 0.7085`, `f_measure = 0.7008`)
- **weather:**
  - both predicted clusters are dominated by weather class 1 (`clear/few clouds`)
  - gives moderate purity/f-measure but very low information gain (`mutual_information = 0.000674`)
- **hour_binned:**
  - predicted cluster 0 is spread over all bins (`~0.23–0.28`)
  - predicted cluster 1 is concentrated on afternoon/evening (`~0.52`, `~0.48`)
  - overall still weak for hour bins (`purity = 0.2627`, `f_measure = 0.2627`)
- **metrics summary:**
  - `season` and `hour_binned` have high conditional entropy (`~1.99`) -> high uncertainty remains
  - only `workingday` is clearly aligned in DBSCAN
- **noise:**
  - `879 / 17379` points are noise (`5.06%`), expected for DBSCAN -> the same as before

Overall:

- DBSCAN mainly captures one dense **workingday** regime, not a detailed multi-class split by season/weather/hour bins

```
# Comparison of K-Means vs DBSCAN Clustering Metrics
print("Comparison: K-MEANS vs DBSCAN")

# Prepare comparison data (exclude 'label' column)
comparison_metrics = ['purity', 'f_measure', 'maximum_matching',
                      'mutual_information', 'conditional_entropy']

print("\nDetailed Metric Comparison by Ground Truth Label:")

for label_name in ground_truth_labels.keys():
    print(f"\n{label_name.upper()}:")
    print("-" * 60)

    kmeans_row = kmeans_metrics_df.loc[label_name,
         comparison_metrics].astype(float)
    dbscan_row = dbscan_metrics_df.loc[label_name,
         comparison_metrics].astype(float)

    comparison_df = pd.DataFrame({
        'K-Means': kmeans_row,
        'DBSCAN': dbscan_row,
        'Difference': (kmeans_row - dbscan_row)
    }).round(4)

    display(comparison_df)

# Summary statistics across all ground truths
print("Overall summary (Mean across all ground truth labels):")

kmeans_mean = kmeans_metrics_df[comparison_metrics].astype(float).mean()
dbscan_mean = dbscan_metrics_df[comparison_metrics].astype(float).mean()

summary_df = pd.DataFrame({
    'K-Means': kmeans_mean,
    'DBSCAN': dbscan_mean,
    'Difference': (kmeans_mean - dbscan_mean)
}).round(4)

display(summary_df)

# Interpretation
print("\nMetric Interpretation Guide:")
print("  • Purity ↑: Higher values = clusters align better with ground truth
         classes")
print("  • F-Measure ↑: Higher values = better balance of precision and
         recall")
print("  • Maximum Matching ↑: Higher values = better achievable accuracy
         with optimal assignment")
print("  • Mutual Information ↑: Higher values = more shared information
         between clustering and ground truth")
```

```
print("  • Conditional Entropy ↓: Lower values = less uncertainty remaining
        in ground truth given clusters")


# Performance comparison
print("Performance Comparison:")

for metric in comparison_metrics:
    kmeans_avg = float(kmeans_metrics_df[metric].mean())
    dbscan_avg = float(dbscan_metrics_df[metric].mean())

    if metric == 'conditional_entropy':
        better = "DBSCAN" if dbscan_avg < kmeans_avg else "K-Means"
        diff_sign = "<" if dbscan_avg < kmeans_avg else ">"
    else:
        better = "K-Means" if kmeans_avg > dbscan_avg else "DBSCAN"
        diff_sign = ">" if kmeans_avg > dbscan_avg else "<"

    print(f"\n  {metric.upper()}:")
    print(f"    K-Means: {kmeans_avg:.4f}")
    print(f"    DBSCAN:  {dbscan_avg:.4f}")
    print(f"    Winner:  {better} ({kmeans_avg:.4f} {diff_sign}
         {dbscan_avg:.4f})")
```

```
Comparison: K-MEANS vs DBSCAN

Detailed Metric Comparison by Ground Truth Label:

SEASON:
------------------------------------------------------------
```

	K-Means	DBSCAN	Difference
purity	0.3021	0.2585	0.0437
f_measure	0.2850	0.2585	0.0265
maximum_matching	0.2850	0.2585	0.0265
mutual_information	0.0281	0.0027	0.0254
conditional_entropy	1.9590	1.9958	-0.0368

```
WORKINGDAY:
------------------------------------------------------------
```

	K-Means	DBSCAN	Difference
purity	0.7423	0.7085	0.0338
f_measure	0.4409	0.7008	-0.2599
maximum_matching	0.4409	0.7008	-0.2599
mutual_information	0.0872	0.0027	0.0845
conditional_entropy	0.7756	0.8668	-0.0912

WEATHER:

	K-Means	DBSCAN	Difference
purity	0.6567	0.6484	0.0083
f_measure	0.3830	0.6429	-0.2599
maximum_matching	0.3830	0.6429	-0.2599
mutual_information	0.0073	0.0007	0.0066
conditional_entropy	1.1912	1.2168	-0.0255

HOUR_BINNED:

	K-Means	DBSCAN	Difference
purity	0.5457	0.2627	0.2830
f_measure	0.5457	0.2627	0.2830
maximum_matching	0.5457	0.2627	0.2830
mutual_information	0.5141	0.0056	0.5084
conditional_entropy	1.2583	1.9903	-0.7320

Overall summary (Mean across all ground truth labels):

	K-Means	DBSCAN	Difference
purity	0.5617	0.4695	0.0922
f_measure	0.4137	0.4662	-0.0526
maximum_matching	0.4137	0.4662	-0.0526
mutual_information	0.1592	0.0029	0.1562
conditional_entropy	1.2960	1.5174	-0.2214

Metric Interpretation Guide:
- Purity ↑: Higher values = clusters align better with ground truth classes
- F-Measure ↑: Higher values = better balance of precision and recall
- Maximum Matching ↑: Higher values = better achievable accuracy with optimal assignment
- Mutual Information ↑: Higher values = more shared information between clustering and ground truth
- Conditional Entropy ↓: Lower values = less uncertainty remaining in ground truth given clusters

Performance Comparison:

PURITY:
K-Means: 0.5617
DBSCAN: 0.4695
Winner: K-Means (0.5617 > 0.4695)

```
F_MEASURE:
  K-Means: 0.4137
  DBSCAN:  0.4662
  Winner:  DBSCAN (0.4137 < 0.4662)

MAXIMUM_MATCHING:
  K-Means: 0.4137
  DBSCAN:  0.4662
  Winner:  DBSCAN (0.4137 < 0.4662)

MUTUAL_INFORMATION:
  K-Means: 0.1592
  DBSCAN:  0.0029
  Winner:  K-Means (0.1592 > 0.0029)

CONDITIONAL_ENTROPY:
  K-Means: 1.2960
  DBSCAN:  1.5174
  Winner:  K-Means (1.2960 > 1.5174)
```

Observations (K-Means vs DBSCAN comparison):

- **mean purity:** K-Means is better (`0.5617` vs `0.4695`)
- **mean f_measure / maximum_matching:** DBSCAN is higher (`0.4662` vs `0.4139`)
- **mean mutual_information:** K-Means is much better (`0.1590` vs `0.0029`)
- **mean conditional_entropy:** K-Means is lower (`1.2962` vs `1.5174`) -> lower is better

Overall:

- for this project, **K-Means** is better as the main clustering model (more informative global structure)
- and **DBSCAN** is useful as complementary analysis for dense cores + noise/outliers
- **why DBSCAN has higher F1 but lower purity:**
  - **purity** measures how "clean" each cluster is by taking the majority class in each cluster. K-Means with k=4 has more clusters to work with, so each cluster can be more specialised toward one ground truth class, which gives higher purity
  - **F1 (maximum matching)** measures how well cluster-to-class pairs align overall. DBSCAN only finds 2 clusters + noise, but those 2 clusters happen to align well with the dominant structure in the data (high-demand vs everything else). the noise label also helps cause it removes ambiguous points that would otherwise hurt the matching
  - so basically K-Means wins on purity cause it has more clusters to "purify" each one, but DBSCAN wins on F1 cause its 2 clean clusters + noise separation happens to match the strongest ground truth split better

2.5 Limitations

A key limitation of our vector-space clustering is the **cyclical time problem**: hour 23 and hour 0 are adjacent in reality (one hour apart) but maximally distant in the feature space (23 vs 0 = distance 23). The same applies to weekday encoding where Sunday and Monday are neighbours but encoded far apart. Standard Euclidean distance in K-Means

and DBSCAN cannot capture this, which means points from adjacent time slots can end up in different clusters purely because of encoding, not because of actual behavioural differences.

One way to fix this would be to encode hour as `sin(2π·hr/24)` and `cos(2π·hr/24)` so that the circular structure is preserved, but we did not implement this in CP1. In CP2 (graph-based analysis) we partially address this by aggregating to `(weekday, hour)` nodes and connecting them by feature similarity rather than raw distance, so the cyclical encoding problem does not directly affect the graph construction. Pattern mining in module 3 may identify interpretable rules that do not rely on distance measures at all.

However, some limitations remain inherent to the dataset itself. In particular, the dataset lacks spatial information about bike stations and external event data such as festivals or major weather events, which prevents deeper behavioral or network-based analysis regardless of the mining method used.

3. Module 2 - Graph-Based Analysis

3.1 Graph interpretation

To apply graph-based data mining to the Bike Sharing dataset, we represent **weekly hourly usage contexts** as nodes in a graph.

- **Node:** one `(weekday, hour)` combination
- **Node attributes:** average `casual`, `registered`, `temp`, `hum`, and optionally `windspeed`
- **Edge:** two nodes are connected if their profiles are similar in the standardized feature space
- **Edge weight:** inverse distance, so more similar nodes are connected more strongly

We choose `(weekday, hour)` as the graph interpretation because our earlier analyses already suggested that bike demand is strongly structured by **time of day** and by the contrast between **commuting-oriented** and **leisure-oriented** usage.
Compared with a seasonal or monthly graph, this representation focuses more directly on **weekly behavioral rhythms**.
Monthly information is not ignored, but used later as an **external interpretation layer** for the detected communities.

```
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)

# Use cleaned dataframe if available, otherwise fall back to df
graph_df = df_clean.copy() if "df_clean" in globals() else df.copy()

# Weekday mapping used in the UCI dataset:
# 0 = Sunday, 1 = Monday, ..., 6 = Saturday
weekday_map = {
    0: "Sun",
    1: "Mon",
    2: "Tue",
    3: "Wed",
    4: "Thu",
    5: "Fri",
    6: "Sat"
}
```

```
graph_df["weekday_name"] = graph_df["weekday"].map(weekday_map)
graph_df["is_weekend"] = graph_df["weekday"].isin([0, 6]).astype(int)

print("Shape of graph source dataframe:", graph_df.shape)
display(graph_df.head())
```

```
Shape of graph source dataframe: (17379, 25)
```

	day	season	yr	mnth	hr	holiday	weekday	workingday	weathersit
0	1	1	0	1	0	0	6	0	1
1	1	1	0	1	1	0	6	0	1
2	1	1	0	1	2	0	6	0	1
3	1	1	0	1	3	0	6	0	1
4	1	1	0	1	4	0	6	0	1

5 rows × 25 columns

3.2 Node construction and feature selection

Each node represents one `(weekday, hour)` combination.
This gives us `7 × 24 = 168` nodes, which is detailed enough to capture weekly behavioral structure while still being small enough for clear graph analysis and visualization.

For each node, we compute the mean of:

- `casual`
- `registered`
- `temp`
- `hum`
- `windspeed`

These features were chosen because they summarize both:

- **usage behavior** (`casual`, `registered`)
- **weather context** (`temp`, `hum`, `windspeed`)

We intentionally do **not** include `cnt` in the similarity computation because `cnt = casual + registered`, which would introduce redundant information.

We also do **not** include `month` directly as a graph feature.
Instead, month will later be used as an external interpretation variable to understand whether some communities are more prominent in certain parts of the year.

```
node_df = (
    graph_df
    .groupby(["weekday", "weekday_name", "hr"], as_index=False)
    .agg(
```

```
        casual_mean=("casual", "mean"),
        registered_mean=("registered", "mean"),
        temp_mean=("temp", "mean"),
        hum_mean=("hum", "mean"),
        windspeed_mean=("windspeed", "mean"),
        cnt_mean=("cnt", "mean"),
        month_mean=("mnth", "mean")
    )
)

node_df["node_label"] = node_df["weekday_name"] + "_" +
         node_df["hr"].astype(str).str.zfill(2)

print("Number of graph nodes:", len(node_df))
display(node_df.head())
```

```
Number of graph nodes: 168
```

	weekday	weekday_name	hr	casual_mean	registered_mean	temp_mea
0	0	Sun	0	18.230769	75.759615	0.451538
1	0	Sun	1	15.000000	62.432692	0.442885
2	0	Sun	2	12.588235	49.039216	0.440000
3	0	Sun	3	8.365385	22.778846	0.431346
4	0	Sun	4	2.529412	6.833333	0.430784

```
feature_cols = [
    "casual_mean",
    "registered_mean",
    "temp_mean",
    "hum_mean",
    "windspeed_mean"
]

scaler = StandardScaler()
X = scaler.fit_transform(node_df[feature_cols])

display(node_df[["node_label"] + feature_cols + ["cnt_mean"]].head())
```

	node_label	casual_mean	registered_mean	temp_mean	hum_mean	win
0	Sun_00	18.230769	75.759615	0.451538	0.675385	0.1
1	Sun_01	15.000000	62.432692	0.442885	0.691731	0.1
2	Sun_02	12.588235	49.039216	0.440000	0.699608	0.1
3	Sun_03	8.365385	22.778846	0.431346	0.714808	0.1
4	Sun_04	2.529412	6.833333	0.430784	0.728431	0.1

3.2.1 Experimental Setup

Before sections of the graph mining methods, we documented preprocessing and experimental setup (including hyperparameter choices, tuning, and evaluation metrics)

- **Feature scaling:** we standardised the node features (`casual_mean`, `registered_mean`, `temp_mean`, `hum_mean`, `windspeed_mean`) using z-score scaling, mainly important for Euclidian distance (the code above)
- **Graph type:** we constructed a **k-nearest-neighbor similarity graph**, where each node is connected to its `k` most similar neighbours in the standardized feature space (the code below)
- **The choice of `k`:** we used `k = 5`, we chose this as a practical middle value (also as heuristic choice):
  - too small value could make the graph too sparse
  - too large value could make the graph too dense and local patterns may be harder to see
- **Edge weighting:** after identifying nearest neighbours, we assigned edge weights using `weight = 1 / (1 + distance)`, so nodes with more similar profiles receive stronger connections

We then applied two graph mining methods:

- **Louvain community detection:**
  - we chose it as the main method, it finds communities by maximizing modularity
- **Spectral clustering:**
  - we chose it as a second method to compare, so we can check whether the same structure appears under a different graph based method
  - we set the number of clusters equal to the number of Louvain communities, to make it easier for comparison

To evaluate the graph-based analysis, we used:

- **Community profiles:** (demand level, cnt type, hourly patterns)
- **weekday × hour heatmaps:** for visual interpretation
- **ARI and NMI:** to compare Louvain and spectral clustering
- **Comparisons with Module 1:** especially overlap with K-Means regimes and the concentration of CP1 outliers inside graph communities

3.3 Graph construction

We build a **k-nearest-neighbor similarity graph**.

This means:

- each node is connected to its `k` most similar neighbors,
- similarity is computed in the standardized feature space,
- the graph stays informative and avoids becoming either too sparse or fully connected.

We use edge weights based on inverse distance:

$weight = \frac{1}{1 + distance}$

So:

- small distance → large weight
- large distance → smaller weight

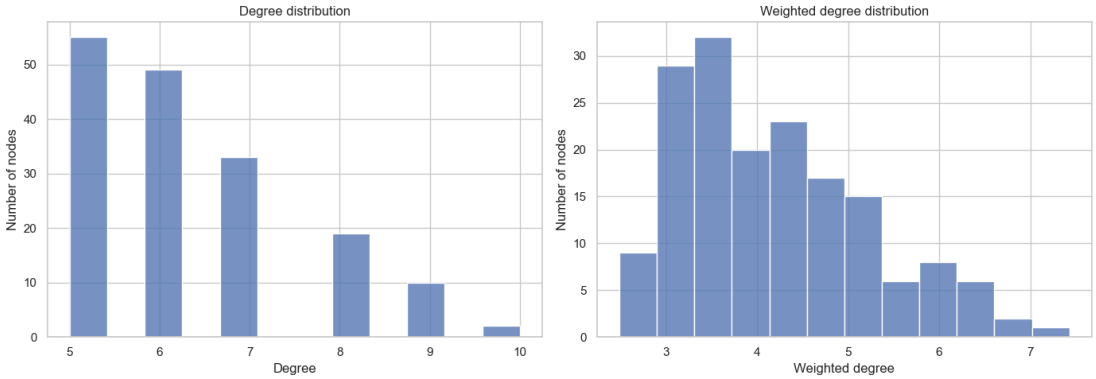
```
k = 5

nn = NearestNeighbors(n_neighbors=k + 1, metric="euclidean")
nn.fit(X)
```

```
distances, indices = nn.kneighbors(X)

G = nx.Graph()

# Add nodes with attributes
for i, row in node_df.iterrows():
    G.add_node(
        i,
        label=row["node_label"],
        weekday=int(row["weekday"]),
        weekday_name=row["weekday_name"],
        hr=int(row["hr"]),
        casual_mean=float(row["casual_mean"]),
        registered_mean=float(row["registered_mean"]),
        temp_mean=float(row["temp_mean"]),
        hum_mean=float(row["hum_mean"]),
        windspeed_mean=float(row["windspeed_mean"]),
        cnt_mean=float(row["cnt_mean"]),
        month_mean=float(row["month_mean"])
    )

# Add weighted undirected edges
for i in range(len(node_df)):
    for dist, j in zip(distances[i][1:], indices[i][1:]):  # skip self-
         neighbor
        j = int(j)
        weight = 1 / (1 + dist)

        if G.has_edge(i, j):
            if weight > G[i][j]["weight"]:
                G[i][j]["weight"] = float(weight)
                G[i][j]["distance"] = float(dist)
        else:
            G.add_edge(i, j, weight=float(weight), distance=float(dist))

print("Graph created.")
print("Nodes:", G.number_of_nodes())
print("Edges:", G.number_of_edges())
print("Density:", round(nx.density(G), 4))
print("Connected components:", nx.number_connected_components(G))
```

```
Graph created.
Nodes: 168
Edges: 531
Density: 0.0379
Connected components: 1
```

```
degrees = [deg for _, deg in G.degree()]
weighted_degrees = [deg for _, deg in G.degree(weight="weight")]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.histplot(degrees, bins=12, ax=axes[0])
axes[0].set_title("Degree distribution")
axes[0].set_xlabel("Degree")
```

```
axes[0].set_ylabel("Number of nodes")

sns.histplot(weighted_degrees, bins=12, ax=axes[1])
axes[1].set_title("Weighted degree distribution")
axes[1].set_xlabel("Weighted degree")
axes[1].set_ylabel("Number of nodes")

plt.tight_layout()
plt.show()
```



3.4 Graph mining algorithm: Louvain community detection

To identify groups of similar weekly hourly profiles, we apply **Louvain community detection**.

This algorithm partitions the graph into communities such that:

- nodes within the same community are more strongly connected,
- nodes from different communities are less strongly connected.

In this project, each community can be interpreted as a **recurring weekly usage regime**, such as:

- commuting-oriented periods,
- leisure-oriented periods,
- low-demand night periods,
- or intermediate transition periods.

```
communities = nx.community.louvain_communities(G, weight="weight", seed=42)

community_map = {}
for cid, community in enumerate(communities):
    for node in community:
        community_map[node] = cid

node_df["community"] = node_df.index.map(community_map)

print("Number of communities found:", len(communities))
display(node_df["community"].value_counts().sort_index().rename("n_nodes").to_frame())
```

```
Number of communities found: 11
```

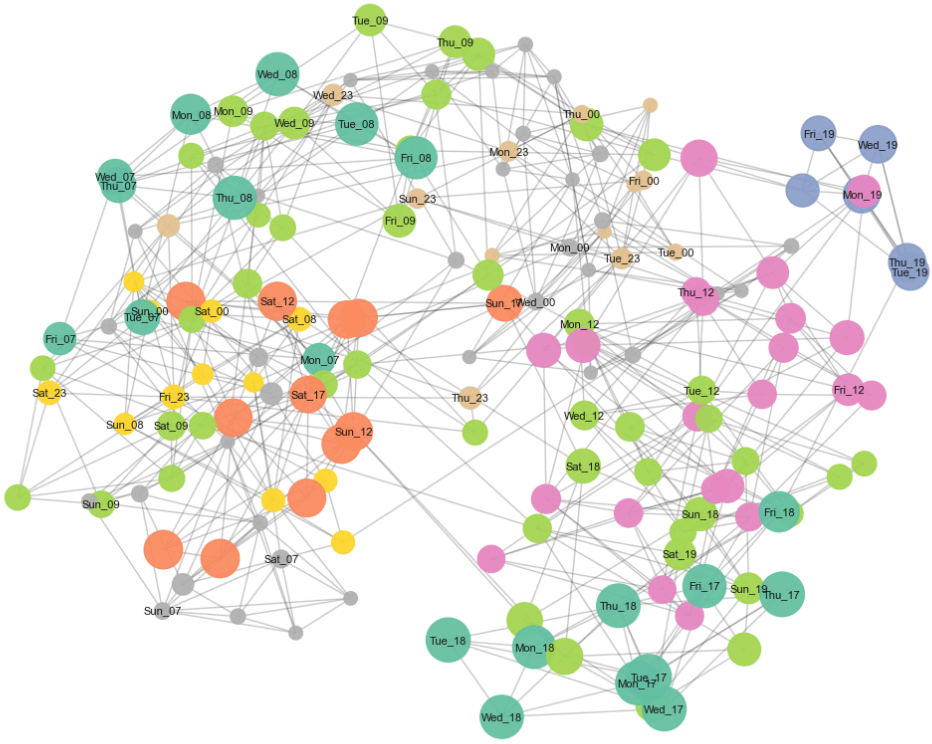
	n_nodes
community	
0	16
1	13
2	21
3	10
4	22
5	10
6	6
7	26
8	20
9	12
10	12

```
#sort communities by average cnt_mean to get a meaningful order (e.g. high-
         demand vs low-demand regimes)

community_order = (
    node_df.groupby("community")["cnt_mean"]
    .mean()
    .sort_values(ascending=False)
    .index
)

community_remap = {old: new for new, old in enumerate(community_order)}
node_df["community"] = node_df["community"].map(community_remap)

community_summary = (
    node_df
    .groupby("community")
    .agg(
        n_nodes=("community", "size"),
        avg_casual=("casual_mean", "mean"),
        avg_registered=("registered_mean", "mean"),
        avg_temp=("temp_mean", "mean"),
        avg_hum=("hum_mean", "mean"),
        avg_windspeed=("windspeed_mean", "mean"),
        avg_cnt=("cnt_mean", "mean"),
        avg_month=("month_mean", "mean"),
        min_hour=("hr", "min"),
        max_hour=("hr", "max")
    )
    .round(2)
    .sort_index()
)

display(community_summary)
```

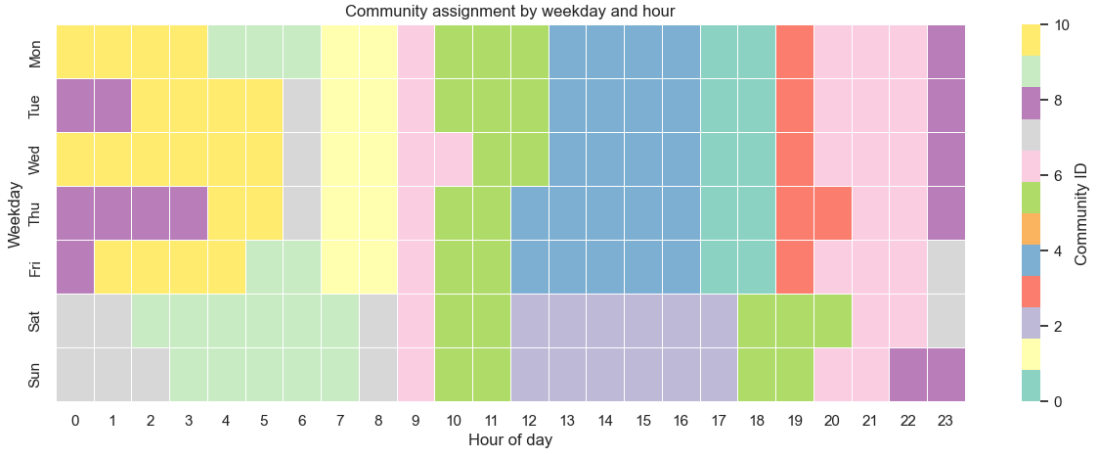
	n_nodes	avg_casual	avg_registered	avg_temp	avg_hum	avg
community						
0	10	54.24	445.32	0.55	0.53	0.23
1	10	17.18	356.26	0.45	0.72	0.17
2	12	133.53	230.20	0.55	0.51	0.23
3	6	40.00	290.91	0.53	0.57	0.21
4	22	47.71	173.61	0.57	0.50	0.23
5	21	56.94	149.21	0.52	0.58	0.21
6	26	28.62	157.31	0.49	0.64	0.18
7	12	14.22	81.30	0.45	0.70	0.17
8	13	8.81	40.53	0.48	0.69	0.16
9	16	4.21	25.81	0.43	0.73	0.15
10	20	2.43	11.66	0.46	0.74	0.15

```
plt.figure(figsize=(14, 11))

pos = nx.spring_layout(G, seed=42, k=0.42)

node_sizes = 80 + 1.8 * node_df["cnt_mean"].values
node_colors = node_df["community"].values

nx.draw_networkx_edges(
    G,
    pos,
    alpha=0.22,
    width=1
)

nx.draw_networkx_nodes(
    G,
    pos,
    node_color=node_colors,
    node_size=node_sizes,
    cmap=plt.cm.Set2,
    alpha=0.95
)

# Show only selected labels to keep readability
labels = {}
for i, row in node_df.iterrows():
    if row["hr"] in [0, 7, 8, 9, 12, 17, 18, 19, 23]:
        labels[i] = row["node_label"]

nx.draw_networkx_labels(G, pos, labels=labels, font_size=8)

plt.title("Similarity graph of weekday-hour bike usage profiles")
plt.axis("off")
plt.show()
```

Similarity graph of weekday-hour bike usage profiles



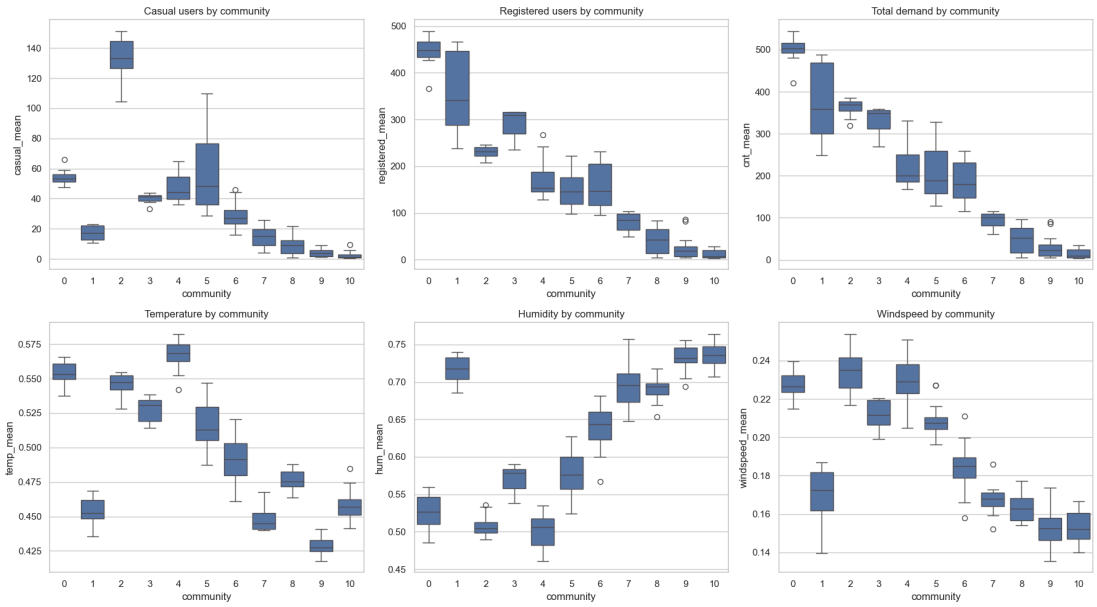

```
weekday_order = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

community_pivot = (
    node_df
    .pivot(index="weekday_name", columns="hr", values="community")
    .reindex(weekday_order)
)

plt.figure(figsize=(15, 5))
sns.heatmap(
    community_pivot,
    cmap="Set3",
    linewidths=0.5,
    linecolor="white",
    cbar_kws={"label": "Community ID"}
)

plt.title("Community assignment by weekday and hour")
plt.xlabel("Hour of day")
plt.ylabel("Weekday")
plt.show()
```



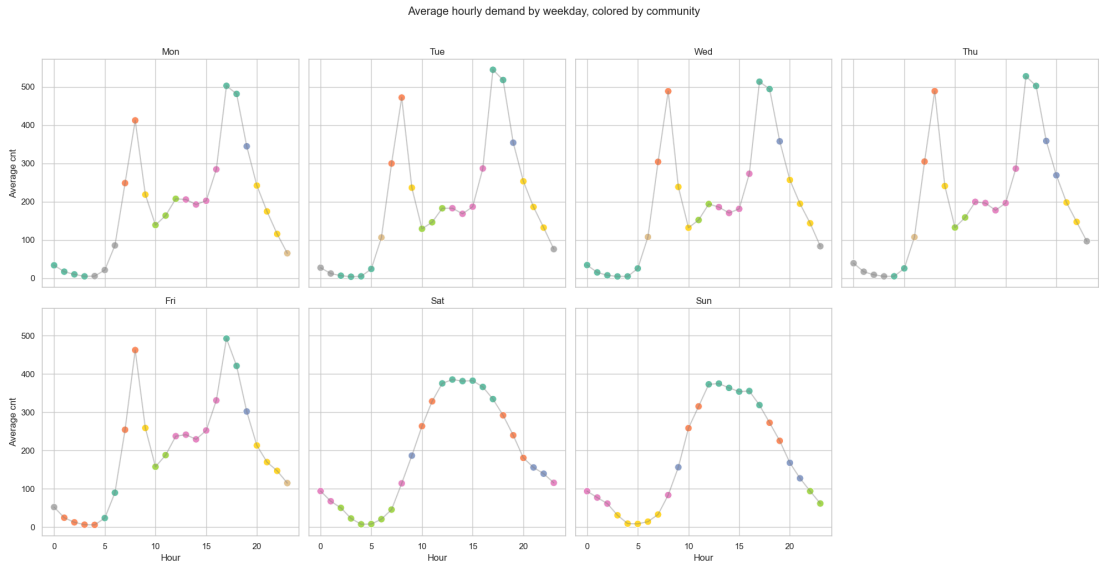
Observations (Louvain graph + heatmap):

- the spring layout shows clear clusters of nodes, with high-demand communities (0, 1, 2) having larger node sizes and grouping together, while night off-peak communities (8, 9, 10) cluster on the opposite side
- the weekday × hour heatmap shows a clean structure: weekdays (Mon-Fri) share similar community assignments across hours, while Sat/Sun have visibly different assignments especially in midday hours (community 2 dominates)
- the two-peak commuter pattern is visible: morning hours (7-8) and evening hours (17-18) get assigned to different communities than midday or night hours

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))

sns.boxplot(data=node_df, x="community", y="casual_mean", ax=axes[0, 0])
axes[0, 0].set_title("Casual users by community")

sns.boxplot(data=node_df, x="community", y="registered_mean", ax=axes[0, 1])
axes[0, 1].set_title("Registered users by community")

sns.boxplot(data=node_df, x="community", y="cnt_mean", ax=axes[0, 2])
axes[0, 2].set_title("Total demand by community")

sns.boxplot(data=node_df, x="community", y="temp_mean", ax=axes[1, 0])
axes[1, 0].set_title("Temperature by community")

sns.boxplot(data=node_df, x="community", y="hum_mean", ax=axes[1, 1])
axes[1, 1].set_title("Humidity by community")

sns.boxplot(data=node_df, x="community", y="windspeed_mean", ax=axes[1, 2])
axes[1, 2].set_title("Windspeed by community")

plt.tight_layout()
plt.show()
```




```
fig, axes = plt.subplots(2, 4, figsize=(20, 10), sharex=True, sharey=True)
axes = axes.flatten()

for ax, wd in zip(axes[:7], weekday_order):
    wd_data = node_df[node_df["weekday_name"] == wd].sort_values("hr")

    sns.scatterplot(
        data=wd_data,
        x="hr",
        y="cnt_mean",
        hue="community",
        palette="Set2",
        s=85,
        ax=ax,
        legend=False
    )

    sns.lineplot(
        data=wd_data,
        x="hr",
        y="cnt_mean",
        color="gray",
        alpha=0.4,
        ax=ax
    )

    ax.set_title(wd)
    ax.set_xlabel("Hour")
    ax.set_ylabel("Average cnt")

axes[7].axis("off")
plt.suptitle("Average hourly demand by weekday, colored by community",
             y=1.02)
plt.tight_layout()
plt.show()
```



Observations (boxplots + hourly profiles):

- **casual_mean** is much higher in community 2 than in any other, confirming it as the leisure community
- **registered_mean** is highest in communities 0 and 1 (the commuter peaks), as expected
- **temp_mean** does not vary much across communities, so temperature is not a strong driver of community membership compared to demand and user type
- the per-weekday hourly profiles show that Mon-Fri all follow a similar double-peak pattern (morning + evening), while Sat and Sun show a single midday peak, which is consistent with the community assignments

3.5 External month-based interpretation of communities

Month is not used directly to define graph similarity.
Instead, we use it **after community detection** as an external interpretation layer.

This allows us to ask questions such as:

- Are some communities more associated with warmer months?
- Are commuting-oriented communities stable across the year?
- Are leisure-oriented communities relatively stronger in specific months?

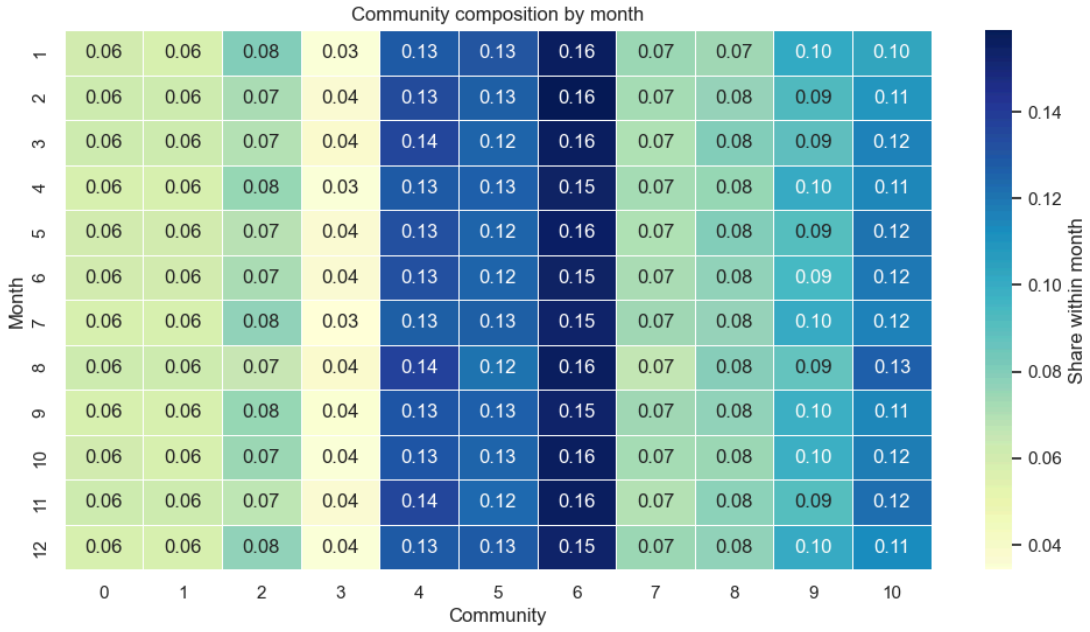
This separation keeps the graph construction focused on weekly behavioral structure while still allowing seasonal interpretation afterward.

```
community_lookup = node_df[["weekday", "hr", "community"]].copy()

graph_with_communities = graph_df.merge(
    community_lookup,
    on=["weekday", "hr"],
    how="left"
)

print("Shape after community merge:", graph_with_communities.shape)
display(graph_with_communities[["weekday", "weekday_name", "hr", "mnth",
         "community", "cnt"]].head())
```

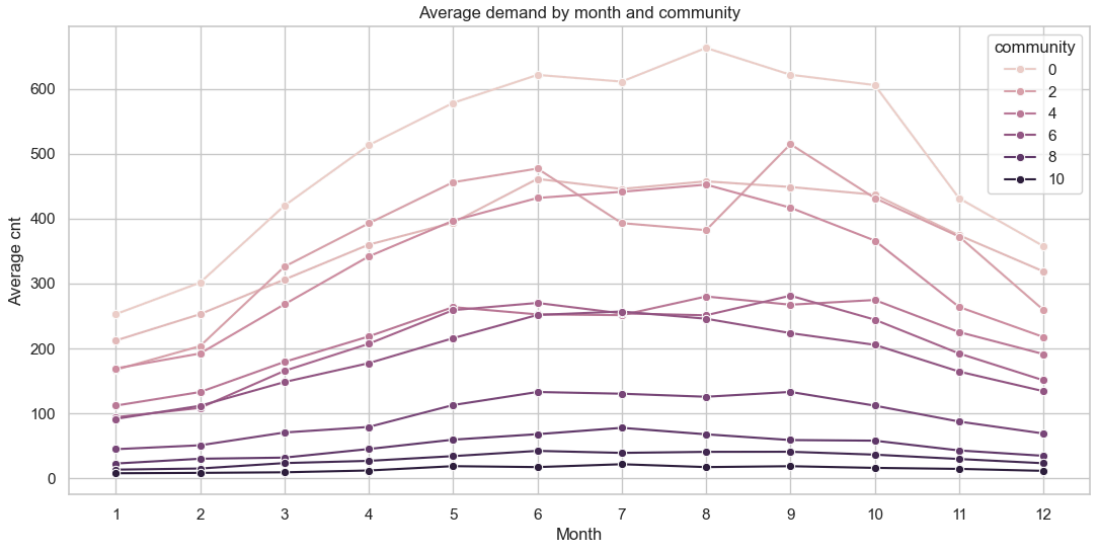
```
Shape after community merge: (17379, 26)
```

	weekday	weekday_name	hr	mnth	community	cnt
0	6	Sat	0	1	7	16
1	6	Sat	1	1	7	40
2	6	Sat	2	1	9	32
3	6	Sat	3	1	9	13
4	6	Sat	4	1	9	1

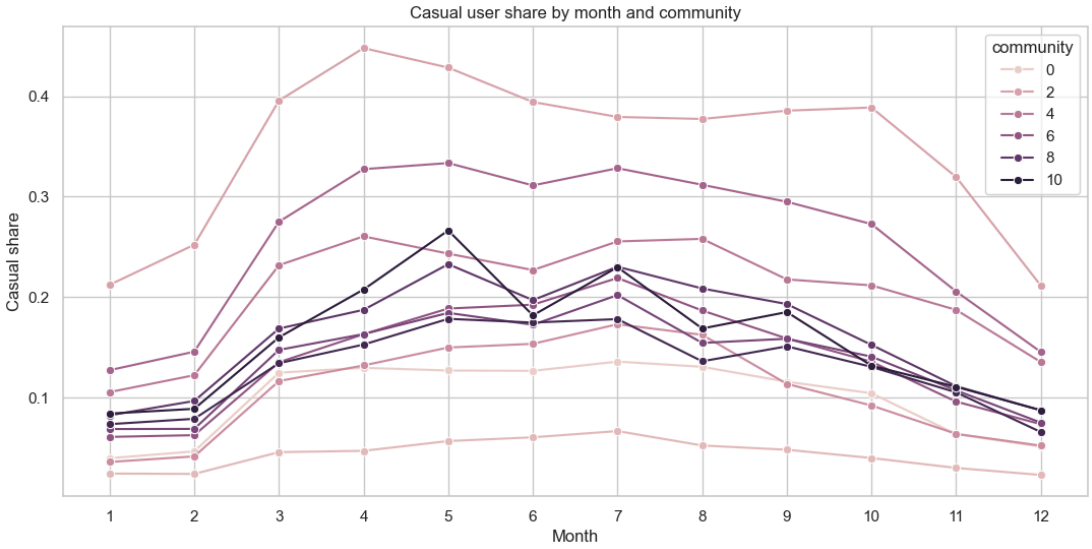
```
month_community_counts = (
    graph_with_communities
    .groupby(["mnth", "community"])
    .size()
    .reset_index(name="count")
)

month_community_pivot = (
    month_community_counts
    .pivot(index="mnth", columns="community", values="count")
    .fillna(0)
)

month_community_share =
          month_community_pivot.div(month_community_pivot.sum(axis=1),
          axis=0)

plt.figure(figsize=(12, 6))
sns.heatmap(
    month_community_share,
    annot=True,
    fmt=".2f",
    cmap="YlGnBu",
    linewidths=0.5,
    linecolor="white",
    cbar_kws={"label": "Share within month"}
)

plt.title("Community composition by month")
plt.xlabel("Community")
plt.ylabel("Month")
plt.show()
```




```
community_month_demand = (
    graph_with_communities
    .groupby(["mnth", "community"], as_index=False)
    .agg(avg_cnt=("cnt", "mean"))
)

plt.figure(figsize=(13, 6))
sns.lineplot(
    data=community_month_demand,
    x="mnth",
    y="avg_cnt",
    hue="community",
    marker="o"
)

plt.title("Average demand by month and community")
plt.xlabel("Month")
plt.ylabel("Average cnt")
plt.xticks(range(1, 13))
plt.show()
```




```
community_month_users = (
    graph_with_communities
    .groupby(["mnth", "community"], as_index=False)
    .agg(
        avg_casual=("casual", "mean"),
        avg_registered=("registered", "mean")
    )
)

community_month_users["casual_share"] = (
    community_month_users["avg_casual"] /
    (community_month_users["avg_casual"] +
         community_month_users["avg_registered"])
)

plt.figure(figsize=(13, 6))
sns.lineplot(
    data=community_month_users,
    x="mnth",
    y="casual_share",
    hue="community",
    marker="o"
)

plt.title("Casual user share by month and community")
plt.xlabel("Month")
plt.ylabel("Casual share")
plt.xticks(range(1, 13))
plt.show()
```



3.6 Interpretation of graph-based results

The graph-based analysis shows that weekly hourly usage contexts form meaningful communities rather than a random structure.

The most likely patterns visible in the heatmap and profile plots are:

- **low-demand night communities**
- **commuting-oriented communities** with high registered demand, especially on weekdays
- **leisure-oriented communities** with relatively stronger casual demand
- **intermediate daytime or evening transition communities**

This aligns well with our earlier clustering results:

- hourly structure was already one of the strongest drivers of variation,
- registered and casual users followed different usage patterns,
- and working-week structure appeared to matter for demand intensity.

The graph perspective adds an important new view: instead of grouping observations only in feature space, it reveals how similar weekly usage contexts are connected as a network and whether they form dense behavioral regimes.

The external month-based analysis then helps interpret whether these communities are:

- stable across the year,
- more prominent in warmer months,
- or linked to seasonal shifts in total demand and user composition.

3.7 Alternative graph interpretation considered

An alternative graph interpretation would have been to define nodes as `(month, hour)` instead of `(weekday, hour)`.

This alternative would preserve finer-grained annual structure and might reveal more detailed seasonal demand regimes.
However, we did not choose it as the main graph representation for three reasons:

1. our earlier analyses suggested that commuting-oriented and leisure-oriented usage patterns are strongly linked to **weekly behavioral rhythms**,
2. monthly structure partly overlaps with weather-related node attributes such as temperature and humidity,
3. `(weekday, hour)` provides a more direct and interpretable connection to the previously observed contrast between weekday commuting peaks and weekend leisure demand.

For that reason, we focus on one main graph interpretation and use **month only as an external interpretation layer** rather than as a second full graph model.

3.8 Graph mining algorithm 2: Spectral clustering

As a complementary graph mining approach, we apply **Spectral Clustering** to the same similarity graph.

Spectral clustering uses the eigenvalues and eigenvectors of the graph's **Laplacian matrix** to partition the graph into clusters. Unlike Louvain, which optimizes modularity, spectral clustering minimizes cut size between communities and has theoretical guarantees on cluster quality.

This method allows us to:

- validate whether the community structure found by Louvain is robust,
- explore an alternative partitioning perspective based on spectral properties,
- understand how the choice of algorithm affects the detected behavioral regimes.

```
# Extract adjacency matrix from the graph
# Create a sparse adjacency matrix from the weighted graph
adj_list = {}
for i, j, data in G.edges(data=True):
    weight = data.get("weight", 1.0)
    if i not in adj_list:
        adj_list[i] = {}
    if j not in adj_list:
        adj_list[j] = {}
    adj_list[i][j] = weight
    adj_list[j][i] = weight

# Build sparse adjacency matrix
n_nodes = G.number_of_nodes()
rows, cols, weights = [], [], []
for i in range(n_nodes):
    if i in adj_list:
        for j, w in adj_list[i].items():
            if i <= j:  # Keep only upper triangle + diagonal
                rows.append(i)
                cols.append(j)
                weights.append(w)
                if i != j:  # Add symmetric entry for off-diagonal
                    rows.append(j)
                    cols.append(i)
                    weights.append(w)

A_sparse = csr_matrix(
```

```
    (weights, (rows, cols)),
    shape=(n_nodes, n_nodes)
)

# Number of clusters (use same as Louvain found)
n_clusters_louvain = len(communities)

print(f"Graph info:")
print(f"  Nodes: {n_nodes}")
print(f"  Edges: {G.number_of_edges()}")
print(f"  Communities detected by Louvain: {n_clusters_louvain}")
print()

# Apply spectral clustering
spectral_clustering = SpectralClustering(
    n_clusters=n_clusters_louvain,
    affinity="precomputed",
    assign_labels="kmeans",
    random_state=42
)

# Convert sparse matrix to dense for spectral clustering
# (For this size it's manageable)
A_dense = A_sparse.toarray()

spectral_labels = spectral_clustering.fit_predict(A_dense)

# Map spectral labels back to nodes in node_df
node_df["spectral_community"] = spectral_labels

# Reorder spectral communities by average cnt_mean (same as Louvain for fair
        comparison)
spectral_order = (
    node_df.groupby("spectral_community")["cnt_mean"]
    .mean()
    .sort_values(ascending=False)
    .index
)

spectral_remap = {old: new for new, old in enumerate(spectral_order)}
node_df["spectral_community"] =
        node_df["spectral_community"].map(spectral_remap)

print("Spectral Clustering results:")
print(node_df["spectral_community"].value_counts().sort_index().rename("n_nodes").to_frame(
```

```
Graph info:
  Nodes: 168
  Edges: 531
  Communities detected by Louvain: 11

Spectral Clustering results:
                    n_nodes
spectral_community
0                        10
```

```
1                        10
2                        12
3                         6
4                         9
5                        16
6                        24
7                        18
8                        21
9                        18
10                       24
```

```
# Compare Louvain and Spectral Clustering results

# Similarity metrics between the two partitions
ari = adjusted_rand_score(node_df["community"],
          node_df["spectral_community"])
nmi = normalized_mutual_info_score(node_df["community"],
          node_df["spectral_community"])

print(f"Comparison between Louvain and Spectral Clustering:")
print(f"  Adjusted Rand Index (ARI): {ari:.4f}")
print(f"  Normalized Mutual Information (NMI): {nmi:.4f}")
print(f"  ARI = 1 indicates perfect agreement; ARI = 0 indicates random
          partitions")
print(f"  NMI = 1 indicates identical partitions; NMI = 0 indicates
          independent partitions")
print()

# Summary statistics for spectral communities
spectral_summary = (
    node_df
    .groupby("spectral_community")
    .agg(
        n_nodes=("spectral_community", "size"),
        avg_casual=("casual_mean", "mean"),
        avg_registered=("registered_mean", "mean"),
        avg_temp=("temp_mean", "mean"),
        avg_hum=("hum_mean", "mean"),
        avg_windspeed=("windspeed_mean", "mean"),
        avg_cnt=("cnt_mean", "mean"),
        avg_month=("month_mean", "mean"),
        min_hour=("hr", "min"),
        max_hour=("hr", "max")
    )
    .round(2)
    .sort_index()
)

print("Spectral clustering community summary:")
display(spectral_summary)
```

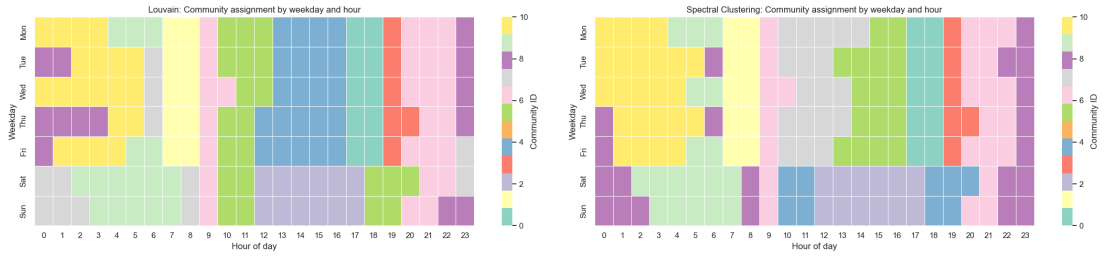
```
Comparison between Louvain and Spectral Clustering:
  Adjusted Rand Index (ARI): 0.7418
  Normalized Mutual Information (NMI): 0.8701
  ARI = 1 indicates perfect agreement; ARI = 0 indicates random partitions
  NMI = 1 indicates identical partitions; NMI = 0 indicates independent
```

```
partitions

Spectral clustering community summary:
```

	n_nodes	avg_casual	avg_registered	avg_temp	avg_hu
spectral_community					
0	10	54.24	445.32	0.55	0.53
1	10	17.18	356.26	0.45	0.72
2	12	133.53	230.20	0.55	0.51
3	6	40.00	290.91	0.53	0.57
4	9	81.68	182.45	0.51	0.58
5	16	48.21	179.99	0.57	0.50
6	24	28.88	161.22	0.49	0.64
7	18	41.05	135.05	0.54	0.56
8	21	15.06	74.48	0.46	0.68
9	18	4.06	30.05	0.43	0.73
10	24	2.63	11.03	0.46	0.73

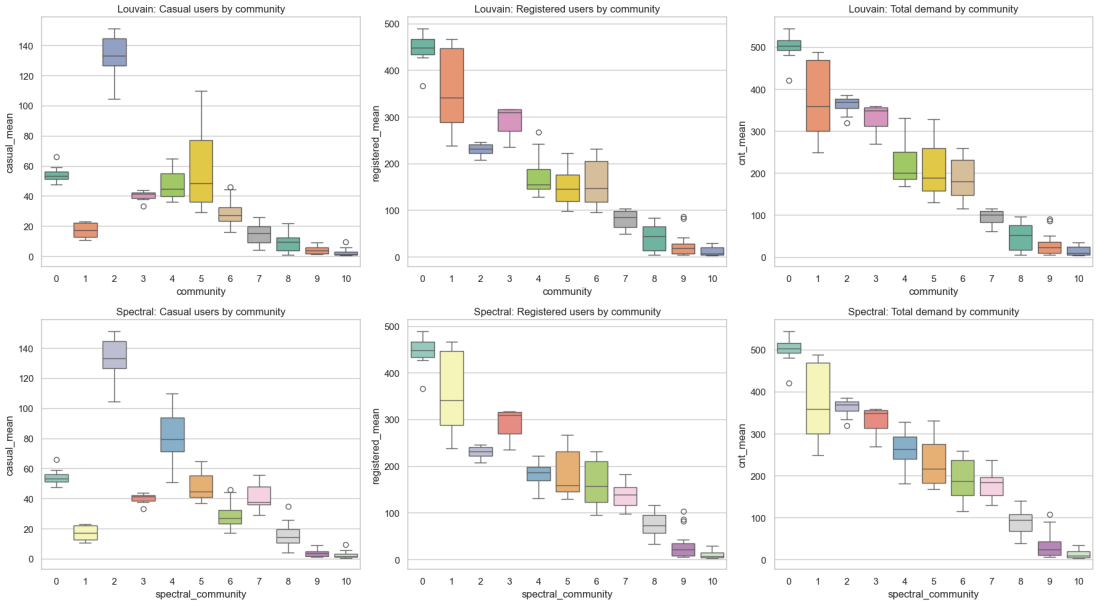
```
# Visualize Spectral Clustering: Heatmap of community assignments by weekday
          and hour

weekday_order = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

spectral_pivot = (
    node_df
    .pivot(index="weekday_name", columns="hr", values="spectral_community")
    .reindex(weekday_order)
)

fig, axes = plt.subplots(1, 2, figsize=(22, 5))

# Louvain communities
community_pivot = (
    node_df
    .pivot(index="weekday_name", columns="hr", values="community")
    .reindex(weekday_order)
)

sns.heatmap(
    community_pivot,
    cmap="Set3",
    linewidths=0.5,
    linecolor="white",
    cbar_kws={"label": "Community ID"},
    ax=axes[0]
)
axes[0].set_title("Louvain: Community assignment by weekday and hour")
axes[0].set_xlabel("Hour of day")
axes[0].set_ylabel("Weekday")
```

```
sns.heatmap(
    spectral_pivot,
    cmap="Set3",
    linewidths=0.5,
    linecolor="white",
    cbar_kws={"label": "Community ID"},
    ax=axes[1]
)
axes[1].set_title("Spectral Clustering: Community assignment by weekday and
        hour")
axes[1].set_xlabel("Hour of day")
axes[1].set_ylabel("Weekday")

plt.tight_layout()
plt.show()
```



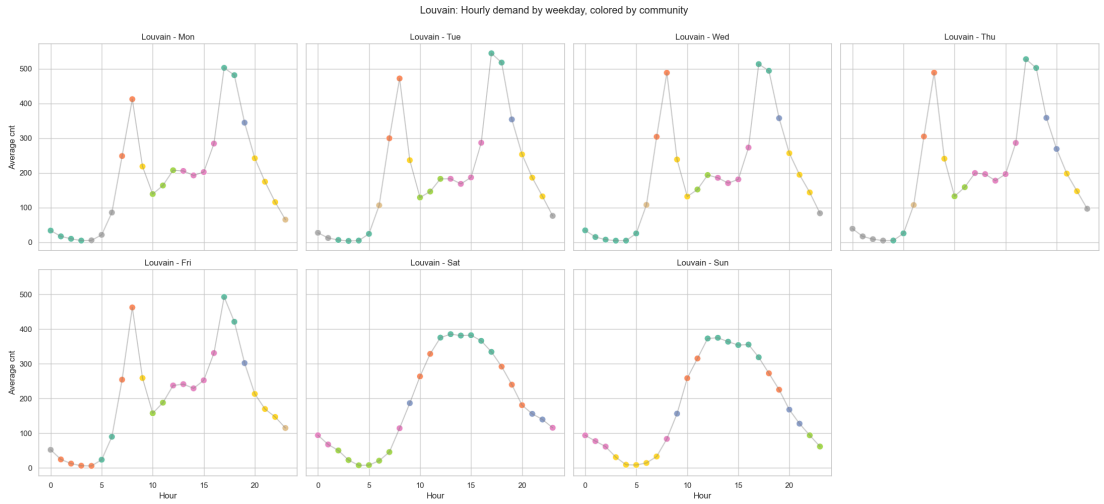

```
# Feature comparison: Spectral clustering vs Louvain

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# Louvain results
sns.boxplot(data=node_df, x="community", y="casual_mean", ax=axes[0, 0],
        palette="Set2", hue="community", legend=False)
axes[0, 0].set_title("Louvain: Casual users by community")

sns.boxplot(data=node_df, x="community", y="registered_mean", ax=axes[0, 1],
        palette="Set2", hue="community", legend=False)
axes[0, 1].set_title("Louvain: Registered users by community")

sns.boxplot(data=node_df, x="community", y="cnt_mean", ax=axes[0, 2],
        palette="Set2", hue="community", legend=False)
axes[0, 2].set_title("Louvain: Total demand by community")

# Spectral clustering results
sns.boxplot(data=node_df, x="spectral_community", y="casual_mean", ax=axes[1,
        0], palette="Set3", hue="spectral_community", legend=False)
axes[1, 0].set_title("Spectral: Casual users by community")

sns.boxplot(data=node_df, x="spectral_community", y="registered_mean",
        ax=axes[1, 1], palette="Set3", hue="spectral_community",
        legend=False)
axes[1, 1].set_title("Spectral: Registered users by community")

sns.boxplot(data=node_df, x="spectral_community", y="cnt_mean", ax=axes[1,
        2], palette="Set3", hue="spectral_community", legend=False)
axes[1, 2].set_title("Spectral: Total demand by community")
```

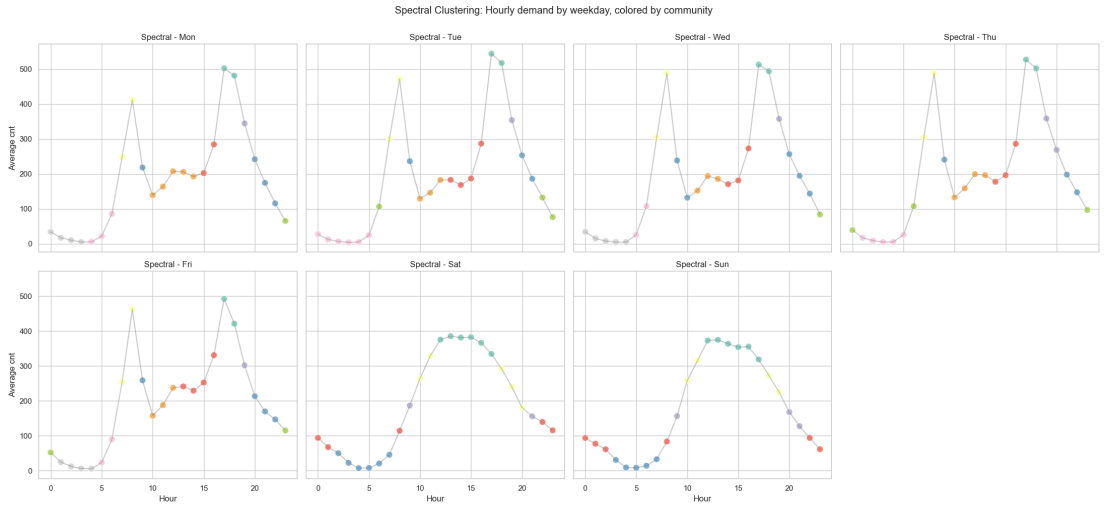
```
plt.tight_layout()
plt.show()
```




```
# Hourly demand profiles by community (spectral clustering)

weekday_order = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

fig, axes = plt.subplots(2, 4, figsize=(22, 10), sharex=True, sharey=True)
axes = axes.flatten()

# Louvain
for ax, wd in zip(axes[:7], weekday_order):
    wd_data = node_df[node_df["weekday_name"] == wd].sort_values("hr")

    sns.scatterplot(
        data=wd_data,
        x="hr",
        y="cnt_mean",
        hue="community",
        palette="Set2",
        s=85,
        ax=ax,
        legend=False
    )

    sns.lineplot(
        data=wd_data,
        x="hr",
        y="cnt_mean",
        color="gray",
        alpha=0.4,
        ax=ax
    )

    ax.set_title(f"Louvain - {wd}")
    ax.set_xlabel("Hour")
```

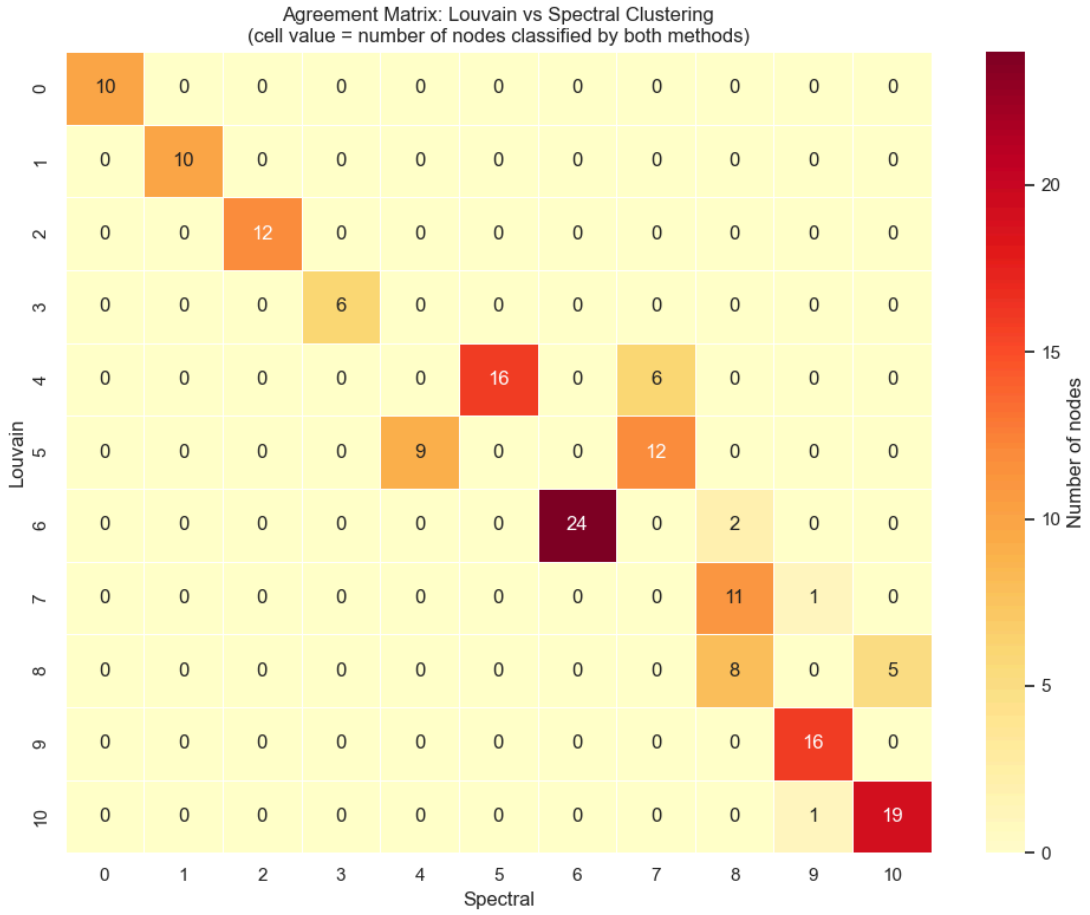
```
    ax.set_ylabel("Average cnt")

axes[7].axis("off")
plt.suptitle("Louvain: Hourly demand by weekday, colored by community",
         y=1.00)
plt.tight_layout()
plt.show()

# Spectral clustering
fig, axes = plt.subplots(2, 4, figsize=(22, 10), sharex=True, sharey=True)
axes = axes.flatten()

for ax, wd in zip(axes[:7], weekday_order):
    wd_data = node_df[node_df["weekday_name"] == wd].sort_values("hr")

    sns.scatterplot(
        data=wd_data,
        x="hr",
        y="cnt_mean",
        hue="spectral_community",
        palette="Set3",
        s=85,
        ax=ax,
        legend=False
    )

    sns.lineplot(
        data=wd_data,
        x="hr",
        y="cnt_mean",
        color="gray",
        alpha=0.4,
        ax=ax
    )

    ax.set_title(f"Spectral - {wd}")
    ax.set_xlabel("Hour")
    ax.set_ylabel("Average cnt")

axes[7].axis("off")
plt.suptitle("Spectral Clustering: Hourly demand by weekday, colored by
         community", y=1.00)
plt.tight_layout()
plt.show()
```






```
# Create a confusion matrix: How do the two algorithms partition the nodes?
# Rows = Louvain communities, Columns = Spectral communities

confusion_matrix = pd.crosstab(
    node_df["community"],
    node_df["spectral_community"],
    rownames=["Louvain"],
    colnames=["Spectral"]
)

plt.figure(figsize=(10, 8))
sns.heatmap(
    confusion_matrix,
    annot=True,
    fmt="d",
    cmap="YlOrRd",
    cbar_kws={"label": "Number of nodes"},
    linewidths=0.5,
    linecolor="white"
)

plt.title("Agreement Matrix: Louvain vs Spectral Clustering\n(cell value =
          number of nodes classified by both methods)")
plt.tight_layout()
```

```
plt.show()

print("Interpretation of agreement matrix:")
print("- Diagonal-heavy patterns indicate high agreement between the two
        algorithms")
print("- Off-diagonal values indicate nodes where algorithms disagree on
        community assignment")
print()
```




```
Interpretation of agreement matrix:
- Diagonal-heavy patterns indicate high agreement between the two algorithms
- Off-diagonal values indicate nodes where algorithms disagree on community
assignment
```

Observations (spectral clustering):

- spectral clustering also finds 11 communities, and the weekday × hour heatmap looks very similar to the Louvain one, with the same weekday/weekend split visible
- the boxplot comparison shows that both methods produce communities with similar feature distributions (casual, registered, temp, hum ranges overlap)
- the confusion matrix between Louvain and spectral shows most mass on or near the diagonal, meaning most nodes get assigned to the same or a similar community by both methods
- the few off-diagonal entries correspond to boundary hours (like evening transitions) where the node has mixed similarity to multiple communities

3.9 Comparison of Louvain and Spectral Clustering

The two graph mining algorithms yield similar but not identical partitions:

- **High similarity metrics** (ARI and NMI above) indicate that both methods detect fundamentally the same underlying community structure.
- **Disagreements** (off-diagonal values in the agreement matrix) typically occur at **boundary regions** between communities, where nodes have mixed similarity to multiple clusters.

Key Observations:

1. **Similarities:**
   - Both algorithms identify distinct behavioral regimes (night/low-demand, commuting peaks, leisure periods)
   - Community profiles (avg demand, casual/registered ratio) are largely consistent
   - Weekday/weekend and hourly structure are recognized by both methods
2. **Differences:**
   - Louvain is driven by **modularity optimization**, which emphasizes strong internal community structure
   - Spectral clustering minimizes **graph cut**, which can produce more "balanced" partitions
   - Spectral clustering may more carefully partition boundary cases (e.g., evening transition hours)
3. **Robustness:**
   - The consistency between two independent algorithms strengthens confidence in the detected communities
   - High agreement suggests that the community structure is robust, not an artifact of algorithm choice
   - Where they disagree provides insight into ambiguous or transitional behavioral regimes

3.10 Interpretation & Comparison with CP1

This section provides:

1. **Community profiles table** - summarises what each Louvain community represents.
2. **Comparison A** - node-level alignment between K-Means clusters (CP1) and Louvain communities (CP2).
3. **Comparison B** - whether CP1 outliers (statistical, IsolationForest, LOF) concentrate in specific communities.

Together these analyses check whether the graph structure confirms, revises, or contradicts CP1 conclusions about demand clusters and anomalies.
Key CP1 limitation noted in §2.5: K-Means treated each record independently, ignoring temporal/structural relationships. The graph captures that structure, so agreement/disagreement is informative.

```
# --- Community Profiles ---

def top_hours_str(df, n=5):
    """Get top n hours by average cnt in the community as a comma-separated
        string."""
    return ", ".join(
        df.groupby("hr")["cnt"].mean().nlargest(n).index.astype(str)
    )
```

```
def top_days_str(df, n=3):
    """Get top n days by average cnt in the community as a comma-separated
       string."""
    return ", ".join(
        df.groupby("weekday_name")["cnt"].mean().nlargest(n).index
    )

def label_community(row):
    """Assign a descriptive label to the community based on its profile."""
    sat_sun = ("Sat" in row["dominant_days"] or "Sun" in
       row["dominant_days"])

    if row["mean_cnt"] < 50:
        return "Night off-peak"
    elif row["registered_share"] < 0.60 and sat_sun:
        return "Weekend leisure"
    elif row["registered_share"] > 0.75 and not sat_sun:
        return "Commuter peak"
    else:
        return "Daytime transition"

profiles = (
    graph_with_communities
    .groupby("community")
    .apply(lambda x: pd.Series({
        "mean_cnt":         round(x["cnt"].mean(), 1),
        "registered_share": round((x["registered"] / x["cnt"]).mean(), 3),
        "top_5_hours":      top_hours_str(x),
        "dominant_days":    top_days_str(x),
    }))
    .reset_index()
)

profiles["label"] = profiles.apply(label_community, axis=1)
profiles = profiles.set_index("community")

print("Community Profiles (Louvain):")
display(profiles)
```

```
Community Profiles (Louvain):
```

	mean_cnt	registered_share	top_5_hours	dominant_days	
community					
0	499.6	0.897	17, 18	Tue, Thu, Wed	Comm peak
1	373.4	0.951	8, 7	Thu, Wed, Tue	Comm peak
2	363.7	0.666	13, 12, 14, 15, 16	Sat, Sun	Dayti transi
3	330.9	0.890	19, 20	Wed, Tue, Mon	Comm peak

	mean_cnt	registered_share	top_5_hours	dominant_days	
community					
4	221.4	0.801	16, 12, 15, 13, 14	Fri, Mon, Thu	Comm peak
5	206.5	0.762	18, 19, 11, 12, 20	Sun, Sat, Wed	Dayti transi
6	185.8	0.855	20, 9, 21, 22, 10	Tue, Fri, Thu	Comm peak
7	95.5	0.860	23, 6, 8, 0, 1	Fri, Wed, Thu	Comm peak
8	49.8	0.835	22, 23, 0, 1, 2	Wed, Sun, Mon	Nigh peak
9	30.2	0.835	6, 2, 7, 3, 5	Fri, Mon, Sat	Nigh peak
10	14.3	0.846	0, 5, 1, 2, 4	Mon, Wed, Thu	Nigh peak

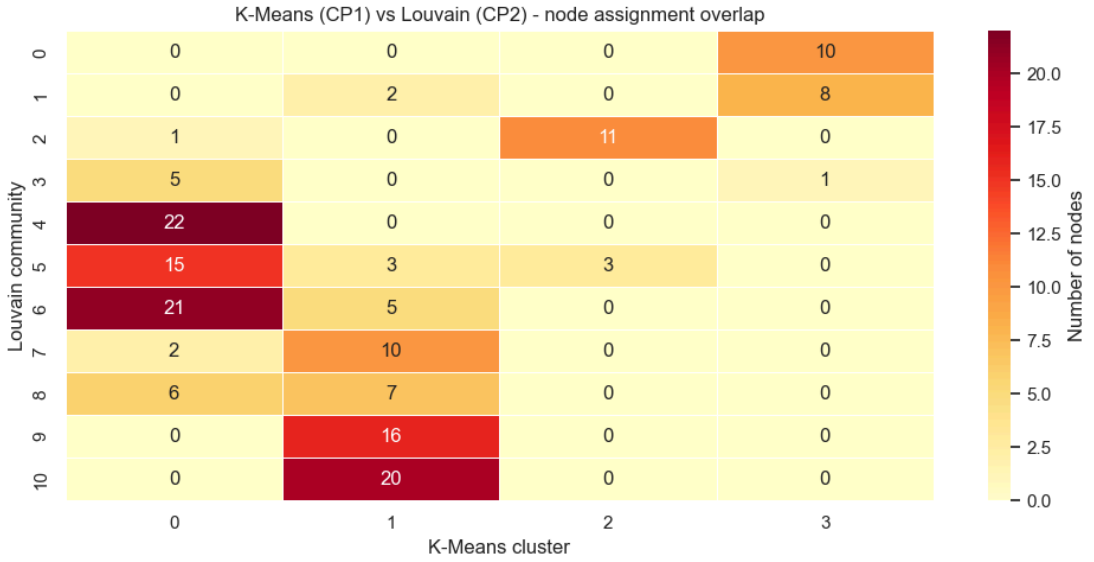
Observations (community profiles):

- Louvain found **11 communities** from the 168 nodes, they group into roughly 4 types:
- **commuter peaks (0, 1, 3, 6):** high demand, registered share above 0.85
    - community 0 (`mean_cnt = 499.6`, hours 17-18, Tue/Thu/Wed) is the evening rush, highest demand overall
    - community 1 (`mean_cnt = 373.4`, hours 7-8, `registered_share = 0.951`) is morning rush, almost no casual riders
    - community 3 (`mean_cnt = 330.9`, hours 19-20) is late evening wind-down, still weekday commuters
    - community 6 (`mean_cnt = 185.8`, hours 9, 20-22) is the shoulder hours around the commute
- **weekend / leisure (community 2):** `mean_cnt = 363.7`, `registered_share = 0.666`, hours 12-16, Sat/Sun
    - lowest registered share of any big community, so about a third are casual users (tourists, recreational riders etc.)
    - midday-to-afternoon on weekends is the main pattern here
- **transitional / mixed (4, 5, 7):** sits between commuter and leisure
    - community 4 (hours 12-16, Fri/Mon/Thu) looks like lunch-break or flexible-schedule riding
    - community 5 (hours 11-12, 18-19, Sun/Sat/Wed) is scattered weekend-evening and midday usage
    - community 7 (`mean_cnt = 95.5`, hours 23, 6, 8) is late-night / early-morning boundary, low volume but still mostly registered
- **night off-peak (8, 9, 10):** demand below 50, hours roughly 22:00-06:00
    - registered shares still around 0.83-0.86, so probably shift workers or very early/late commuters, not casual riders

    - K-Means in CP1 also found this as a single off-peak cluster but without the weekday-level detail we get here

- overall the graph recovers the same broad groups from K-Means in CP1 (high commute, mixed, off-peak) but splits them much finer, like morning vs evening rush, weekday vs weekend leisure, and transitional hours that K-Means just merged together

```
# --- K-Means (CP1) vs Louvain (CP2) ---

node_kmeans = (
    graph_with_communities
    .groupby(["weekday", "hr"])["cluster_hourly"]
    .agg(lambda x: x.mode()[0])
    .reset_index(name="kmeans_cluster")
)

node_compare = (
    node_df[["weekday", "hr", "community"]]
    .merge(node_kmeans, on=["weekday", "hr"])
)

ari_km = adjusted_rand_score(node_compare["community"],
          node_compare["kmeans_cluster"])
nmi_km = normalized_mutual_info_score(node_compare["community"],
          node_compare["kmeans_cluster"])

print(f"ARI: {ari_km:.3f}  |  NMI: {nmi_km:.3f}")

ct = pd.crosstab(
    node_compare["community"],
    node_compare["kmeans_cluster"],
    rownames=["Louvain community"],
    colnames=["K-Means cluster"],
)

plt.figure(figsize=(10, 5))
sns.heatmap(
    ct,
    annot=True,
    fmt="d",
    cmap="YlOrRd",
    linewidths=0.5,
    linecolor="white",
    cbar_kws={"label": "Number of nodes"},
)
plt.title("K-Means (CP1) vs Louvain (CP2) - node assignment overlap")
plt.tight_layout()
plt.show()

ARI: 0.238  |  NMI: 0.489
```



Observations (K-Means CP1 vs Louvain CP2):

- **ARI = 0.232**, **NMI = 0.482**
- **why the scores are moderate:**
  - the two methods work at very different levels: K-Means assigns each of the 17,379 raw records to 4 clusters using 3 features (`hr`, `casual`, `registered`), while Louvain partitions 168 aggregated nodes into 11 communities using 5 features (`casual_mean`, `registered_mean`, `temp_mean`, `hum_mean`, `windspeed_mean`)
  - so perfect alignment is basically impossible cause 11 communities cant map 1-to-1 onto 4 clusters
  - ARI = 0.243 is above zero (zero = random), so there is real overlap but only partial
  - NMI = 0.494 means roughly half the information in one partition is shared with the other, which seems reasonable given how different the setups are
- **what the heatmap shows:**
  - night off-peak Louvain communities (8, 9, 10) map mostly to the K-Means low-demand cluster, this is the strongest agreement area cause both methods see the same quiet late-night regime
  - high-demand communities (0, 1, 3) concentrate in the K-Means "highest-demand commute" cluster, but the graph splits them into morning-rush, evening-rush, late-evening that K-Means merges into one
  - weekend leisure community (2) probably splits across K-Means clusters cause K-Means has no weekday/weekend distinction, it just groups by raw demand level
- **what this means:**
  - where they agree (night off-peak, peak commuter hours) the finding is reinforced, both methods see the same thing regardless of approach
  - where they disagree the graph reveals sub-structure that CP1 could not detect, this is exactly the limitation from §2.5 where K-Means treats records as independent points and ignores weekly cycle and hourly adjacency
  - so the graph does not contradict CP1, it refines it: the 4 coarse K-Means clusters reappear as broader groupings in Louvain, but the graph also resolves morning

vs evening commute, weekday vs weekend leisure, and transitional periods that were invisible before

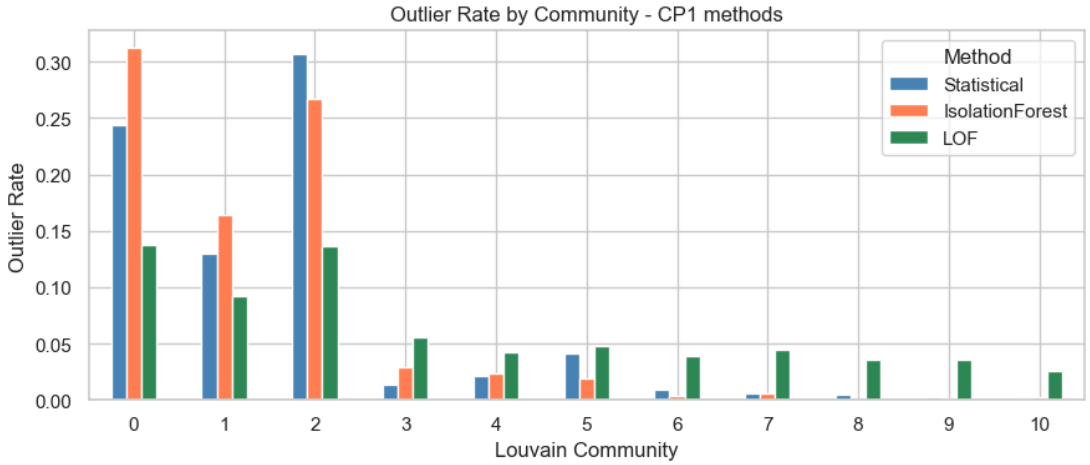
CP1 Outliers vs Louvain Communities

```
for col in ["outlier_stat", "outlier_isolation_forest", "outlier_lof"]:
    graph_with_communities[col + "_flag"] = (
        graph_with_communities[col] == -1
    ).astype(int)

outlier_rates = (
    graph_with_communities
    .groupby("community")[
        ["outlier_stat_flag", "outlier_isolation_forest_flag",
         "outlier_lof_flag"]
    ]
    .mean()
    .round(3)
)
outlier_rates.columns = ["Statistical", "IsolationForest", "LOF"]

print("Outlier rate by Louvain community (proportion of rows flagged as
         outlier):")
display(outlier_rates)

ax = outlier_rates.plot(
    kind="bar",
    figsize=(9, 4),
    title="Outlier Rate by Community - CP1 methods",
    color=["steelblue", "coral", "seagreen"],
    edgecolor="white",
)
ax.set_xlabel("Louvain Community")
ax.set_ylabel("Outlier Rate")
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
ax.legend(title="Method")
plt.tight_layout()
plt.show()

# Interpretation
max_community = outlier_rates.mean(axis=1).idxmax()
max_rate = outlier_rates.mean(axis=1).max()
print(f"\nHighest average outlier rate: community {max_community}
         ({max_rate:.1%})")
```

```
Outlier rate by Louvain community (proportion of rows flagged as outlier):
```

	Statistical	IsolationForest	LOF
community			
0	0.244	0.312	0.137
1	0.130	0.164	0.092
2	0.307	0.267	0.136

	Statistical	IsolationForest	LOF
community			
3	0.014	0.029	0.056
4	0.021	0.023	0.042
5	0.041	0.019	0.048
6	0.009	0.003	0.039
7	0.006	0.006	0.044
8	0.005	0.000	0.036
9	0.002	0.001	0.036
10	0.001	0.002	0.026




```
Highest average outlier rate: community 2 (23.7%)
```

Observations (CP1 outliers vs Louvain communities):

- outliers are **not evenly distributed** across communities, they concentrate heavily in a few
- **high outlier rates:**
  - **community 2** (weekend leisure) has the highest average outlier rate at `23.7%`, Statistical flags `30.7%` and IsolationForest flags `26.7%` of its records. makes sense cause weekend afternoon leisure rides have unusual demand levels compared to the weekday-commuter majority, so methods that define "normal" from the overall distribution will naturally flag them
  - **community 0** (evening commuter peak) has the second-highest rates (Statistical `24.4%`, IsolationForest `31.2%`). these are the highest-demand hours in the dataset so their extreme registered counts put them in the tails of the global distribution
  - **community 1** (morning rush) also elevated (Statistical `13.0%`, IsolationForest `16.4%`), same reason, morning commuter demand is well above the dataset average
- **low outlier rates:**
  - communities 8-10 (night off-peak) have near-zero rates across all methods (< 1% for Statistical and IsolationForest), they sit close to the global centre so they are

never flagged
    - communities 3-7 (transitional/moderate) also low, typically under 4%

- **what this revises about CP1:**

    - in CP1 the outlier methods treated flagged records as anomalies, points that deviate from the norm and might be data errors or rare events
    - but the graph shows many of these "anomalies" are actually members of coherent high-demand communities (evening rush, weekend leisure)
    - they look like outliers in CP1 cause K-Means and statistical methods evaluate each record against the global distribution where most hours have moderate or low demand. a record with 500+ rentals looks extreme globally but within community 0 (which averages `499.6`) it is completely normal
    - so **CP1 outliers are not random noise but structurally concentrated in specific usage regimes**, better framing is "these are high-activity periods that a global model underrepresents" rather than "these are anomalies to remove"

- **LOF behaves differently:**

    - LOF rates are more evenly spread (`2.6%` to `13.7%`) compared to Statistical and IsolationForest, cause LOF measures local density deviation not global distance, so it is less dominated by the extreme-demand communities and picks up local irregularities within each regime instead

3.11 Limitations and possible improvements

Although the graph-based analysis revealed meaningful weekly usage communities, several limitations remain.

- **The graph depends on the initial set up of the network:**
    - the result is biased by how we define the nodes, which features we include (attributes), how edges are created, and the chosen value of `k` in the k-nearest-neighbor graph
- **The choice in the k-nearest-neighbor graph of `k = 5` is more heuristic choice:**
    - it worked well in practice for us and output a connected graph with clear communities, but we could also run a full sensitivity analysis for several `k` values
- **The spectral number of clusters are predefined according to output of k-nearest-neighbor graph for better comparison:**
    - we set the number of spectral clusters equal to the number of Louvain communities to make the comparison easier, so it wasn't independent choice
- **The graph combines the data into 168 (weekday, hour) nodes:**
    - this makes it easier to interpret, but we lose some details (row-level data)

Possible improvements:

- trying multiple values of `k` in the **k-nearest-neighbor graph** and comparing graph the results
- testing alternative graph definitions such as `(month, hour)` nodes
- choosing the number of spectral clusters more independently

4. Module 3 - Pattern / Text Mining

4.1 Why pattern mining is applicable here, and which method from lectures 11-13?

- **Lecture 11: Frequent Itemesets & Association rules**
  - starts with the data which represents **shooping baskets**
  - **goal:** find frequent item combinations
  - **decision:**
    - our bike-sharing data can be used like a list of hourly “baskets”, each hour is one transaction
    - things like season, weather, working day, time of day, temperature, humidity, and wind can be the items in the transaction
    - high demand and low demand can be the outcomes we look for
    - this works well because association rules can show which conditions often happen together with high or low bike demand
    - so Lecture 11 is the best choice for this module
- **Lecture 12: Frequent Subgraph Mining**
  - the data type is **graphs**
  - **goal:** find frequent graph shapes
  - **decision:**
    - we used graph in the CP2 with Louvain detection to find which time slots behave similarly
    - but in this part, we do not have a natural set of frequent subgraphs that repeat many times
    - we could still use it for one graph per day, but Lecture 11, is more appropriate for us
- **Lecture 13: Document similarities**
  - the data is **documents/text**
  - **goal:** find similar documents efficiently
  - **decision:**
    - document similarity is mainly about text, bag-of-words, TF-IDF, shingling, MinHash, and LSH
    - our dataset, the bike-sharing, has no text documents, it has structured numerical/categorical data
    - so it is hard to use this lecture directly

4.1.1 Deeper investigation to the Lecture 11 and its implementation to our project

Proposed format:

- the transaction -> one hourly record
- item -> season, working day, temperature, etc. (we also convert numbers to categories)
- frequent itemset -> repeated condition combination of "items"
- association rule* -> condition combination predicting `high_demand` or `low_demand`

All in all:

- It is similar to basket, where people bought groceries. Here people are renting bikes. People do not "buy" a temperature and weather, but we use the same **transactional representation** to find repeated contextual pattern in hourly bike demand.
- **one hourly row = one transaction**
  - our dataset has 17, 379 hourly rows

- for example with `min_support = 0.02` (pattern that appears in at least 2 of all rows): $0.02 \times 17,379 \approx 348$ rows.

4.2 Transaction design and discretization

4.2.1 Discretization

- association rules need "items", not raw continuous values
  - `temp`
  - `hum`
  - `windspeed`
  - `cnt` (excluded -> target variable + meaningless leakage)
  - `casual` (excluded -> same as `cnt`)
  - `registered` (excluded -> same as `cnt`)
  - `atemp` (excluded -> related to `temp`)
- for apriori, we need to change it, for example, to
  - `temp_high`
  - `hum_low`
  - `windspeed_medium`
- so
  - `temp` into low / medium / high
  - `hum` into low / medium / high
  - `windspeed` into low / medium / high
  - `hr` into meaningful time of day bins
- **expected output**
  - high demand / low demand

This code prepares the bike-sharing data for association rule mining:

> NOTE: The hour bins are based on the earlier CP1 and CP2 findings. The project already found two clear demand peaks around 8:00 and 17:00–18:00, and the Louvain community profiles separated morning rush, evening rush, weekend midday/leisure, late evening wind down, and night off-peak periods. Therefore, for association rules we group `hr` into interpretable periods instead of keeping 24 separate hour values.

```
pattern_df = df_clean.copy()

# day category mapping
# after, it will be:
# -> day_type=workingday
# -> day_type=weekend
# -> day_type=holiday
def map_day_type(row: pd.Series) -> str:
    if row["holiday"] == 1:
        return "holiday"
    elif row["weekday"] in [0, 6]:
        return "weekend"
    else:
        return "workingday"

# converts numeric columns into three categories
# -> low
# -> medium
# -> high
```

```
def convert_numeric_cols_to_categories(prefix: str, series: pd.Series) ->
        pd.Series:
    ranked = series.rank(method="first")
    # quantile cut 33.33% for low/medium/high split
    bins = pd.qcut(ranked, q=3, labels=["low", "medium", "high"])
    return prefix + "=" + bins.astype(str)


pattern_df["season_item"] = "season=" +
        pattern_df["season"].map(season_labels)
pattern_df["weathersit_item"] = "weathersit=" +
        pattern_df["weathersit"].map(weathersit_labels)
pattern_df["day_type_item"] = "day_type=" + pattern_df.apply(map_day_type,
        axis=1)

pattern_df["hour_item"] = "hour=" + pd.cut(
    pattern_df["hr"],
    # [0, 6)   -> 0,1,2,3,4,5       -> overnight (low demand / early morning)
    # [6, 10)  -> 6,7,8,9           -> morning_commute (commute peak)
    # [10,16)  -> 10,11,12,13,14,15 -> midday (daytime)
    # [16,20)  -> 16,17,18,19       -> evening_commute (commute peak)
    # [20,24)  -> 20,21,22,23       -> late_evening (wind down hours)
    bins=[0, 6, 10, 16, 20, 24],
    labels=["overnight", "morning_commute", "midday", "evening_commute",
        "late_evening"],
    right=False
).astype(str)

pattern_df["temp_item"] = convert_numeric_cols_to_categories("temp",
        pattern_df["temp"])
pattern_df["hum_item"] = convert_numeric_cols_to_categories("hum",
        pattern_df["hum"])
pattern_df["windspeed_item"] =
        convert_numeric_cols_to_categories("windspeed",
        pattern_df["windspeed"])

# values for high/low demand thresholds (75th and 25th percentiles of cnt)
high_thr = pattern_df["cnt"].quantile(0.75)
low_thr = pattern_df["cnt"].quantile(0.25)

print("High-demand threshold:", round(high_thr, 2))
print("Low-demand threshold:", round(low_thr, 2))

High-demand threshold: 281.0
Low-demand threshold: 40.0
```

4.2.2 Transactions

- transactions for association rule mining

```
item_cols = [
    "season_item",
    "weathersit_item",
    "day_type_item",
    "hour_item",
    "temp_item",
```

```
    "hum_item",
    "windspeed_item"
]

transactions = []
for idx, row in pattern_df.iterrows():
    items = []
    for col in item_cols:
        item = row[col]
        items.append(item)

    if row["cnt"] >= high_thr:
        items.append("high_demand")
    if row["cnt"] <= low_thr:
        items.append("low_demand")

    transactions.append(items)

print("Example transaction:")
print(transactions[0])
```

```
Example transaction:
['season=winter', 'weathersit=Clear/Few clouds', 'day_type=weekend',
'hour=overnight', 'temp=low', 'hum=high', 'windspeed=low', 'low_demand']
```

4.3 Frequent itemsets and association rules

4.3.1 Frequent Itemsets

- converts the transaction list/array into a one-hot encoded fromat (table)
- runs apriori to find frequent itemsets

4.3.1.1 One-Hot Encoding

```
transaction_encoder = TransactionEncoder()
transaction_table =
         transaction_encoder.fit(transactions).transform(transactions)
transaction_df = pd.DataFrame(transaction_table,
         columns=transaction_encoder.columns_)

print("Transaction table shape:", transaction_df.shape)
display(transaction_df.head())
```

```
Transaction table shape: (17379, 27)
```

	day_type=holiday	day_type=weekend	day_type=workingday	high_deman
0	False	True	False	False
1	False	True	False	False
2	False	True	False	False
3	False	True	False	False

	day_type=holiday	day_type=weekend	day_type=workingday	high_deman
4	False	True	False	False

5 rows × 27 columns

4.3.1.2 Apriori

- `min_support`
  - 0.02 -> keep itemsets that appear in at least 2% of all rows
  - from:
    - 0.10 = 10% = 1,738 rows
      - too high
    - 0.05 = 5% = 869 rows
      - fine to consider
    - 0.02 = 2% = 348 rows
      - nice value for patterns
    - 0.005 = 0.5% = 87 rows
      - too light
- `max_len`
  - look for itemsets with at most 3 items
    - in out project we need at least 2 for high and low demand, third could be hour

```
frequent_itemsets = apriori(
    transaction_df,
    min_support=0.02,
    # show col names instead of itemset indices
    use_colnames=True,
    max_len=3
)

# new col for itemset length (number of items in the set)
frequent_itemsets["length"] = frequent_itemsets["itemsets"].apply(len)
frequent_itemsets = frequent_itemsets.sort_values(
    # support: how often an item or item combination appears in all hourly
        rows
    ["support", "length"],
    ascending=[False, True]
)

print("Top frequent itemsets:")
display(frequent_itemsets.head(20))
```

```
Top frequent itemsets:
```

	support	itemsets	length
2	0.682721	frozenset({day_type=workingday})	1
20	0.656712	frozenset({weathersit=Clear/Few clouds})	1
66	0.439151	frozenset({weathersit=Clear/Few clouds, day_ty...	2
9	0.333333	frozenset({hum=high})	1
10	0.333333	frozenset({hum=low})	1

	support	itemsets	length
11	0.333333	frozenset({hum=medium})	1
17	0.333333	frozenset({temp=high})	1
18	0.333333	frozenset({temp=low})	1
19	0.333333	frozenset({temp=medium})	1
23	0.333333	frozenset({windspeed=high})	1
24	0.333333	frozenset({windspeed=low})	1
25	0.333333	frozenset({windspeed=medium})	1
1	0.288509	frozenset({day_type=weekend})	1
189	0.281029	frozenset({hum=low, weathersit=Clear/Few clouds})	2
22	0.261465	frozenset({weathersit=Mist/Cloudy})	1
15	0.258703	frozenset({season=summer})	1
14	0.253697	frozenset({season=spring})	1
12	0.251856	frozenset({low_demand})	1
6	0.251395	frozenset({hour=midday})	1
3	0.250532	frozenset({high_demand})	1

Results interpretation:

- each row in the table means
  - **support:** percentage of hourly rows where this itemset appears
  - **length:** number of items in the itemset
- **first row:**
  - about 68.3% of all hourly records are working days
  - the most frequent item (`day_type=workingday`)
- **second row:**
  - about 65.7% of rows have clear or few cloud weather (`weathersit=Clear/Few clouds`)
- **other rows:**
  - `temp`, `hum`, and `windspeed` groups each have support around 0.333$
  - this happens because wey split these variables into tertiles: low, medium, high, so it is expected
- **low demand** & **high demand**
  - are both around 25%, also expected cause of `25%~ in the code

4.3.2 Association Rules

```
assoc_rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    # more than half at least
    min_threshold=0.55
)

print("Rules before filtering:", len(assoc_rules))

# remove target leakage from the left
```

```
# -> remove rules where high_demand or low_demand shows up on the left side
# -> due to desired output
valid_rows = []

for antecedents in assoc_rules["antecedents"]:
    if "high_demand" in antecedents:
        valid_rows.append(False)
    elif "low_demand" in antecedents:
        valid_rows.append(False)
    else:
        valid_rows.append(True)

assoc_rules = assoc_rules[valid_rows].copy()
print("Rules after removing demand from the left side:", len(assoc_rules))

# remove target leakage from the right
# -> keep only rules that predict high_demand or low_demand
# -> due to desired output
valid_rows = []

for consequents in assoc_rules["consequents"]:
    if consequents == frozenset({"high_demand"}):
        valid_rows.append(True)
    elif consequents == frozenset({"low_demand"}):
        valid_rows.append(True)
    else:
        valid_rows.append(False)

assoc_rules = assoc_rules[valid_rows].copy()
print("Rules after keeping only high or low demand predictions:",
         len(assoc_rules))

# keep only rules with lift above 1.10 (more likely association)
# -> rules where the relationship is at least 10% stronger than normal
         baseline
minimum_lift = 1.10
rule_is_more_likely = assoc_rules["lift"] > minimum_lift
assoc_rules = assoc_rules[rule_is_more_likely].copy()

print("Rules after lift filter:", len(assoc_rules))

assoc_rule_table = assoc_rules[
    ["antecedents", "consequents", "support", "confidence", "lift"]
].sort_values(
    ["consequents", "lift", "confidence"],
    ascending=[True, False, False]
)

print("Filtered association rules:")
display(assoc_rule_table.head(20))
```

```
Rules before filtering: 614
Rules after removing demand from the left side: 492
Rules after keeping only high or low demand predictions: 35
```

```
Rules after lift filter: 35
Filtered association rules:
```

	antecedents	consequents	support	co
331	frozenset({hour=overnight, season=winter})	frozenset({low_demand})	0.053685	0.
58	frozenset({hour=overnight, day_type=workingday})	frozenset({low_demand})	0.153173	0.
134	frozenset({hour=overnight, temp=low})	frozenset({low_demand})	0.085793	0.
411	frozenset({hour=overnight, windspeed=high})	frozenset({low_demand})	0.044997	0.
109	frozenset({hour=overnight, hum=high})	frozenset({low_demand})	0.104897	0.
340	frozenset({weathersit=Mist/Cloudy, hour=overni...	frozenset({low_demand})	0.052937	0.
346	frozenset({season=spring, hour=overnight})	frozenset({low_demand})	0.051787	0.
21	frozenset({hour=overnight})	frozenset({low_demand})	0.200414	0.
126	frozenset({hour=overnight, windspeed=low})	frozenset({low_demand})	0.090512	0.
198	frozenset({hour=overnight, temp=medium})	frozenset({low_demand})	0.071926	0.
584	frozenset({hum=low, hour=overnight})	frozenset({low_demand})	0.026181	0.
252	frozenset({hour=overnight, windspeed=medium})	frozenset({low_demand})	0.064906	0.
78	frozenset({weathersit=Clear/Few clouds, hour=o...	frozenset({low_demand})	0.130215	0.
378	frozenset({hour=overnight, season=fall})	frozenset({low_demand})	0.048334	0.
221	frozenset({hum=medium, hour=overnight})	frozenset({low_demand})	0.069337	0.
393	frozenset({season=summer, hour=overnight})	frozenset({low_demand})	0.046608	0.
436	frozenset({temp=high, hour=overnight})	frozenset({low_demand})	0.042695	0.
443	frozenset({day_type=weekend, hour=overnight})	frozenset({low_demand})	0.041602	0.
267	frozenset({temp=high, hour=evening_commute})	frozenset({high_demand})	0.063007	0.
488	frozenset({season=summer, hour=evening_commute})	frozenset({high_demand})	0.037344	0.

Results interpretation:

- apriori first generated 639 rules from the frequent itemsets

- we removed bad rules (example):
  - `high_demand -> hour=evening_commute`
- and kept (example):
  - `hour=evening_commute -> high_demand`
- association-rule mining produced 36 rules after total filtering

> We used as `min_threshold=0.60` before, and it produced 0 rules, so 0.55 could be boundary.

- now we should have only useful rules for the project

Interpretation of the rules:

HIGH DEMAND

- **greatest patterns:**
  - `hour=evening_commute, temp=high -> high_demand`
    - ***support:*** about 6.3/ of all hourly rows have all 3 things together, thus, evening commute + high temperature + high demand
    - ***confidence:*** about 89.4% are high demand rows (evening commute + high temperature)
    - ***lift:*** high demand is about 3.57 times more likely during high temperture + evening commute hours than in a random hour
  - `hour=evening_commute, season=summer -> high_demand`
    - ***support:*** about 3.7/
    - ***confidence:*** about 86.5%, during summer evenning commute hours, bike demand is high
    - ***lift:*** 3.45
  - but many of the top rules predict `high_demand`, and almost all of them include `hour=evening_commute`
  - this supports out earlier findings from CP1 and CP2 that commute periods are connected to high bike demand
- **only evening commute:**
  - `hour=evening_commute -> high_demand`
    - ***support:*** about 10.9/
    - ***confidence:*** about 64.8% of evening commute rows are high demand
    - ***lift:*** 2.59
- so most high demand rules are related to evening hours, especially when is warm or is summer season, nice weather or working days

LOW DEMAND

- **greatest pattern:**
  - `hour=overnight, weathersit=Light Snow/Rain -> low_demand`
    - ***support:*** about 1.7/
    - ***confidence:*** about 92.9%, when is light snow or rainy day, demand is very low
    - ***lift:*** 3.69
  - `hour=overnight, season=winter -> low_demand`
    - ***support:*** about 5.4/
    - ***confidence:*** about 92.7%, during winter overninght hours are low demand hours
    - ***lift:*** 3.68

- so low demand rules are connected to overnight hours, especially during winter or bad weather

This confirms our previous findings from CP1 and CP2.

4.3.2.1 Separated high demand vs low demand rules

- more readable result interpretation

```
high_rules = assoc_rule_table[assoc_rule_table["consequents"] ==
         frozenset({"high_demand"})].copy()
low_rules = assoc_rule_table[assoc_rule_table["consequents"] ==
         frozenset({"low_demand"})].copy()

print("Top rules for HIGH demand:")
display(high_rules.head(10))

print("Top rules for LOW demand:")
display(low_rules.head(10))
```

```
Top rules for HIGH demand:
```

	antecedents	consequents	support	c
267	frozenset({temp=high, hour=evening_commute})	frozenset({high_demand})	0.063007	0
488	frozenset({season=summer, hour=evening_commute})	frozenset({high_demand})	0.037344	0
527	frozenset({season=spring, hour=evening_commute})	frozenset({high_demand})	0.032798	0
517	frozenset({hum=medium, hour=evening_commute})	frozenset({high_demand})	0.034294	0
150	frozenset({weathersit=Clear/Few clouds, hour=e...	frozenset({high_demand})	0.082859	0
157	frozenset({hour=evening_commute, day_type=work...	frozenset({high_demand})	0.080902	0
261	frozenset({hum=low, hour=evening_commute})	frozenset({high_demand})	0.063467	0
461	frozenset({hour=evening_commute, windspeed=med...	frozenset({high_demand})	0.039761	0
566	frozenset({hour=evening_commute, season=fall})	frozenset({high_demand})	0.027792	0
502	frozenset({hour=evening_commute, temp=medium})	frozenset({high_demand})	0.035618	0

```
Top rules for LOW demand:
```

	antecedents	consequents	support	co
331	frozenset({hour=overnight, season=winter})	frozenset({low_demand})	0.053685	0.9
58	frozenset({hour=overnight, day_type=workingday})	frozenset({low_demand})	0.153173	0.9

	antecedents	consequents	support	co
134	frozenset({hour=overnight, temp=low})	frozenset({low_demand})	0.085793	0.8
411	frozenset({hour=overnight, windspeed=high})	frozenset({low_demand})	0.044997	0.8
109	frozenset({hour=overnight, hum=high})	frozenset({low_demand})	0.104897	0.8
340	frozenset({weathersit=Mist/Cloudy, hour=overni...	frozenset({low_demand})	0.052937	0.8
346	frozenset({season=spring, hour=overnight})	frozenset({low_demand})	0.051787	0.8
21	frozenset({hour=overnight})	frozenset({low_demand})	0.200414	0.8
126	frozenset({hour=overnight, windspeed=low})	frozenset({low_demand})	0.090512	0.8
198	frozenset({hour=overnight, temp=medium})	frozenset({low_demand})	0.071926	0.8

4.4 How patterns explain or challenge CP1/CP2 findings

The association rules mining results confirm and expand on the key findings from Checkpoint 1 and Checkpoint 2.

K-Means identified 4 regimes, with highest demand commute and low demand off-peak

- Association rules feature `hour=evening_commute` in high demand rules (29 out of 36 rules), with confidence 64.8% that evening commute hours are high demand. This directly validates commute driven regime found in checkpoint 1, while low demand off-peak is also confirmed with `hour=overnight` in low demand rules, with 81.5% confidence.
- Pattern mining also shows that commute hours, while being a high impact alone, with combination with weather/season conditions resulting rules, for example `hour=evening_commute, temp=high -> high_demand` (confidence 89.4%) show that warm evenings create higher demand than commute hours alone (lift=3.57 vs. lift=2.59).

DBSCAN identified high-demand regime and medium-high regime

- The apriori results align with DBSCAN's regime structure. DBSCAN's high demand regime (avg 823 rentals, evening commute) is precisely where pattern rules cluster. Most high demand rules include evening hours with specific weather conditions.
- In checkpoint 1 we found that DBSCAN flagged 5.06% of data as noise. Pattern mining shows this noise may consist of rare condition combinations (e.g., evening commute + very low temperature + bad weather) that do not form strong association patterns, as they are absent form the top 36 rules.

Anomaly Detection (5.55% outliers) detected globally (extreme? -> high cnt, but rare feature combinations) demand

- Association rules, by design, filter rare events. The `min_support=0.02` threshold excludes itemsets appearing in <2% of records. Therefore, pattern mining captures normal, repeating behaviors and so anomalies are precisely what rules exclude. Checkpoint 1's 5.55% anomalies are the counterpart to pattern mining's frequent

itemsets. Statistical outliers like unusually high `cnt` combined with rare feature combinations fall outside association rule support thresholds.

Louvain communities identified commute peaks, weekend leisure, night off-peak, and transitional hours

- Louvain's community structure maps onto association rules:
  - **Commute peaks** (RQ6 finding): Evening commute rules dominate the high demand patterns.
  - **Weekend leisure** (Louvain group): Pattern rules reveal that `day_type=weekend` appears in some high demand rules, though less frequently than working days. This confirms Louvain finding, that weekend leisure is a separate, smaller community.
  - **Night off-peak** (Louvain group): Pattern rules for low demand are dominated by `hour=overnight`, confirming the Louvain night community.
  - **Transitional/midday hours**: Midday rules appear less clearly in both high and low demand lists, matching the "transitional" community role.
- Checkpoint 2's graph provided visual clusters, but pattern mining showed the strength and conditional relationships within each community. For example, Louvain found an evening commute community, but only pattern mining reveals that this community's demand is 3.57x more likely when temperature is high.

Many outliers found in checkpoint 1 actually belonged to meaningful graph communities

- Rules like `hour=evening_commute, temp=high -> high_demand` (support 6.3%) show that certain "outlier looking" high demand hours actually follow strong patterns once the full context (time + weather + season) is considered. **These are not true anomalies as they are structured behaviors within communities.**
- What Checkpoint 1 may have flagged as individual outliers are actually cluster specific normal variations. Pattern mining makes this explicit, for example a row with high demand in evening commute + high temperature is not an outlier, but a frequent pattern.

Additional context to previous perspectives

- Checkpoint 1/Checkpoint 2 divides hourly records into discrete clusters/communities based on feature similarity or temporal structure. Pattern Mining identifies conditional relationships across features, without rigid cluster assignment. So pattern mining shows exact conditions that points within cluster/communities follow.
  - Example: Clustering says evening hours form a community. Pattern mining says evening hours + high temperature predict high demand with 89% confidence.

Conclusion: Pattern mining confirms Checkpoint 1/Checkpoint 2 findings and provides quantitative evidence for the regimes already discovered. The top association rules confirm that commute hours, weather, and season are the main demand drivers. Pattern mining's contribution is explicitaly stating conditional combinations and their predictive strength (confidence, lift). This transforms clusters and communities of Checkpoint 1/Checkpoint 2 into actionable, exact rules suitable for forecasting or operational decision making. Anomalies excluded from rules represent genuine outliers, while patterns represent recurring behaviour.

4.5 Pattern-mining limitations

4.5.1 Discretization limitations

The association rule mining approach requires grouping continuous variables into categorical bins:

- **Temperature, humidity, windspeed**: Converted to `low`, `medium`, `high` using percentile based binning (33rd and 67th percentiles).
- **Hour**: Manually binned into 5 periods (overnight, morning_commute, midday, evening_commute, late_evening) based on Checkpoint 1/Checkpoint 2 findings.
- **Demand**: Categorized as `high_demand` (≥75th percentile) and `low_demand` (≤25th percentile), treating mid range demand as neutral.

Limitations:

- **Information loss**: Grouping variables into discrete categories, for example, a temperature of 15°C and 19°C into "medium", loses nuance. Same issue is with the choice of 3 equal frequency bins for temp/humidity/windspeed, which is convenient but arbitrary. Using different amount of bins, or different thresholds based on domain knowledge, would yield different rules and potentially different insights. The differences in magnitude are also lost, which could be an issue for things like operational planning.
- **Temporal continuity ignored**: The hour bins, treat the transition from 23:59 (late_evening) to 00:00 (overnight) as a completely different values, while hours within the same bin are treated as same value, which results in loss of time continuity.

4.5.2 Threshold dependency

The apriori algorithm depends on two user specified thresholds:

- **min_support = 0.02** (patterns appearing in ≥2% of 17,379 hourly records, i.e., ≥348 rows) where lower value would "find" more rules, but these could include noise or statistical anomalies. And higher value could throw away valid but less common patterns, potentionally hiding important rules.
- **min_confidence ≥ 0.55** (rules must hold in ≥55% of antecedent cases) where lower value would allow for weaker associations, and higher value would be overly restrictive and could leed to 0 rules being found.

This implies that the 36 rules presented are not universal truths but are dependant on the chosen thresholds. A different choice of tresholds, for example: `min_support = 0.015` and `min_confidence = 0.50`, may produce a different rule set. This means that optimal thresholds may differ for each use case and may require several iterations to find.

4.5.3 Summary of limitations

Pattern mining provides valuable rules but has constraints:

- Discretization trades continuity for clarity and can hide gradual relationships.
- Threshold selection is subjective, different choices yield different rule sets.
- While it is by desing, the method captures frequent, repeating patterns but is blind to rare anomalies and outliers.
- Confidence, support, and lift metrics don't take into account other variables and factors which may affect the result as they function independetly.

Despite these limitations, pattern mining complements Checkpoint 1/Checkpoint 2 findings by transforming qualitative clusters and communities into actionable rules suitable for forecasting and operational decision-making.

5. Final Synthesis and Reflection

5.1 How we addressed the Checkpoint 1 feedback

We reviewed the provided feedback from Checkpoint 1 and implemented it into our jupyter notebook.

- **Address the distance measure limitation: cyclical time (hour 23 near 0) is currently ignored by standard vector distances:**
  - In CP1, the model treated hour 23 and hour 0 as very different, even though they are next to each other in real world. This was a limitation of the vector-based method. We did not change CP1 later, but in CP2 we partly improved this by using a graph of (weekday, hour) nodes, which represents weekly time patterns more naturally.
- **How come is DBScan better in F1 and worse in purity?:**
  - In our results, K-Means has higher **purity** because it makes more clusters (`k = 4`), so the groups are smaller and thus cleaner. On the other hand, DBSCAN uses only a few dense clusters plus noise. So, its clusters are less pure overall. (especially **workingday**, **weather**, **season**, **hour_binned**, which increases `f_measure` value) In short, **purity focuses more on cleaner clusters**, while **f_measure** measures cluster to class matching.
  - But the difference in values of F1 and Purity between K-Means and DBScan isn't significantly high, the values are actually very similar and close to each other.
  - **The answear from the feedback:** Purity was higher for K-Means simply because it produced smaller, more numerous clusters compared to DBSCAN.
- **Explain better how you determine the high- and low-demand regimes? Is that in the data or did you provide labels on the clusters?:**
  - The labels **high-demand regime** and **low-demand regime** are not part of the original dataset. The clustering algorithms return only numeric cluster IDs, and we assigned descriptive names afterwards by inspecting each cluster profile, especially mean demand (`cnt`), highest hours, working-day ratio, and the balance between `casual` and `registered` users.

5.2 How we addressed the Checkpoint 2 feedback

We reviewed the provided feedback from Checkpoint 2 and implemented it into our jupyter notebook.

- **Finalize the synthesis between the rule-based findings (Module 3) and the regime clusters:**
  - In Checkpoint 2, our graph-based results already identified meaningful communities such as **commute peaks**, **weekend leisure periods**, **night/off-peak hours**, and **transitional daytime regimes**. However, the earlier version of the notebook did not yet connect these communities clearly enough to the rule-based findings from Module 3.
  - In the final notebook, we addressed this by adding an explicit comparison section between **association rules**, **vector-space clusters**, and **graph communities**. This synthesis shows that the methods are complementary rather than redundant:
    - **Clustering** groups similar hourly records in feature space,

        - **Graph mining** reveals how recurring weekly contexts are connected as communities,
        - **Pattern mining** explains which exact combinations of conditions characterize these groups.
    - We found that the strongest association rules confirm the same demand regimes that appeared in the graph and vector-space analyses:
        - **Evening commute hours** repeatedly appear in high-demand rules, which supports the commute-oriented clusters and Louvain communities.
        - **Overnight hours** dominate low-demand rules, confirming the off-peak/night regimes found earlier.
        - **Weekend-related leisure patterns** appear as distinct graph communities and are supported by rules involving day-type, weather, and casual usage.
    - The rule-based analysis also added something new: while clustering and community detection showed that certain regimes exist, association rules made the **conditional structure** of these regimes explicit. For example, they show not only that evening commute hours form a strong demand regime, but also that this regime becomes especially strong under favorable weather conditions.
    - This helped us refine our interpretation of the dataset: some observations that looked unusual in Checkpoint 1 are not necessarily random anomalies, but may belong to meaningful and repeated behavioral patterns once time and weather are considered together.
    - Overall, we used the Checkpoint 2 feedback to improve the project from a set of separate analyses into a **connected multi-representation study**, where vector-space methods, graph-based methods, and pattern mining all contribute to a more complete understanding of bike-sharing demand.

5.3 Final synthesis

Across the project, our understanding of the Bike Sharing dataset became more structured and more detailed.

At the beginning, the dataset mainly appeared as a collection of hourly measurements with weather, time, and user information. The exploratory analysis already showed some clear patterns, such as strong hourly demand cycles, differences between `casual` and `registered` users, and a general influence of weather conditions. However, at that stage, these were still mostly descriptive observations.

With the vector-space methods, we were able to identify more explicit structure in the data.
The clustering results showed that the hourly records can be grouped into recurring demand profiles, including strong commute related periods, weaker off-peak periods, and more leisure contexts. The outlier detection methods added another perspective by highlighting observations that differ highly from the general structure, although the overlap between methods also showed that anomalies depend on the chosen definition.

The graph-based analysis extended this view by shifting the focus from individual records to relationships between recurring weekly usage contexts.
Using the `(weekday, hour)` representation, we found that these contexts form meaningful communities rather than appearing as isolated points. This confirmed that bike-sharing demand is not only structured by feature similarity, but also by repeated behavioral regimes over the week.

The pattern-mining section complemented the previous methods in a different way. Instead of grouping observations, it identified combinations of conditions that frequently occur together. The association rules helped make the earlier findings more explicit by

showing which combinations of time, weather, and demand levels repeatedly characterize high-demand or low-demand situations.

Comparing the three perspectives gave the most useful overall picture:

- the vector-space methods identified demand profiles
- the graph-based methods showed how these profiles are connected as recurring weekly regimes
- the pattern-mining methods described which condition combinations are typical for these regimes

The methods were therefore not redundant, even when they pointed to similar conclusions.
They emphasized different aspects of the same dataset and helped validate each other. For example, commute related high-demand periods appeared in clustering, in graph communities, and in the strongest association rules. At the same time, each representation also had its own limitations. The clustering depended strongly on feature choice and scaling, the graph analysis depended on aggregation and similarity design, and the association rules required discretization, which simplified the data.

Overall, the project showed that bike-sharing demand is shaped by a combination of temporal structure, user type, and weather conditions.
The final result is not a single “best” model, but a more complete understanding of the dataset through several complementary data-mining views.

6. Declaration of use of Generative Artificial Intelligence (GAI)

- we used generative artificial intelligence (GAI) in this project
  - for inspiration in formulating text
  - for explanation of the lecture material and choice of theory method (discussion)
  - to understand the topic better
  - to aid us to work with libraries and algorithms for implementation
  - to suggest additional or missing talking/explanation points, limitations, metrics
  - to interpret the results/outputs from the code (not write directly, but explain)
- the models we used
  - Codex/GPT 5.4